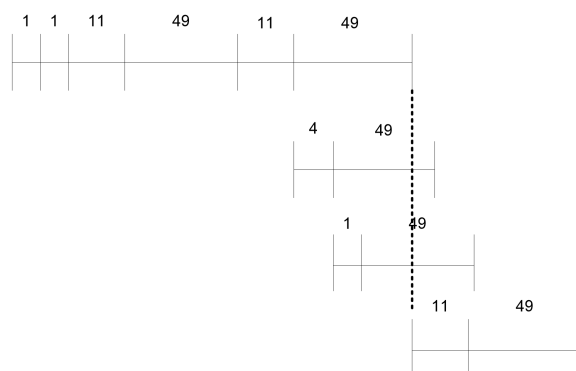


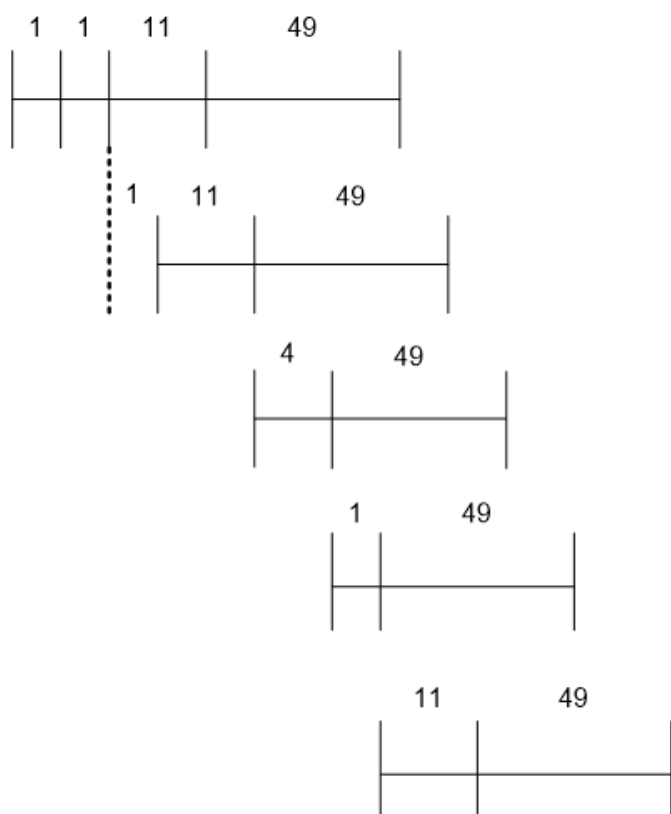
向量链接

Vector chaining: Data forwarding from one vector functional unit to another



182 cycles

单端口（所以两个VLD操作不能pipelined，VLD和ST操作也不能）



Chaining and 2 load ports, 1 store port in each bank
79 cycles 19X perf. improvement!

单端口存储体（Single-port Memory Bank）的带宽瓶颈，导致访存操作（读 / 读、读 / 写）无法并行，只能串行执行，因此无法流水线化。

数据超过VLEN的情况，需要改变VLEN——**vector stripmining**

What if # data elements > # elements in a vector register

Idea: **Break loops so that each iteration operates on # elements in a vector register**

E.g., 527 data elements, 64-element VREGs

8 iterations where VLEN = 64

1 iteration where VLEN = 15 (need to change value of VLEN)

What if vector data is not stored in a strided fashion in memory? (**irregular memory access to a vector**)

Idea: Use indirection to combine/pack elements into vector registers

Called **scatter/gather operations**

Gather (收集)

代码块

```
1  vector = gather(base_addr, indices[])
```

从不规则内存地址中读取多个元素，打包进一个向量寄存器。

Scatter (散射)

把向量寄存器里的数据写回到不规则内存地址。

代码块

```
1  scatter(base_addr, indices[], vector)
```

Want to vectorize loops with indirect accesses:

```
for (i=0; i<N; i++)  
    A[i] = B[i] + C[D[i]]
```

Indexed load instruction (*Gather*)

```
LV vD, rD          # Load indices in D vector  
LVI vC, rC, vD     # Load indirect from rC base  
LV vB, rB          # Load B vector  
ADDV.D vA,vB,vC    # Do add  
SV vA, rA          # Store result
```

Masking / Predication

代码块

```
1  for (i = 0; i < N; i++)
```

```
2      if (a[i] != 0) b[i] = a[i] * b[i];
```

Idea: **Masked operations**

VMASK register is a bit mask determining which data element should not be acted upon

代码块

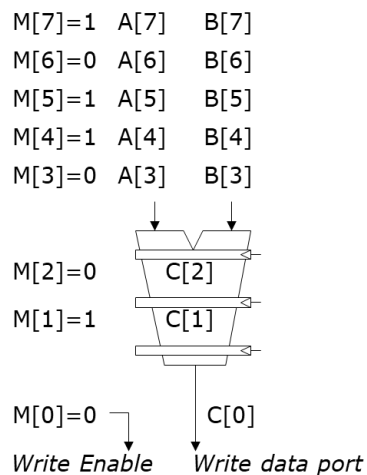
```
1  VLD V0 = A
2  VLD V1 = B
3  VMASK = (V0 != 0)
4  VMUL V1 = V0 * V1
5  VST B = V1
```

两种实现方式：

1. 全部计算，最后根据M决定是否回写
2. 扫描M，M=1，才计算回写（硬件要求高）

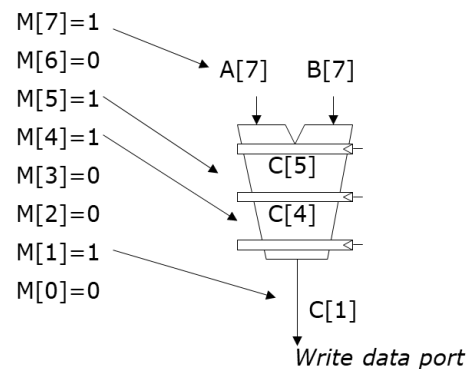
Simple Implementation

- execute all N operations, turn off result writeback according to mask



Density-Time Implementation

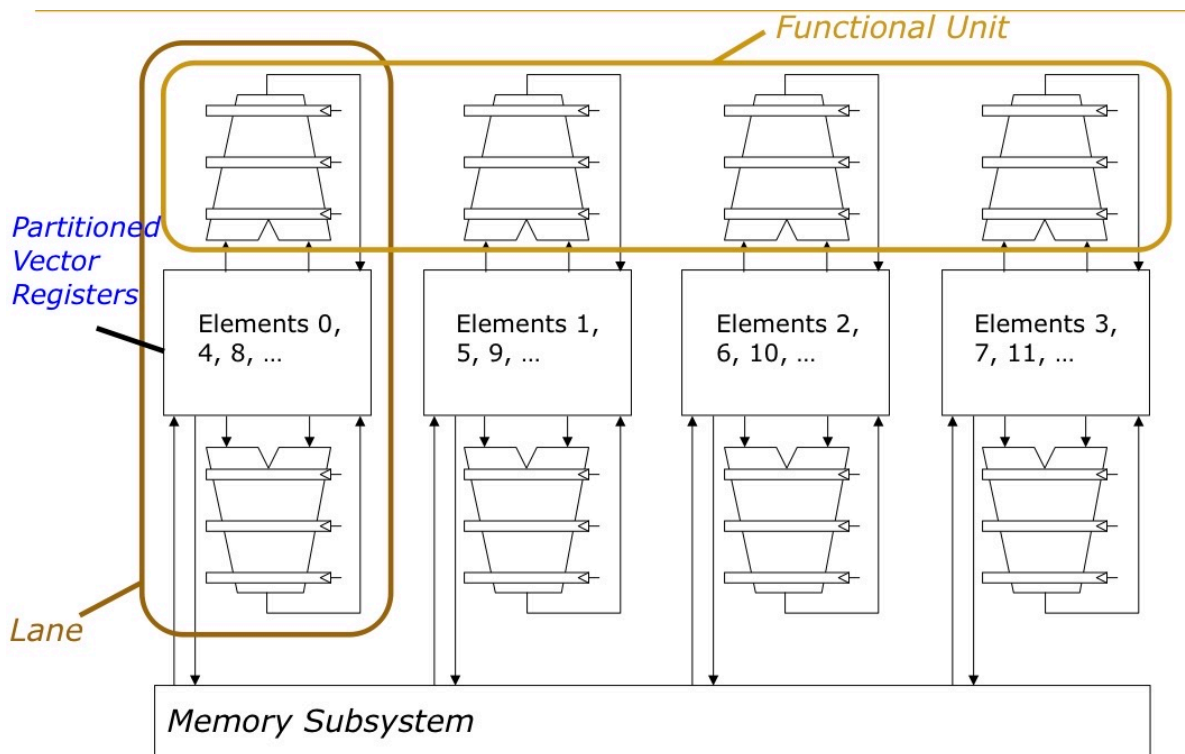
- scan mask vector and only execute elements with non-zero masks



行优先和列优先会导致stride不同，导致冲突

Lane 是向量功能单元的一个并行分片。在多 Lane 架构中，每个 Lane 包含以下组件：

- **向量寄存器的一部分**：例如，如果总共有 32 个元素，4 条 Lane，那么每条 Lane 负责存储和处理 8 个元素（如 Lane 0 处理元素 0, 4, 8...）。



- 私有的功能单元：每个 Lane 都有自己的加法器、乘法器和加载/存储单元。
- 内存子系统的接口：通过与内存系统的连接，不同 Lane 可以同时获取其负责的元素

SIMD ISA Extensions

carry (进位)：低位加法产生的进位，继续传给高位。

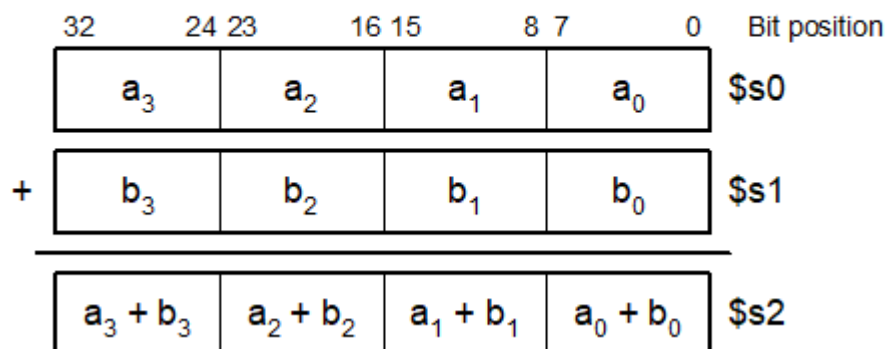
普通 scalar adder (标量加法器) 默认把整个字当成一个完整整数算，所以会发生 **carry propagation (进位传播)**。

packed data (打包数据)：把多个小元素装进一个宽寄存器。

packed arithmetic (打包算术)：对这些小元素并行做同一种运算。

packed SIMD 的关键要求是：**lane (通道)** 之间彼此独立，**不能跨 lane 传 carry**。所以 SIMD 不是“一个数算更快”，而是“多个小数一起算”。

```
padd8 $s2, $s0, $s1
```



Intel Pentium MMX Operations

Intel 当年没有把 CPU 做成完整的 **vector processor**（向量处理器：带可变向量长度/跨距控制的处理器），而是走了一条更保守的工程路线——在传统 ISA（Instruction Set Architecture，指令集架构）上加一组 **SIMD extension**（SIMD 扩展指令：一条指令同时处理多个数据元素）。**一个 64-bit（64 位）寄存器里塞多个小整数，一条指令同时算它们。**

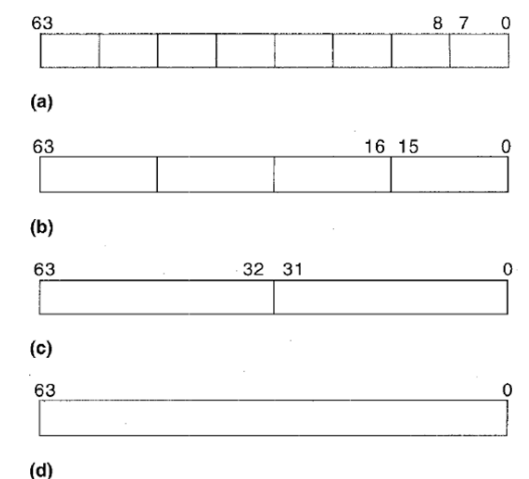


Figure 1. MMX technology data types: packed byte (a), packed word (b), packed doubleword (c), and quadword (d).

No VLEN register

Opcode determines data type:

8 8-bit bytes

4 16-bit words

2 32-bit doublewords

1 64-bit quadword

Stride is always equal to 1.

Intel: MMX → SSE → AVX → AVX2 → AVX-512

ARM: NEON (Advanced SIMD, 高级 SIMD)

Apple: 其 CPU 核心也大量依赖 SIMD datapath (SIMD 数据通路)

AI inference (AI 推理) 里的 low-precision (低精度) 整数算子，本质上也在复用这条思想

总结:

1. **MMX 是一种 SIMD ISA extension**，不是完整的 **vector processor**。
2. 它的核心机制是：一个 64-bit register 里放多个小元素，一条指令同时操作它们。
3. **Opcode determines data type** 意味着元素宽度由指令决定，不靠 **VLEN register**。
4. **Stride is always equal to 1** 说明它主要面向连续数据，灵活性远弱于传统 vector machine。
5. 真正收益来源是：减少指令数、提高前端利用率、提升规则小数据并行吞吐。
6. 最容易误解的点：MMX 不是“通用向量架构”，它只是“受限但便宜、能工业落地的 packed-SIMD”。

FGMT（细粒度多线程）

OoO（Out-of-Order，乱序执行）

在**同一个线程内部**找独立指令先执行。例如：

代码块

```
1    T0: I0, I1, I2, I3
```

如果 **I1** 卡住了，OoO 会看看 **I2/I3** 能不能先跑。

本质：在**一个线程内部**挖 ILP（Instruction-Level Parallelism，指令级并行：一个线程中多条独立指令可并行）

和FGMT的区别：FGMT

不在同一个线程里搞乱顺序，而是：**T0** 卡住了 直接切去 **T1**

本质：在**多个线程之间**挖 TLP（Thread-Level Parallelism，线程级并行：多个线程可以并发推进）

FGMT 中，每个周期通常只从一个线程取一条指令进入 pipeline。（几乎每个周期切线程隐藏等待）

- **线程（thread）** 是独立执行上下文，包含 **PC** 和寄存器状态。
- **指令（instruction）** 是该线程当前 PC 所指向的下一步操作。
- FGMT 中，**每个周期通常只从一个线程取一条指令进入 pipeline。**
- 不同线程的指令会 **interleave（交错）**，但每个线程内部顺序仍保持不变。
- “pipeline 深度是 4，再回到 Thread 0 时上一条基本走远了”的意思是：**用别的线程把同线程相邻指令在时间上拉开。**

代码块

```
1    T0: I0, I1
2    T1: J0, J1
3    T2: K0, K1
```

假设 **I1** 依赖 **I0** 的结果，流水线深度 3。

代码块

```
1    Cycle 0: I0
2    Cycle 1: J0
3    Cycle 2: K0
4    Cycle 3: I1    <- 这时 I0 结果更可能已经准备好了
```

- FGMT 解决的是 **latency hiding**，不是减少单线程指令依赖本身。

GPU

区分：

概念	关注层次	核心含义	典型对象
SISD	硬件分类	单指令、单数据	标量处理器
SIMD	硬件分类	单指令、多数据	SSE/AVX、阵列机、向量机底层
MIMD	硬件分类	多指令、多数据	多核 CPU
MISD	硬件分类	多指令、单数据	很少见，基本可忽略
SPMD	编程模型	同一程序处理不同数据	CUDA/OpenCL/并行 for
SIMT	编程+执行桥梁	线程编程，硬件成组执行	GPU

顺序执行 (SISD)：依赖超标量或乱序执行处理器（如传统的CPU）在硬件底层动态展开循环，找出可以并行执行的指令。

数据并行 (SIMD)：也就是传统向量处理器的方式。程序员或编译器直接生成一条向量指令（比如 `VADD`），一条指令下去，硬件同时处理整个数组的数据。

多线程 (MIMD / SPMD)：为每一次循环迭代分配一个独立的线程。所有的线程都在执行同一段程序，但各自处理不同的数据（这就是 **SPMD - 单程序多数据** 模型）。

- GPU底层本质上是SIMD（单指令多数据流）引擎，但其编程并不是直接使用SIMD指令，而是通过**线程 (Threads)** 来实现。

SIMD 是 ISA 层暴露的

方案	优点	缺点	
传统 SIMD	控制精确	编程复杂	程序员写 vector instruction → ISA 暴露 SIMD
GPU SIMT	编程简单	divergence 问题	→ 程序员写很多 thread → 硬件自动 grouping 成 warp → warp 在 SIMD pipeline 上执行

- GPU采用的是SPMD（单程序多数据）**编程模型**：每个线程执行相同的代码，但操作不同的数据片段。在硬件执行层面，GPU使用的是**SIMT（单指令多线程）模型**

It is **programmed using threads** (SPMD programming model)

Each thread executes the same code but operates a different piece of data

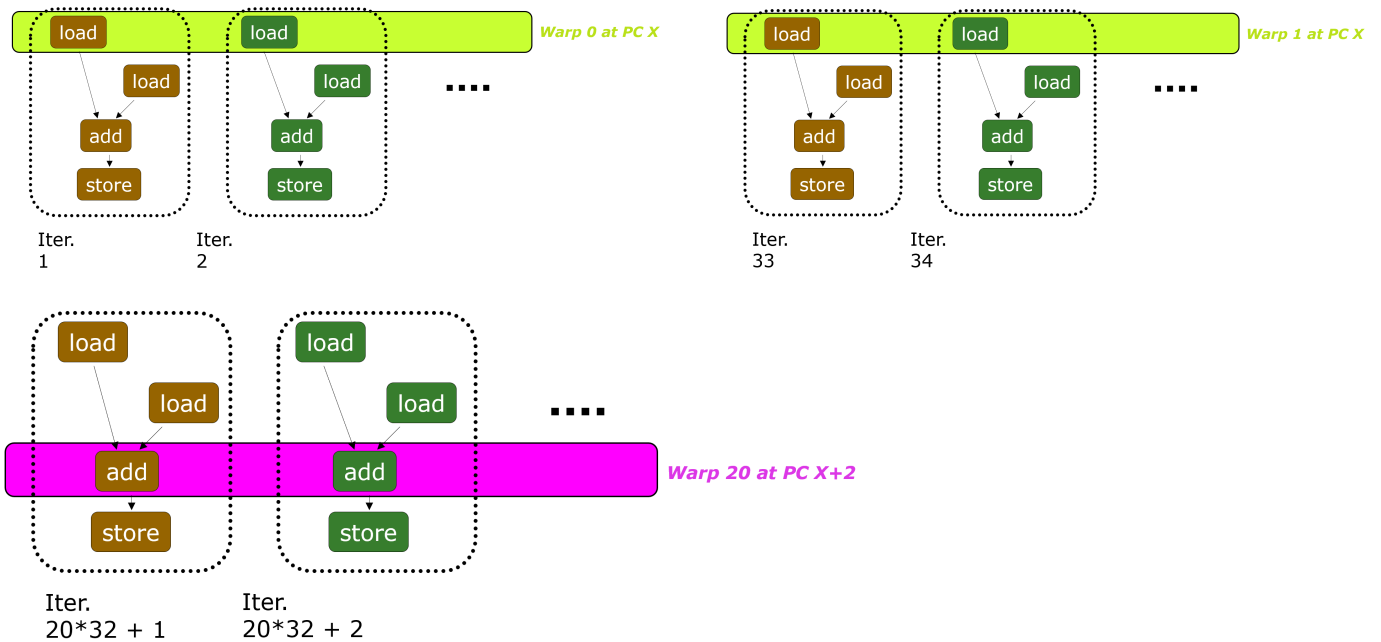
Each thread has its own context

- 在SIMT模型中，执行相同指令的多个标量线程会被硬件动态分组为一个**Warp（线程束）** 虽然编程模型里最小单位叫 thread，但硬件里真正经常一起执行的是 **warp**。

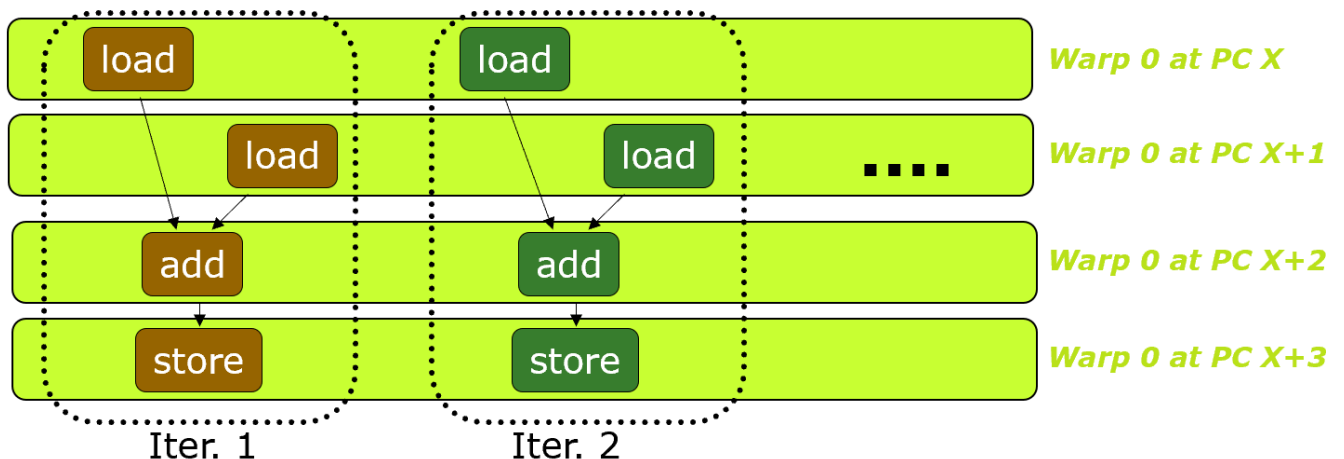
- GPU通过**Warp级别的细粒度多线程（FGMT）**来隐藏长延迟操作（如内存访问延迟）

代码块

1	Cycle	C0	C1	C2	C3	C4	C5
2	-----						
3	Warp 0	LD	stall	stall	stall	ADD	ST
4	Warp 1	-	LD	stall	stall	stall	ADD
5	Warp 2	-	-	ADD	ST	-	-
6	-----						
7	Pipeline选中	W0	W1	W2	W2	W0	W1



- 在流水线中交错执行不同的Warp：当一个Warp因为等待内存数据而停顿时，硬件可以切换到另一个就绪的Warp继续执行。
- 所有线程的寄存器值都保留在寄存器文件中，这样切换Warp时没有额外的开销，从而能够容忍极长的延迟。



- 同一时刻，warp 内活跃线程执行同一条指令。

- 图里 Warp 0 at PC X / X+1 / X+2 / X+3 表示的是同一个 warp 随时间推进到不同 PC
- 真正会让同一 warp 内线程“不一致”的，是 **branch divergence**（分支发散），不是这张图本身。

在 NVIDIA GPU 中，通常：1 warp=32 threads

也就是说：

- 一个 block 里如果有 256 个线程
- 那它会被分成 8 个 warp

因为：256/32=8

软件概念	硬件含义
thread	逻辑线程，实际常按 warp 成组执行
block	被放到一个 SM 上执行的线程组
grid	一次 kernel 启动产生的所有 block
warp	硬件调度和锁步执行的重要单位
shared memory	SM 内 block 共享的片上存储
global memory	GPU 显存，容量大延迟高

4. 当同一个Warp内的线程在条件分支处走向不同的执行路径时，就会发生**分支发散**。这会导致部分 SIMD计算通道被闲置，降低SIMD的利用率。

Warp 对程序员通常是隐藏的；程序员看到的是 CPU 端串行代码 + GPU 端 kernel + block（线程块）/thread。

代码块

```

1  Serial Code (host) //主机侧
2
3  KernelA<<< nBlk, nThr >>>(args);//
4
5  Serial Code (host)
6
7  KernelB<<< nBlk, nThr >>>(args);

```

<<< nBlk, nThr >>>:

- nBlk = number of blocks（线程块数量）
- nThr = number of threads per block（每个 block 内线程数）

CPU code

```
代码块
1  for (i = 0; i < 10000; ++i) {
2      C[i] = A[i] + B[i];
3  }
```

cuda code

代码块

```
1  __global__ void KernelFunction(...) {
2      int tid = blockDim.x * blockIdx.x + threadIdx.x;
3      int varA = a[tid];
4      int varB = b[tid];
5      C[tid] = varA + varB;
6  }
```

二维循环

cpu code

代码块

```
1  for (int i = 0; i < N; ++i)
2      for (int j = 0; j < N; ++j) {
3          index = i + j*N;
4          c[index] = a[index] + b[index];
5      }
```

cuda code

代码块

```
1  __global__ add_matrix(float *a, float *b, float *c, int N) {
2      int i = blockIdx.x * blockDim.x + threadIdx.x;
3      int j = blockIdx.y * blockDim.y + threadIdx.y;
4      index = i + j*N;
5      if (i < N && j < N)
6          c[index] = a[index] + b[index];
7  }
```

主机端

代码块

```

1  dim3 dimBlock(blocksize, blocksize);
2  dim3 dimGrid(N/dimBlock.x, N/dimBlock.y);
3  add_matrix<<<dimGrid, dimBlock>>>(a, b, c, N);

```

注意：硬件同时能跑的线程数

这个由 GPU 资源限制：

- SM (Streaming Multiprocessor, 流式多处理器) 数量
- 每个 SM 的 warp 容量
- 寄存器文件大小
- shared memory (共享内存) 占用
- block 资源限制

处理iteration数大于总线程数的情况

最主流的方法：grid-stride loop (网格跨距循环：每个线程按固定步长处理多个元素)

代码块

```

1  __global__ void kernel(float* A, float* B, float* C, int N) {
2      int tid = blockIdx.x * blockDim.x + threadIdx.x;
3      int stride = blockDim.x * gridDim.x;
4
5      for (int i = tid; i < N; i += stride) {
6          C[i] = A[i] + B[i];
7      }
8  }

```

举例：假设：

- N = 100
- 只 launch 了 16 个线程
- 那么 stride = 16

代码块

```

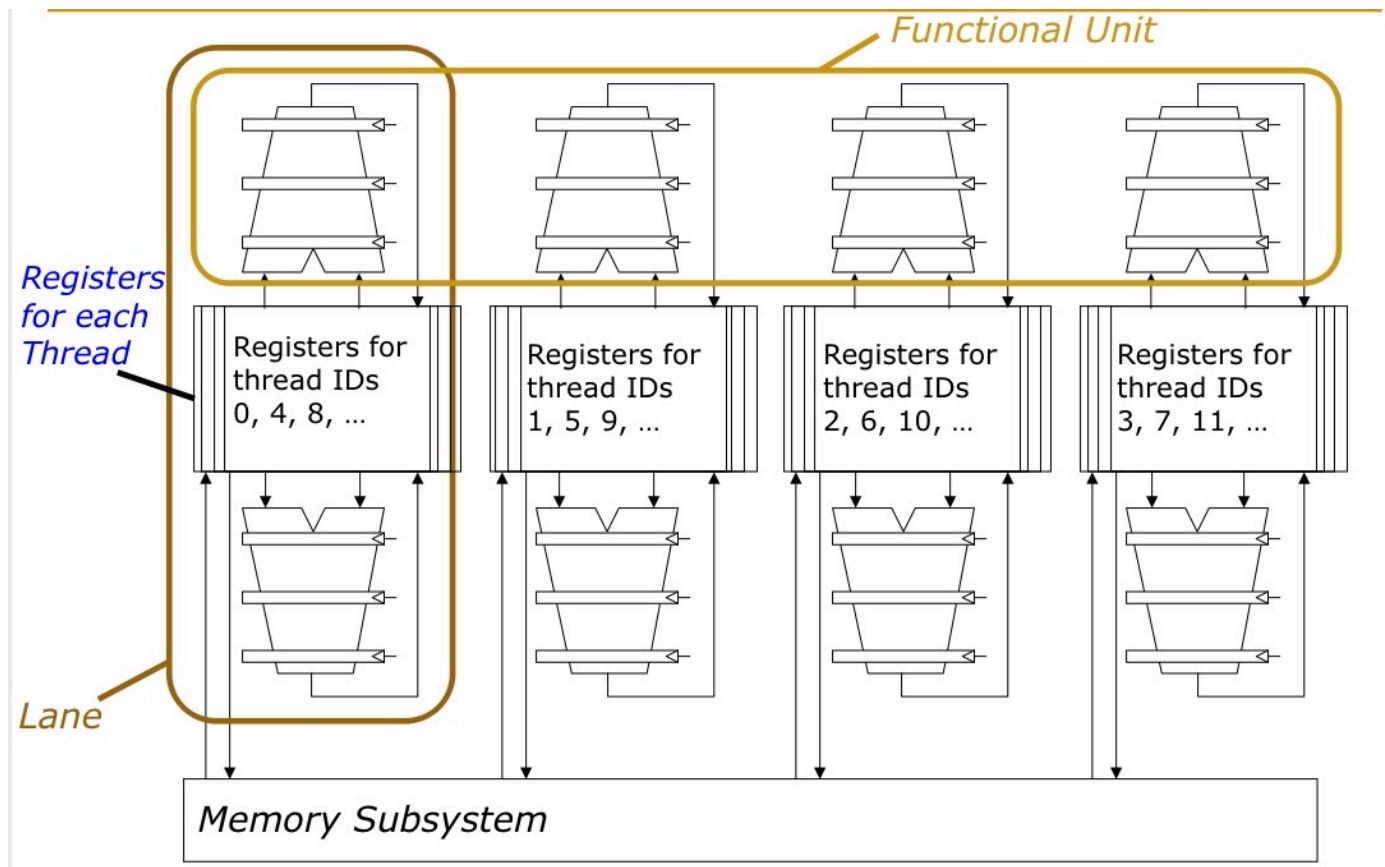
1  thread 0 -> 0, 16, 32, 48, 64, 80, 96
2  thread 1 -> 1, 17, 33, 49, 65, 81, 97
3  thread 2 -> 2, 18, 34, 50, 66, 82, 98
4  ...
5  thread 15 -> 15, 31, 47, 63, 79, 95

```

实际执行的系统图

lane（执行通道）= SIMD datapath 里的一个“并行执行位置”

warp 内的 threads 会被映射到一组 lane 上执行。



代码块

```
1  你写的 kernel
2  -----
3  C[tid] = A[tid] + B[tid];
4
5  软件层展开为 threads
6  -----
7  T0: C[0] = A[0] + B[0]
8  T1: C[1] = A[1] + B[1]
9  T2: C[2] = A[2] + B[2]
10 T3: C[3] = A[3] + B[3]
11 T4: C[4] = A[4] + B[4]
12 T5: C[5] = A[5] + B[5]
13 ...
14
15 硬件层分组成 warps
16  -----
17 Warp 0 = {T0, T1, T2, T3}
18 Warp 1 = {T4, T5, T6, T7}
```



```

19  ...
20
21  某一时刻 Warp 0 映射到 lanes
22  -----
23  L0 <- T0
24  L1 <- T1
25  L2 <- T2
26  L3 <- T3
27
28  下一时刻 Warp 1 映射到 lanes
29  -----
30  L0 <- T4
31  L1 <- T5
32  L2 <- T6
33  L3 <- T7

```

SPMD branch divergence

warp 内线程走不同路径 → SIMD 利用率下降

warp 内部分 lane 被 mask 掉导致利用率下降

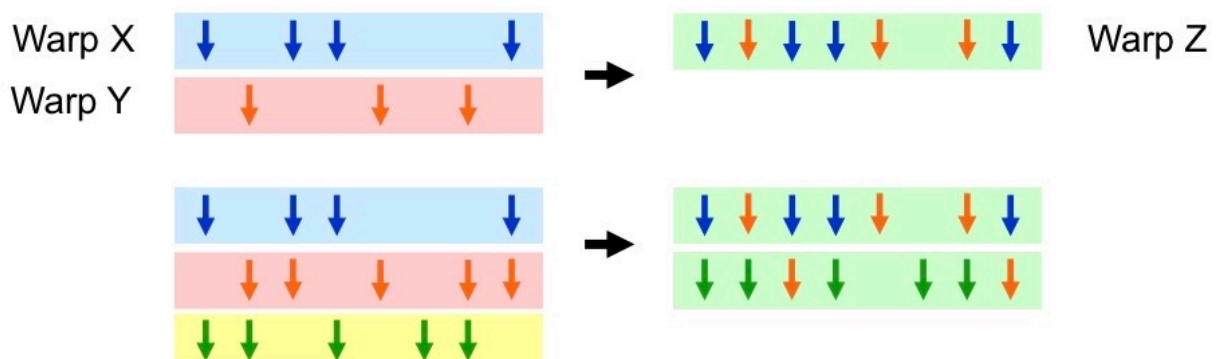
divergence 本质：warp 内线程路径不同 → lane 空转

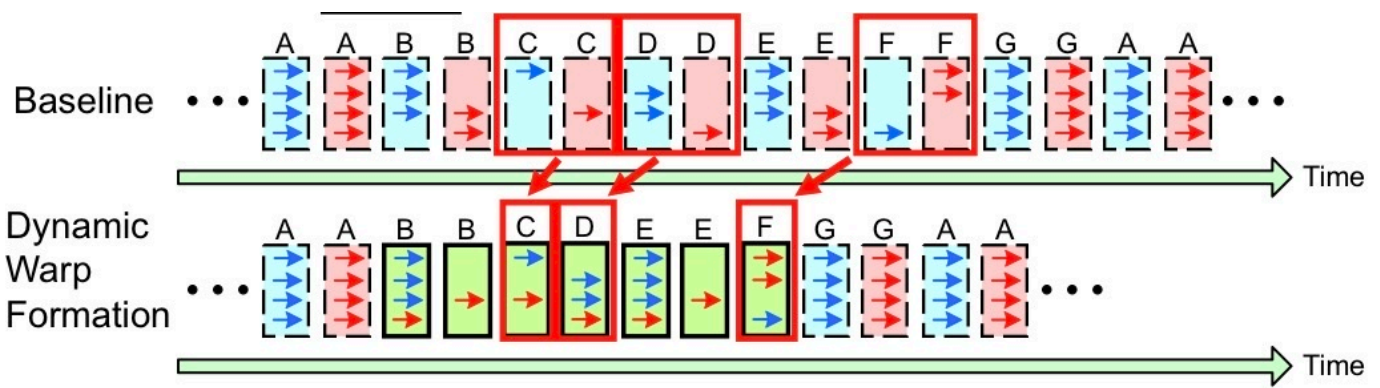
Two main reasons for GPU resources be underutilized

- Branch divergence
- Long latency operations

Branch divergence解决方法：

1. Dynamic Warp Formation（动态 warp 重组）：运行时重新拼 warp





2. Hardware constraint (硬件约束)

3. Large Warp + Two-Level Scheduling: 工程妥协方案

先group后warp

4. Large Warp 微架构

Narasiman et al., “[Improving GPU Performance via Large Warps and Two-Level Warp Scheduling](#),” MICRO 2011.

- 思路1: 扩大Warp规模

用“更大的Warp”替代传统固定大小的Warp（如32线程），静态地将更多线程纳入同一Warp中。

- 思路2: 动态拆分子Warp (Sub-warps)

在译码阶段，硬件会根据活跃线程掩码（Active Mask），将大Warp动态拆分为多个子Warp

Nvidia V100

代码块

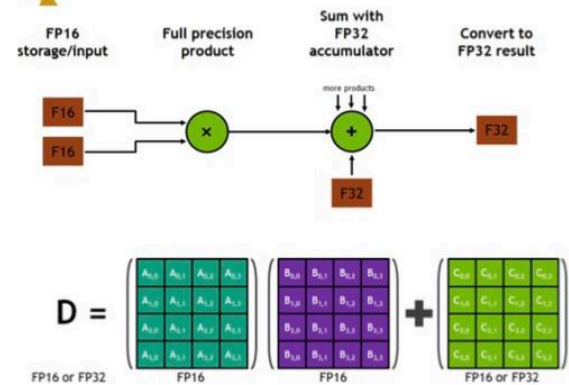
```

1  V100 GPU
2  |—— 80 个 SM / GPU cores
3  |   |—— warp scheduler
4  |   |—— SIMD lanes / CUDA cores
5  |   |—— register file
6  |   |—— shared memory / L1
7  |   |—— Tensor Cores
8  |—— HBM2 memory system
  
```

NVIDIA V100 Core



15.7 TFLOPS Single Precision
7.8 TFLOPS Double Precision
125 TFLOPS for Deep Learning (Tensor cores)



<https://devblogs.nvidia.com/inside-volta/>

112

一个 SM = 多个 warp scheduler + 大量执行单元 + 巨大寄存器文件 + Tensor Core 专用矩阵单元

- Tensor Core 是专门做 MMA: $D = A \times B + C$ 的硬件。
- GEMM 是大矩阵乘，由很多小块 MMA 组成。
- 它比普通 CUDA core 快，是因为数据流固定、控制简单、低精度密集乘加。
- 它是 SIMD-like: 一条指令触发大量同类计算，但计算对象是 matrix tile，不是简单 vector lane。

Tensor Core 算力对比 (TFLOPS)



精度格式对比

FP32 (单精度) 4字节, 完整精度, 内存占用高

BF16 (脑浮点) 2字节, 大动态范围, 精度适中

FP16 (半精度) 2字节, 动态范围小, 易溢出

FP8 (8位浮点) 1字节, 极小内存占用, 硬件优化

训练性能对比

模型类型	GPU型号	精度格式	每秒样本数	显存占用	收敛精度
ResNet-50	V100	FP16	120	24 GB	76.1%
ResNet-50	A100	BF16	300	18 GB	76.2%
ResNet-50	H100	FP8	690	9 GB	76.2%
BERT-Large	V100	FP16	30	28 GB	89.5%
BERT-Large	A100	BF16	90	21 GB	89.7%
BERT-Large	H100	FP8	270	10.5 GB	89.7%
GPT-3 175B	V100	FP16	0.1	30 GB	12.5
GPT-3 175B	A100	BF16	0.4	22 GB	12.3
GPT-3 175B	H100	FP8	1.6	11 GB	12.3

<https://www.doubao.com/building/render/a3fe511e417f8?a=>

GPU层级

代码块

1 Programmer View

```

2  _____
3  Kernel
4  └─ Grid
5      └─ Block
6          └─ Thread
7
8  Hardware View
9  _____
10 GPU
11 └─ SM 0 (Streaming Multiprocessor, 流式多处理器)
12     └─ Warp Scheduler
13     └─ Register File
14     └─ FP32 / INT lanes
15     └─ Tensor Cores
16     └─ LD/ST Units
17     └─ Shared Memory / L1
18     └─ Active Warps
19
20 └─ SM 1
21     └─ ...
22
23 └─ L2 Cache
24 └─ Memory Controllers
25 └─ HBM / GDDR
26
27
28 Execution Flow
29 _____
30 Threads
31 → grouped into warps
32 → scheduled by warp scheduler
33 → executed on lanes / tensor cores
34 → data from registers/shared memory/L2/HBM

```

代码块

```

1  Grid
2  └─ Block 0
3      └─ Thread 0
4      └─ Thread 1
5      └─ ...
6      └─ Thread 255
7  └─ Block 1
8      └─ Thread 0
9      └─ Thread 1
10     └─ ...

```

```
11  └─ Block 2
12  └─ Block 3
```

同一个 block 会被放到同一个 SM 上执行，所以它们可以访问该 SM 上的 shared memory，也能做 block 内同步。不同 block 可能被调度到不同 SM，甚至执行时刻都不同，因此默认不能依赖同一块 shared memory，也不能做普通的 block 间同步。

CUDA kernel :

- 普通 C/C++ 函数默认在 CPU 上跑
- `global` 函数是在 GPU 上跑

`threadIdx.x` 当前线程在自己 block 内的编号

`blockDim.x` 一个 block 内总共有多少线程

`blockIdx.x` 当前 block 是第几个 block

代码块

```
1  global void vecAdd(float* A, float* B, float* C, int N) {
2      int i = blockIdx.x * blockDim.x + threadIdx.x;
3      if (i < N) {
4          C[i] = A[i] + B[i];
5      }
6  }
```

启动kernel

代码块

```
1  int threadsPerBlock = 256;
2  int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
3  vecAdd<<<blocksPerGrid, threadsPerBlock>>>(A, B, C, N);
```

warp divergence

代码块

```
1  __global__ void foo(int* A) {
2      int i = blockIdx.x * blockDim.x + threadIdx.x;
3      if (i % 2 == 0)
4          A[i] = A[i] + 1;
5      else
6          A[i] = A[i] - 1;
```

```
7 }
```

偶数线程走 `A[i] = A[i] + 1`

奇数线程走 `A[i] = A[i] - 1`

memory coalescing

扩展

针对数据中心（Data Center）领域，NVIDIA 的架构演进主要围绕 计算单元（Compute Unit）、显存（Memory）和 互联（Interconnect）三大方向。

演进方向：

代码块

- 1 图形渲染 GPU
- 2 → 通用并行计算 GPU
- 3 → 深度学习训练 GPU
- 4 → Transformer/LLM 专用数据中心 GPU
- 5 → rack-scale AI supercomputer

架构	代表产品	核心关键词
Volta	V100	Tensor Core 起点
Turing	RTX 20	RT Core + Tensor Core 消费级落地
Ampere	A100 / RTX 30	TF32、MIG、稀疏 Tensor Core
Hopper	H100	Transformer Engine、FP8、DPX、NVLink 4
Ada Lovelace	RTX 40	图形/生成式 AI 消费级优化
Blackwell	B200 / GB200	FP4、2nd-gen Transformer Engine、双 die、NVLink 5、rack-scale

1. 在transformer的模型中向量投影、QKV计算、attention、线性层，存在大量矩阵乘加（GEMM：通用矩阵乘法），**Volta**针对这种矩阵乘法引入了tensor core（张量核心：专门做矩阵乘加的硬件单元）。 $D = A \times B + C$ ：MMA（Matrix Multiply-Accumulate，矩阵乘加）
2. **Turing：图形 GPU 和 AI 单元融合，代表：RTX 20 系列**

Turing 把三类单元放到消费级 GPU 里：

- CUDA Core：普通 shading / compute
- Tensor Core：AI 推理、DLSS

- RT Core (Ray Tracing Core, 光线追踪核心) : 加速 BVH traversal 和 ray-triangle intersection

3. Ampere: A100 是数据中心 AI GPU 的成熟形态

A100 的 Tensor Core 是第三代, 支持: FP16、BF16、TF32、FP64 Tensor Core、sparsity acceleration (稀疏加速)

A100 支持 **MIG (Multi-Instance GPU, 多实例 GPU)** , 可以把一张 GPU 切成最多 7 个 GPU instance。NVIDIA 官方 A100 页面也强调 A100 可被 partition 成 7 个 GPU instances, 用来适配数据中心多租户负载。

4. Hopper: 为 Transformer/LLM 重新定向的 GPU

它支持 FP8, 并在 FP8 和 16-bit 精度之间动态选择, 以适配 Transformer 训练/推理。H100 被 NVIDIA 定位为面向 trillion-parameter AI 的 GPU, 公开资料也强调其 Transformer Engine、FP8 以及相比前代大幅提升的训练/推理能力

趋势: NVLink 4 / NVSwitch: 从单卡到集群

5. Blackwell: 从 GPU 架构变成 rack-scale (机架级架构) AI 架构

代表: B200 / GB200 / GB200 NVL72

NVLink 5: 互联成为核心计算资源

Lecture 17 cuda programming

Function prototypes

```
float serialFunction(...);  
__global__ void kernel(...);
```

main()

- 1) **Allocate memory** space on the device – cudaMalloc(&d_in, bytes);
- 2) Transfer data from **host to device** – cudaMemcpy(d_in, h_in, ...);
- 3) Execution configuration setup: #blocks and #threads
- 4) **Kernel call** – kernel<<<execution configuration>>>(args...);
- 5) Transfer results from **device to host** – cudaMemcpy(h_out, d_out, ...);

Kernel – __global__ void kernel(type args,...)

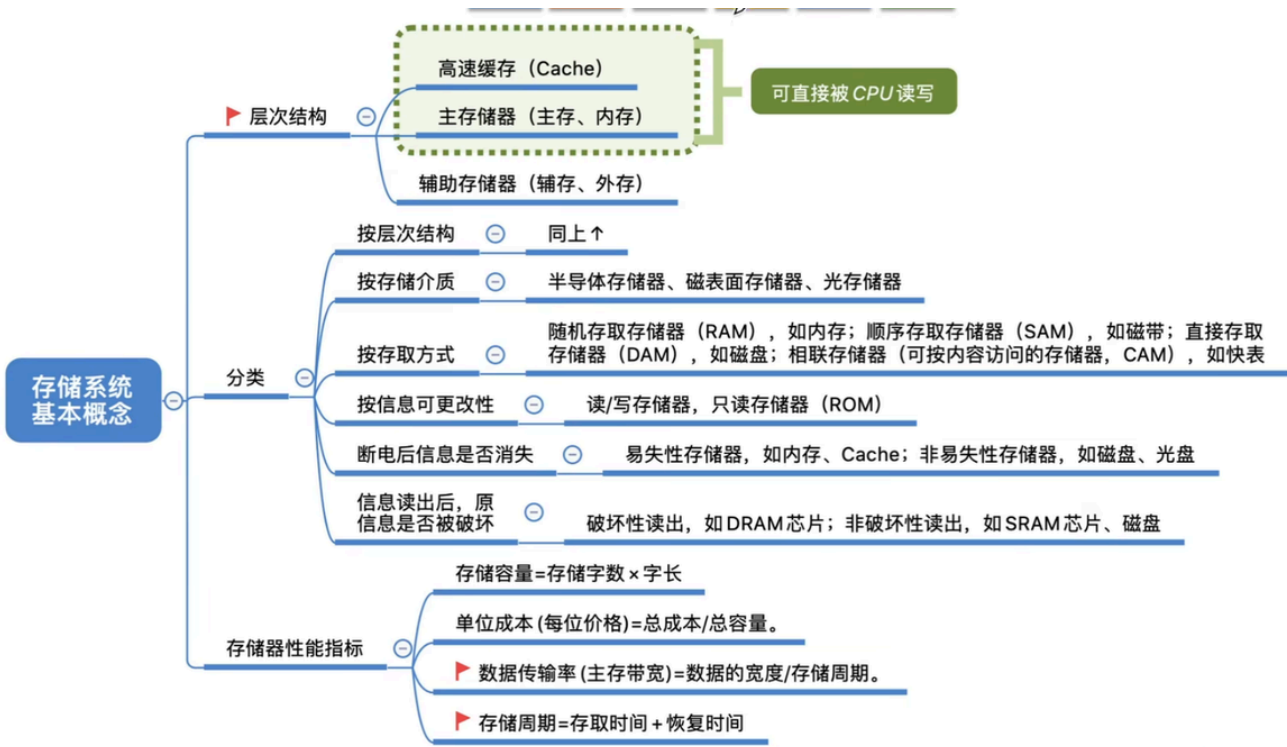
Automatic variables transparently assigned to **registers**

Shared memory: shared

Intra-block **synchronization**: __syncthreads();

论文阅读

存储系统



CPU

寄存器 L1/L2/L3 Cache 核心 (cores)



memory bottleneck: CPU 算一条指令很快, 但如果每一步都等 DRAM, 整个处理器就会经常“站着等数据”。

内存通道 (CPU↔DRAM): CPU 可以直接用 `load/store` 像读写数组一样读写 DRAM。

PCIe (CPU↔外设): 更像“发消息/下命令/搬运数据” (I/O 通信), 不是天然的“像内存一样读写”。

CPU 访问内存, 最关键的 2 个词: `load/store`

CPU 的世界里最朴素的动作就是两类:

- **load**: 从某个地址把数据读到寄存器
- **store**: 把寄存器的数据写回某个地址

总线/互连

CPU 要访问 DRAM: 需要一条路 (内部互连 + 内存控制器)。

CPU 要访问 GPU/网卡/SSD: 需要一条更通用的路——这就是 **PCIe** (你可以把它理解成“高速外设总线/高速公路”)。

PCIe 的核心特性

它主要是给 I/O 用的 (网卡、SSD、GPU 这种设备)。

它不是天然的“内存总线”: 它更像“发请求-收响应”的外设通信, 而不是“CPU 以 `load/store` 像访问内存一样访问它”。

CXL

CXL.io (最像传统 PCIe)

- 配置设备、控制命令、管理
- 你可以粗暴理解: “老 PCIe 能干的活, 它都能干”

CXL.mem (CPU 像访问内存一样访问设备上的内存)

CXL.mem: CPU 用 `load/store` 访问“设备上的内存”。

CPU (核+cache) -- CXL.mem (load/store) --> 设备上的内存 (比如一块扩展 DRAM)

所以如果你插了一个 **Type 3 内存扩展卡**, CPU 看到的效果像“多了一坨内存”。

CXL.cache（设备缓存主机内存，而且要和 CPU 保持一致）

CXL.cache：设备把主机内存的数据缓存到自己那边用，但必须遵守一致性。

主机 DRAM <---(被缓存的数据)--- 设备上的cache/加速器



服务器、cpu和内存

Host和VM

host基本就是“跑虚拟机的物理服务器节点”（更准确一点：一个物理服务器/或一个 socket 域，由 hypervisor 管着）。

Host = 一台物理机（或一个物理机上的一个 NUMA 域） + 它上面的 hypervisor。

host 里面有：

- CPU 核心（很多）
- 本地 DRAM（socket-local 内存）
- hypervisor（例如 Azure 的生产 hypervisor）
- 可以连接 CXL 设备/EMC（池内存）

VM 是什么？

VM（虚拟机）是跑在 host 上的“虚拟出来的一台机器”。每个 VM 有：

- vCPU（虚拟 CPU 核数）
- vMem（分配的内存）
- 虚拟磁盘、虚拟网卡等

一个 host 能同时跑很多个 VM（就像一台物理机上开很多进程/容器，只是隔离更强）。

每个 VM 在同一时刻只跑在一个 host 上（除非做 live migration，把它搬到另一个 host；搬完后还是在一个 host 上）。

云的 **VM Scheduler（调度器）** 负责决定：新来的 VM 放在哪台 host 上。

延迟（latency）

一次访问“从发请求到拿到第一个字节”要多久（ns、 μ s）。

带宽（bandwidth）

单位时间内“能搬多少数据”（GB/s）。

延迟敏感：当访问之间“有依赖”，你只能一下一下等。

- $\rightarrow$ 这时总时间 $\approx$ 访问次数 $\times$ 每次延迟。

带宽敏感：当访问“可以并行/流水线”，你能同时挂很多未完成请求。

- 这时总时间 $\approx$ 总数据量 $\div$ 可持续带宽。

服务器结构：

代码块

1 一台服务器 = 1 个 节点 (Node)

2

3

4 | 节点 (Node 1)

5

6 | Socket 0

```
7 | | 物理 CPU 0
```

8		32 物理核
---	--	--------

9 | 1 个 NUMA node (默认)

10

11

12 | _____ Socket 1

13 | | 物理 CPU 1

14 | 32 物理核

```
15 | | 1 个 NUMA node (默认)
```

16

17

18

1. 1 台服务器 = 1 个节点 (Node)
2. 1 个节点 = 2 个 Socket
3. 1 个 Socket = 1 颗物理 CPU，不可拆分
4. 1 个 Socket 默认 = 1 个 NUMA Node

页 (page) 是什么？——内存管理的“最小搬运箱”

内存很大，操作系统不可能用“一个字节一个字节”去管理。

所以它把内存切成固定大小的小块，像快递箱：

- 一般是 **4KB** (也有 2MB/1GB 的大页)

cache line是什么?

CPU 缓存 (cache) 不是按页工作的，它按更小的单位：**cache line (缓存行)**。

CPU 从内存拿数据时，一般不是只拿你要的那 4 个字节，而是把“包含它的那一整条 64B 小块”搬进缓存。

一致性过程

读 (Read)

1. 节点 A 读地址 X（对应某条 cache line），本地 cache 没有 → miss
2. A 向 home 请求这条 line
3. home 看目录：如果没人持有“修改版”，就从内存（DRAM/CXL mem）取，发给 A
4. home 在目录里登记：A 现在持有这条 line 的共享副本（Shared）

写（Write）

1. 节点 A 要写地址 X，但自己只有 Shared 状态 → 需要升级权限
2. A 向 home 请求“写权限/独占”（Exclusive/Modified）
3. home 查目录：谁持有这条 line 的副本？（比如节点 B、C）
4. home 精确通知 B、C：你们把这条 line 失效（invalidate）
5. 等 B、C 确认失效后，home 授予 A 独占写权限
6. A 写入，必要时将来回写到内存

虚拟地址 vs 物理地址

程序里用的地址（比如 `p[0]`）是 **虚拟地址**。

真正的 DRAM 位置是 **物理地址**。

操作系统维护一个“翻译表”（页表 page table）：

虚拟页号 (VPN) → 物理页号 (PPN)

CPU 每次访问内存，都要先做地址翻译（由 MMU + TLB 加速完成）。

缺页（page fault） 是什么？

当程序访问一个虚拟地址时，CPU 去查页表，发现：

- 这个虚拟页 **还没有映射到任何物理页**
- 或者这页被标记成“不在内存里”（可能在磁盘上）

CPU 就会触发一个异常，交给操作系统处理，这就叫 **缺页（page fault）**。

本地内存读取路径：

CPU Core → (L1/L2/L3 cache) → 内存控制器(MC)
→ PHY → DDR 走线 → DIMM(内存条) → DRAM颗粒

Socket（物理概念）

一块“插在主板上的 CPU 大芯片”，物理 CPU 插槽 → **不可拆**

DIMM（内存条）

就是你插在主板上的那根“内存条”。它上面焊了很多 DRAM 芯片。

PHY（Physical Layer 收发器）

PHY 是“把 0/1 变成电信号、把电信号还原成 0/1”的硬件模块。

它负责非常底层的事：时钟、采样、均衡、训练……

一句话：**PHY 是“电信号那一层的发动机+变速箱”。**

DDR（内存接口协议/电气规范）

DDR 不是“内存条”，它是 CPU 内存控制器和 DIMM 之间通信的**标准**（DDR4/DDR5）。

你可以理解为“内存这条路的交通规则 + 信号时序”。

DDR（Double Data Rate）= 一种 DRAM 内存接口/标准

- 电脑/服务器上那种“内存条”，本质就是 **DRAM**。
- DDR 是 DRAM 的一代代接口标准：DDR4、DDR5……
- “Double Data Rate”的意思是：同一个时钟周期的上升沿+下降沿都传数据（更快）。

内存通道（DDR channel）

CPU/控制器连内存条的“并行高速车道”

关键点：**一条通道通常只能挂有限数量的 DIMM**，而且每条通道的带宽有限。

DDR 的带宽：由“通道数 × 每通道速率 × 总线宽度”决定

典型 DDR 通道的数据总线宽度是 **64-bit**（= 8 bytes）一拍能传 8 字节（实际还有 ECC 等，但先抓主干）。

DDR 的“Double Data Rate”意味着：

每个时钟周期的上升沿/下降沿都传数据，所以有效传输次数翻倍。

DDR5 的具体速率（比如 4800/5600/6400 MT/s），抓住结构就行：

$\text{Bandwidth per channel} \approx \text{DataRate} \times 8 \text{ bytes}$

PCIe / CXL 的带宽：由“lane 数 × 每 lane 速率”决定

lane：链接宽度：x8、x16 等（表示 8 条 lane 或 16 条 lane 并行）。

PCIe 5.0 每 lane 的有效带宽大概在 ~4 GB/s（单向）这个量级

NUMA Node（逻辑概念）

NUMA（Non-Uniform Memory Access）

每个 Socket + 它“旁边的内存” = 一个 NUMA Node

NUMA Node 0: CPU0 + DRAM0

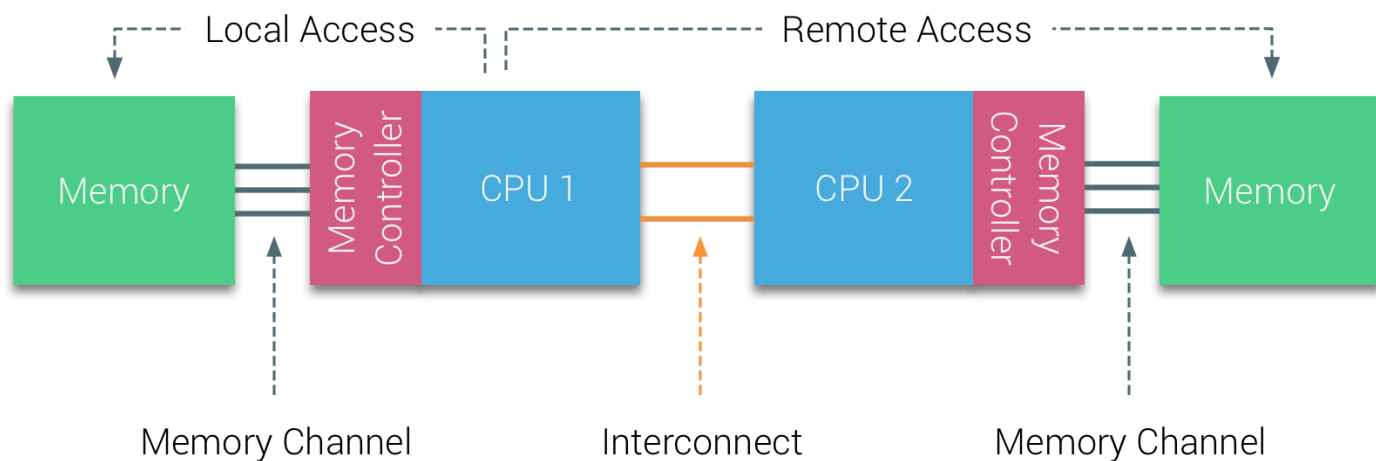
NUMA Node 1: CPU1 + DRAM1

本地内存和远程内存

CPU0 想访问 DRAM0 本地访问（快）CPU0 → MC0 → DDR → DIMM0

CPU0 想访问 DRAM1 远程访问（慢）

CPU0 → 互连(UPI/QPI/InfinityFabric 等) → CPU1 → MC1 → DDR → DIMM1

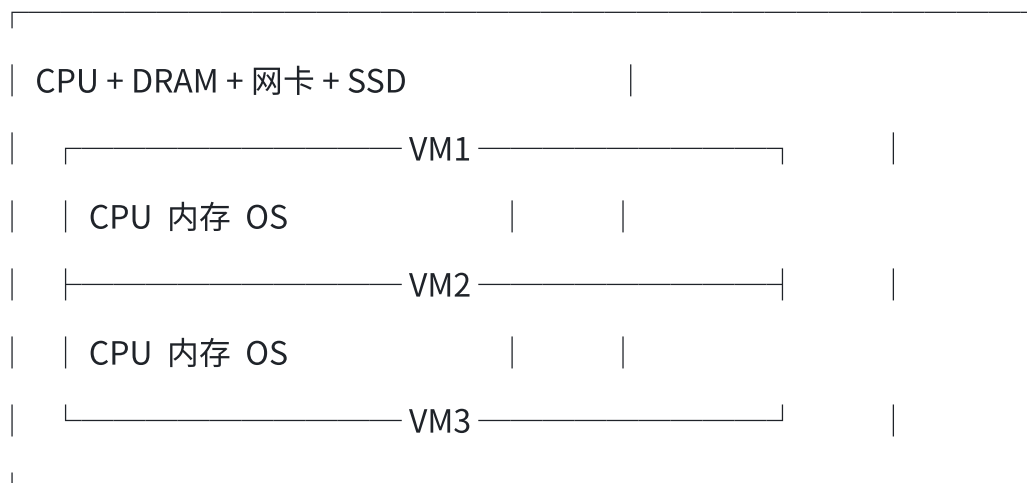


VM (虚拟机)

VM = 一台“假的电脑”

- 有自己的 CPU
- 有自己的内存
- 有自己的操作系统
- 但 其实是跑在别人机器上的

真实硬件



Hypervisor (虚拟化管理者)

- 给 VM 分 CPU
- 给 VM 分内存
- 控制 VM 不能乱碰别人的资源

内存浪费 (Memory Stranding)

云里 VM 调度受 CPU 和内存双资源约束。经常出现：某台机器 CPU 快满了，但内存还剩一堆；或反过来。剩下的那部分资源因为“配不上另一种资源”就浪费了，这叫 **stranding**。

一台 2-socket 服务器:

NUMA Node 0: CPU 32核 + 内存 256GB

NUMA Node 1: CPU 32核 + 内存 256GB

来了一个 VM:

VM1: 32 核 CPU + 64GB 内存

云调度器会这样放：

NUMA Node 0:

CPU 用満 (32/32)

内存只用 64GB / 256GB

NUMA Node 1 呢?

- CPU：没人用
- 内存：也不能随便拆给 VM1

纵轴：Latency (ns) —— 为“读一次要等多久”（越低越爽）。

图里写了 “100% local memory read”：测试场景是 只读本地内存（不跨 NUMA，不跨机器）。

SR-IOV (Single Root I/O Virtualization)

一块网卡/SSD 控制器可以切出很多个“虚拟小设备”（VF），每个 VM 分一个，性能接近直通。

DDA (Direct Device Assignment, 设备直通)

把整块设备（或某个功能）几乎“直接给 VM 用”，绕过很多虚拟化软件路径。

Qos (Quality of Service) = 服务质量。在云里通常指：

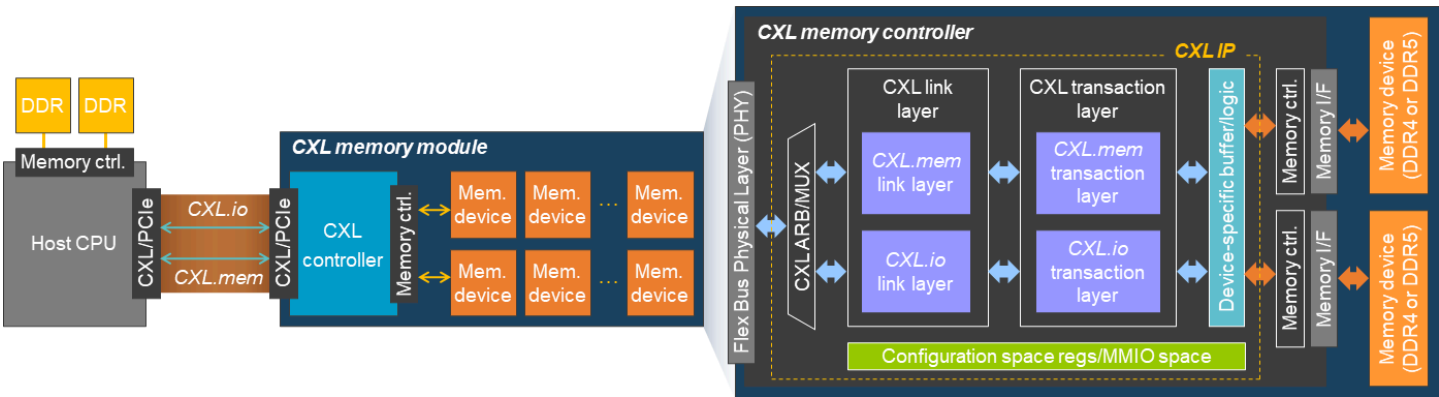
- 延迟 (p99、p999) 不能超过某个阈值
- 吞吐不能低于某个阈值
- SLO/SLA 是否满足

池化内存 (CXL) 相比本地 DRAM 多 70-90ns 甚至更多延迟，对某些 VM 会导致：

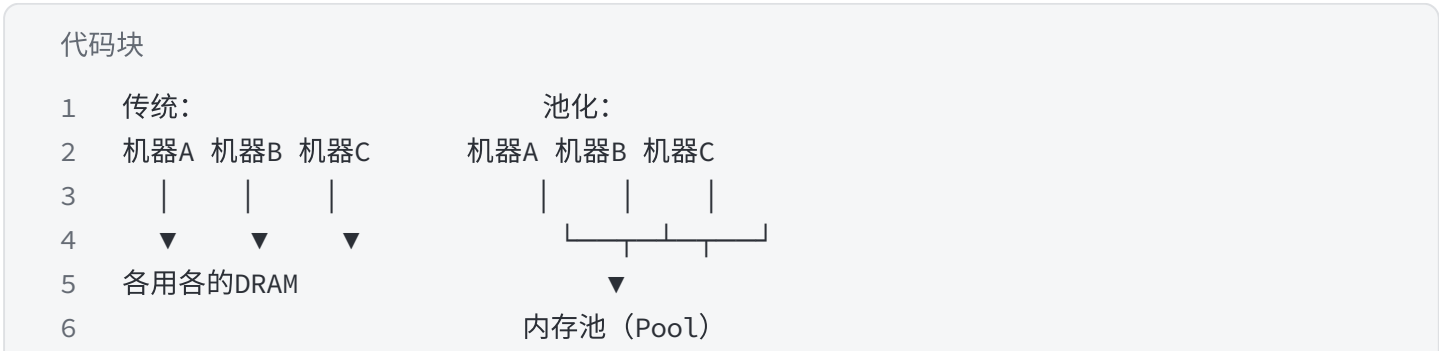
- 请求延迟变大
- tail latency (尾延迟) 变差
- 于是 QoS 不达标

所以 Pond 才需要监控 QoS：发现不行就救火 (reconfiguration)。

CXL内存结构



池化内存：把“很多机器上零碎剩下的内存” 汇成一个共享池



池化内存的“直观好处”

- 某台机器 CPU 满但内存闲 → 内存可以放进池里给别的机器用
- 机房总体需要买的 DRAM 变少 → 省钱

代价是什么？

池里的内存离 CPU 更远（通过互连/交换机/CXL链路），**访问更慢**。

所以池化不是“白捡”，关键是：**哪些数据可以慢一点？**

QEMU

Pond: CXL-Based Memory Pooling Systems for Cloud Platforms

Pond 接近同 NUMA 节点的性能 + 降低硬件成本。关键是依赖 **CXL**：它支持对池化内存进行 **cacheable 的 load/store**（像访问内存一样的读写），延迟是纳秒级；这被称为“game changer”，因为它允许内存**仍然保持静态预分配**，只是**物理位置在共享池里**。但即便如此，CXL 访问延迟仍高于同 NUMA 节点，所以 Pond 要做系统支持来减轻影响。

池化内存（memory pooling）：把原本“每台服务器自己那几条 DIMM 的 DRAM”切出一部分，做成一个**多个主机可共享/可调配**的“内存池”。内存不再严格跟着某台机器走。

Problem:

1. 现实问题：云平台cloud perfomance要求 hardware cost要低

云厂商常常有很苛刻的 SLA/QoS（比如 P99 延迟、吞吐、稳定性）。论文后面会用 PDM/TP 这类指标量化。

2. 为什么云需要池化内存：提高内存的utility
3. 为什么池化难（延迟 + 虚拟化约束）
4. **opaque** = 云厂商看不见/不该看 VM 内部。

所以你不能指望“让用户改程序/改 OS 来适配池化内存”。Pond 必须在平台侧解决。

5. 服务器成本大部分来自DRAM，而memory stranding（内存搁浅）是主要浪费来源：当一台服务器的核心都被租走了，但还剩下内存容量，却没法再出租。原因：**是因为 CPU 没了（占用满了），没法再开一个新 VM 来用这部分内存。**

论文还存在的问题：

1. CXL 池内存有额外延迟，工作负载敏感性差异极大，必须做策略。

Unfortunately, existing pooling systems also have **microsecond access latencies** and require page faults or changes to the VM guest [50, 58].所以 Pond 要解决的是：**谁用池？用多少？什么时候不用？**

云最想要的是“同 NUMA 节点的内存访问效果”，因为它稳定、延迟低。

云默认要 **同 NUMA 本地内存 + pin** 来保证性能/加速——>但 DRAM 很贵，而且会出现 **stranding**（CPU 满、内存剩）所以想做 **内存池化** 来提高利用率但池化会引入延迟/约束，于是 Pond 用 **小池子 + ML 预测本地/池内存分配** 来平衡

Background

公有云工作负载是虚拟化的。为了高性能低开销，hypervisor 做尽量少的内存管理，并依赖虚拟化加速器提升 I/O 性能；常见加速器包括 **DDA** 和 **SR-IOV**。而且 AWS/Azure 默认开启加速网络。关键结论：**虚拟化加速要求静态预分配（pin）VM 的整个地址空间。**

1. 云为了性能和 SR-IOV/DDA，加速默认开，所以 **VM 内存常常需要静态预分配+pin**。
2. Pond 选择 CXL 的核心理由是：CXL.mem 让设备内存也能走 **load/store 的 cacheline 路径**，并且 **不靠 page fault/DMA**，还能通过页表虚拟化与加速兼容。
3. CXL Request Flow：CXL 访问慢的“成本”很大一部分在 **CXL port + 协议层 + PHY + 线**，所以池子做大（更多 retimer/switch）延迟会继续涨（这会在后面的 Figure 7 量化）。

CXL Request Flow

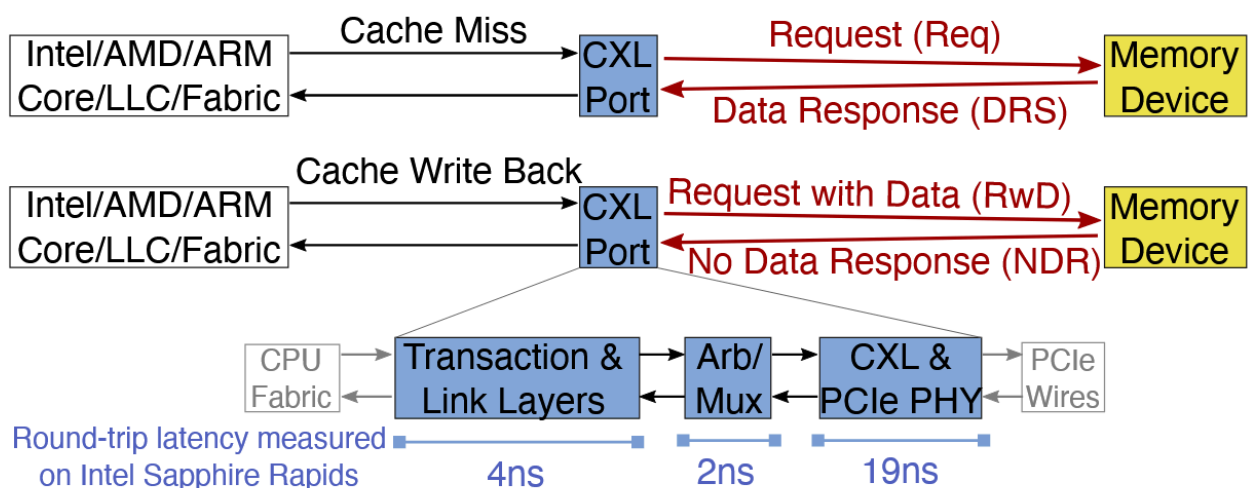


Figure 1: CXL Request Flow (§2). CPU cache misses and write-backs to addresses mapped to CXL devices are translated to requests on a CXL port by the HDM decoder. Intel measures the round-trip port latency to be 25ns.

CPU 发生 LLC miss（缓存没找到那条数据）

地址被判断属于 CXL 设备（靠 **HDM decoder**：Host-Managed Device Memory 解码器）

生成 CXL 请求，从 **CXL Port** 发出去

经过：

- **Transaction & Link layers（事务层/链路层）**：打包请求、序列号、重传等协议活
- **Arb/Mux（仲裁/复用）**：多个请求抢端口资源时排队
- **CXL & PCIe PHY**：把比特变成电信号在“线”上传（PHY=物理层收发器）
- **PCIe wires**：就是走线/线缆

到达 Memory Device（CXL Type-3 设备）

池化内存概念图

Host A (CPU/VMs) —┐

Host B (CPU/VMs) —┴── CXL Fabric / Switch — Memory Pool Device (大DRAM)

Host C (CPU/VMs) —┐

物理上部署一个（或一组）CXL 内存池设备，通过 CXL 交换/链路连接多个主机（socket/服务器）。各主机通过 CXL.mem 把池设备中的一部分内存映射进自己的地址空间，从而为 VM 提供“可借用的远端内存”，实现共享池化。

理解：

在每台机器上，最多能放几个这样的 VM

- CPU：64 核 → 可以放 **4 个 VM**（ $4 \times 16 = 64$ 核）
- 内存： $4 \times 64\text{GB} = \mathbf{256\text{GB}}$

这个刚好满，看起来没浪费？那我们稍微换一个更现实的情况：VM 往往不会刚好整除，而且会有多种规格混用。

我们换成更真实的组合：

每台机器最后被调度成：

- CPU 已经用满（64/64）
- 但只装下了 3 个 VM（因为某些 VM 要 24 核/32 核之类，导致碎片化）
- 这 3 个 VM 一共只用了 $3 \times 64 = \mathbf{192\text{GB}}$

于是每台机器剩余内存：

- $256\text{GB} - 192\text{GB} = \mathbf{64\text{GB}}$

但 CPU 已经没核了，没法再开一个新 VM（因为新 VM 至少要 CPU 核）。

所以这 64GB 就变成：

搁浅内存（stranded memory）：有但卖不出去

整个集群搁浅量：

- $8 \text{ 台} \times 64\text{GB} = \mathbf{512\text{GB}}$ （半个 TB！）

你已经为它付了钱（买了 DRAM），但它在业务上没产生价值。

引入 CXL 池化 + Pond 的分配策略，会发生什么？

现在云厂商说：

我们不想每台机器都插满 256GB（因为会搁浅），我们改成：

- 每台机器本地只配 **192GB**
- 然后机架里放一个 **CXL 内存池盒子**，总共 **256GB**

此时总 DRAM 采购量是：

- 本地： $8 \times 192\text{GB} = \mathbf{1536\text{GB}}$
- 池：256GB
- 合计：**1792GB**

对比原来 2048GB，你现在少买了：

- $2048 - 1792 = \mathbf{256\text{GB}}$ （直接省了 12.5% 的 DRAM）

但 VM 不会爆，原因在 Pond 的关键策略：**热的在本地、冷的去池里**。

对某台机器：

- 它上面 3 个 VM 需要总内存 192GB
- **刚好全在本地**，完全不需要池内存 → 性能几乎不变

对另一些机器：

- 假设某台机器刚好需要 224GB（比本地 192GB 多 32GB）
- Pond 就把这 32GB 从池里借过来
- 而且尽量让这 32GB 对应的内容是“冷/不常访问”的内存（比如缓存、预留但不 touch 的页）

于是：

- 你不需要每台机器都准备“最坏情况的内存”
- 池作为“缓冲垫”覆盖偏差
- 访问池内存确实慢一点，但因为是冷数据，所以整体性能影响很小（这就是 Pond 后面用 ML 去预测/控制的）

问题分析：

1. 机器越忙（CPU 越满），越容易出现“CPU 不够开新 VM，但内存还剩一大截”的尴尬。

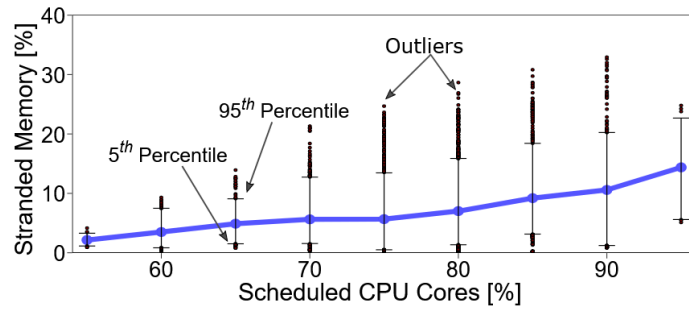


Figure 2: Memory stranding (§3.1). Stranding increases significantly as more CPU cores are scheduled. Error bars indicate the 5th and 95th percentiles (outliers in dots).

6% of memory is stranded in the median snapshot (blue line) where 75% of CPU cores are scheduled for VMs. The median grows to over 10% when ~85% of CPU cores are allocated to VMs.

2. 延迟slowdown以及workload Sensitivity to Memory Latency做的是cxl延迟模拟，而不是真正的cxl

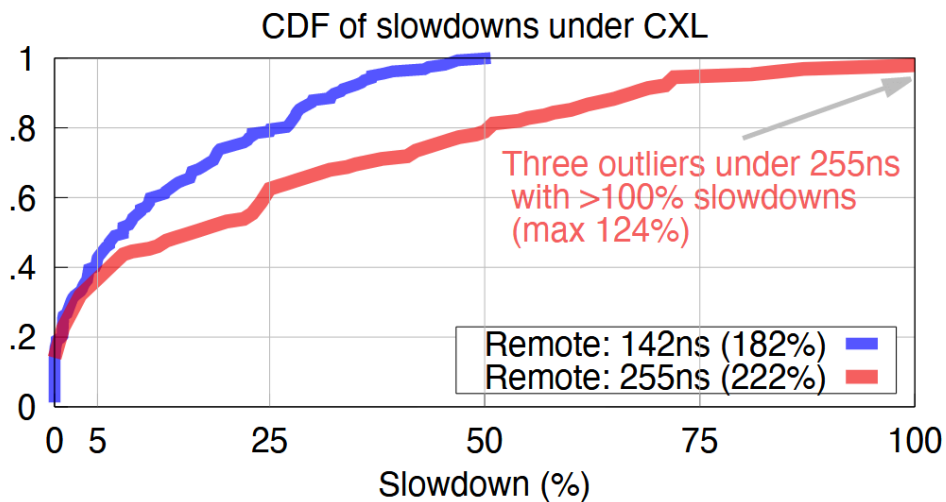


Figure 5: CDF of slowdowns (§3.3). Higher remote latency (red) only slightly affects the head of the distribution (workloads with less than 5% slowdown). The body and tail of the distribution see significantly higher slowdowns.

硬件层面

host 是“使用内存的一方”（CPU 在这一侧）

device 是“提供池内存的一方”（EMC/内存池设备在另一侧）

服务器 + 网卡/SSD/GPU

在 PCIe 里很典型：

- 服务器主板上的 CPU + 芯片组/Root Complex = **host**
- 插在 PCIe 插槽里的网卡（NIC） = **device**
- 插在 PCIe 插槽里的 NVMe SSD = **device**

- 插在 PCIe 插槽里的 GPU = **device**

机架里有：

- 8 台双路服务器（共 16 sockets）
- 1 个 CXL 内存池盒子（里面插了很多 DDR5 DIMM）
- 中间可能有 CXL switch（交换器）连接大家

代码块

```

1  [Server1 CPU0] └─
2  [Server1 CPU1] └─
3  [Server2 CPU0] └─
4  ...           └─ CXL Switch/Fabric ─ [EMC + (DDR5 DIMMs)]
5  [Server8 CPU1] └─

```

- 左边每一个 “ServerX CPUY” 都是一个 **host**（更准确：**host 端实体**）
- 右边的 “EMC + DIMMs” 是一个 **device**
- Host 通过 CXL.mem 把 device 的内存映射成 “可 load/store 的系统地址范围”

Hardware layer

Host port

HDM decoder 是 “硬件级映射”，不是页表

页表映射（OS/MMU 管的：虚拟地址→物理地址）

- CPU 先把进程/VM 的虚拟地址翻译成物理地址（物理页帧号）。
- 这一步由 MMU + 页表完成。

HDM decoder 映射（平台硬件管的：物理地址→本地 DRAM or CXL 设备）

- 当你已经得到一个**物理地址**之后：
- 这个物理地址究竟是在本地 DRAM？还是在 CXL 设备内存？
- **这一步由 HDM decoder 决定。**

两级翻译链：

$VA \xrightarrow{\text{页表/MMU}} PA \xrightarrow{\text{HDM decoder}} (\text{Local DRAM}) \text{ or } (\text{CXL.mem Device})$

│ Pond 里做池化，更多是在 “让一些 PA 落到 CXL 那段范围”，而不是改变 VA→PA 的基本机制。

假设系统把物理地址空间分成两段：

- 0x0000_0000_0000_0000 ~ 0x0000_00FF_FFFF_FFFF：本地 DRAM（256GB）

- 0x0000_0100_0000_0000 ~ 0x0000_013F_FFFF_FFFF：CXL pool（64GB）

这时：

1. 程序访问某个虚拟地址 VA
2. 页表把它翻译成物理地址 PA = 0x0000_0102_1234_5678（落在第二段）
3. CPU 把这个 PA 送到内存系统
4. **HDM decoder 看到 PA 属于 CXL 范围** → 把这次访问导流到某个 CXL 端口（连向 EMC）
5. EMC 收到请求，查表发现这个地址所在的 1GB slice 当前 owner=本 host → 允许访问 → 去 DDR 取数据 → 返回

如果 owner 不匹配，就直接报错。

这就是“HDM decoder + EMC permission check”的完整协作链。

I/O Die（输入输出芯片/IO 芯粒）

在现代 CPU（尤其是 chiplet 架构）里，经常把芯片分成：

- **计算 die**：负责算（cores）
- **I/O die**：负责和外界沟通（内存控制器、PCIe、Infinity Fabric…）

论文里 IOD 的意思一般是：

EMC 这颗芯片需要很多 PCIe5 lanes、很多 DDR5 channels，它的“接口规模/复杂度”接近一个现代 CPU 的 I/O die。

总结一下：

VA→PA 的映射规则由 OS/Hypervisor 设定，翻译由 MMU 执行；PA→本地 DRAM 或 CXL 设备内存由硬件地址解码/路由（如 HDM decoder）决定。Pond 通过 CXL.mem + EMC 把池内存纳入物理地址空间，并用 slice ownership（owner）保证每个切片同一时刻只被一个 host 使用，从而避免跨主机一致性。IOD 是用来类比 EMC 这类芯片接口规模的参照（I/O die）。

Tracking 1024 slices (1TB) and 64 hosts (6 bits) $<2^6>$ requires 768B of EMC state. The EMC implements dynamic slice assignment by checking permission of each memory access, i.e., whether requestor and owner of the cacheline's slice match.

每个 slice 需要 6 bits 来记录 owner。

- 总 bits = 1024 slices × 6 bits/slice = 6144 bits
- 换成 bytes：6144 ÷ 8 = 768 bytes

retimer

- 高速串行信号（PCIe5/CXL）跑很远会失真（衰减、串扰、抖动），错误率上升。
- **retimer** 就像“中继站 + 重新整理节拍”：把接收到的信号重新定时、整形，再发出去。

latency来源：

DDR 带宽 vs CXL 带宽：谁更容易成为瓶颈？

非常重要的一点：

- DDR 侧：你可以用很多通道把带宽堆到几百 GB/s
- CXL/PCIe 侧：每个 host 链路带宽相对有限（几十 GB/s）

所以在池化系统里经常出现一种结构性矛盾：

池后端内存（DDR）很能吐数据，但前端链路（CXL/PCIe）可能先堵住。

这也是 Pond 为什么强调“池规模别太大 + 多头直连减少交换开销”：越大规模通常越依赖交换网络，网络更容易成为瓶颈，同时还增加延迟。

小池更快是因为少用 switch/retimer；带宽瓶颈往往在前端 CXL 链路/仲裁而不是后端 DDR 通道。

总结：

1. CXL 池内存延迟主要不是距离，而是 **端口处理 + retimer + switch**。
2. retimer 为长距离信号完整性服务，每个方向约 +10ns；数据中心 >500mm 时可能需要。
3. switch 为扩池服务，但代价很大， $\geq 70\text{ns}$ （甚至 $>100\text{ns}$ ）。
4. Pond 用“多头 EMC 直连”减少对 switch 的依赖，比 only-switch 方案延迟低约 1/3。
5. 8/16 socket 小池额外只增加 70–90ns，所以最可能落地；大池收益递减且更贵更慢。
6. 想要更多容量/带宽，可以让 CPU 连多个 EMC。

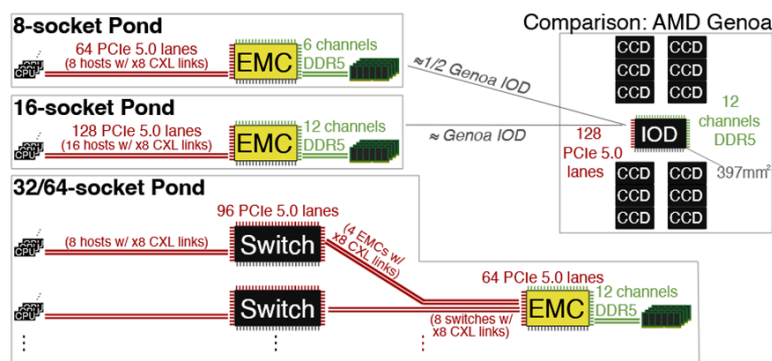


Figure 6: External memory controller (EMC) (§4.1). The EMC is multi-headed which allows connecting multiple CXL hosts and DDR5 DIMMs. A 16-socket Pond requires 128 PCIe 5.0 lanes and 12 DDR5 channels, which is comparable to the IO-die (IOD) on AMD Genoa [10, 11]. Larger Pond configurations combine a switch with the multi-headed EMC.

EMC（External Memory Controller）是 Pond 设计的一颗 ASIC：

- 左边接 **多个 CXL x8 ports**（连很多 host/sockets）
- 芯片内部用 NOC（片上网络）把请求路由到
- 右边的多个 **DDR5 MC**，最后访问 **DIMM**

Pool size（池大小）= 能接入同一池的 socket 数

- 8/16 socket 往往可以 “直连一个 EMC”
- 32/64 socket 往往需要 CXL switch（交换机）扩展规模

EMC 是多端口 CXL 到多通道 DDR 的外置内存控制器；池越大越依赖 switch，成本和延迟更高。

slice / owner：池内存怎么在多个 host 之间安全共享？

关键：Pond 不做跨主机 cache coherence（不搞一致性）

所以它用 **ownership 模型**：

- 把池内存按 **1GB slice（切片）** 切成很多块
- **每个 slice 同时只能被一个 host 拥有（owner）**
- EMC 维护表：`slice_id → owner_host_id`
- **每次内存访问都做权限检查**：请求者 host 是否等于 owner
 - 1TB = 1024 slices；64 hosts 需要 6 bits 表示 owner
 - $1024 \times 6 \text{ bits} = 6144 \text{ bits} = \mathbf{768B}$ （极小的硬件状态）

slice 是按地址切的 1GB 逻辑块；owner 表让每块同一时刻只归一个 host，从而避免跨主机一致性，同时保证隔离安全。

System Software Layer

怎么让 slice “借来借去” 且能回收？：

1. 启动时：池内存地址空间可见，但默认 offline

Host 能发现/映射池内存，但 OS 不把它当可用内存。

2. 运行时：Pool Manager 触发 add/release（热插拔）

- **Add_capacity**：把某个 slice 分给 host → OS online → EMC 更新 owner/权限
- **Release_capacity**：host 把 slice offline → EMC 撤权 → slice 回池

3. 防碎片（特别关键）

- 要回收 1GB slice，必须保证该 1GB 连续范围内没有零碎页被占用
- 所以他们让 driver/agent 之类的零碎分配只用本地内存，池内存尽量只给 hypervisor/VM 大块使用
- 还提到 offline 可能 10–100ms/GB，online 近似微秒级 → 解释了为什么释放要异步

4.2 用内存热插拔在线/离线实现 slice 分配回收，并通过分区策略避免碎片让 1GB slice 可回收。

hypervisor 怎么造出 zNUMA？—— “给内存，不给 CPU”

hypervisor 的操作是：

adding a memory block (node_memblk) without an entry in the node_cpuid in the SLIT/SRAT tables

- **给这个新 NUMA 节点添加内存块（node_memblk）**

- 但不给它挂任何 CPU (`node_cpuid` 没有条目)

SRAT (“资源归属表”) : 哪些 CPU、哪些内存属于哪个 NUMA 节点

SLIT (“距离/延迟表”) : 节点之间的 NUMA distance matrix (延迟远近)

guest OS 会优先用本地 NUMA，再用 zNUMA”

| guest-OS preferentially allocates memory from the local NUMA node before going to zNUMA.

加速器

这里的 “accelerator” **不是 GPU/TPU**，也不是深度学习加速器。

它更像是：**hypervisor 里的一种 “快速路径” 机制**——让 VM 访问内存/设备时走更高效的硬件虚拟化路径（快、开销小）。

当 hypervisor 需要做一些 “重配置/搬家” 的操作（例如迁移、缺页重映射、把池内存搬回本地），它会**暂时关掉这个快速路径**，改走更通用但更慢的路径（论文称为 “slower I/O path”），做完再开回来。

为什么要关？

因为高速路规则很严格、状态很复杂，做 “搬家/重映射” 时容易出错或成本很高；用通用慢路径更容易实现正确性。

| **加速器=虚拟化快速路径开关**，不是某块外设。

memory-bound (内存受限) 翻译成成人话：

| CPU 不是因为算不过来慢，而是因为 “等内存数据等太久” 慢。

Pool 的单位是 host (借 slice、owner=host)，VM 通过所在 host 的 hypervisor 以 zNUMA 的形式间接使用池内存；16-socket pool 说的是能共享池的 socket 数，常见对应 8 台双路 host (每台 host 2 sockets)。

“opaque VMs” 意思是：云提供商往往看不到 VM 内部应用逻辑（用户不告诉你），所以你得靠 “外部可测的信号” 判断它是不是延迟敏感、有没有大量闲置页。

观测方法：

1. CPU 性能计数器 (Core PMU) + TMA (Top-Down Method)

- **PMU (性能监控单元)** : CPU 硬件自带的一堆计数器，能告诉你：指令退休多少、缓存 miss 多不多、流水线 stall 多不多……
- **TMA (Top-Down Method)** : 一种把 “CPU 为什么没跑满” 分解成几大类的分析框架。
Pond 关心其中一个指标：**memory-bound** (因为内存访问导致流水线停顿的比例/程度)。

直觉：

- 如果一个 VM 非常 memory-bound、而且对延迟敏感，那么给它太多池内存 (额外 70-90ns) 就可能让 QoS 崩掉。

- 所以 PMU/TMA 指标帮助 Pond 识别“谁是延迟敏感/内存瓶颈型”。

工程点：他们把这些计数器归因到单个 VM（修改 hypervisor 做 per-VM 统计），并把采样写进分布式数据库；而且用的是简单计数器，开销很低（不像 event-based sampling 那种可能更重）。

2. VM 的“untouched pages（未触碰页）”遥测

Pond 想知道：VM 申请的内存里，有多少页其实从没被访问（冷页/闲置页）。这决定了“能放多少到池里而不伤性能”。

他们用两种信号：

guest-committed memory（客户提交内存）：一种现成计数器，但会“高估已用内存”（偏保守）

扫描页表访问位（access bits）：hypervisor 页表里通常有“最近是否被访问”的 bit。

Pond 只关心“从没被访问”的页，所以不需要频繁 reset 访问位，开销就能很低。

可以复述：

Pond 一边用 PMU/TMA 判断 VM 是否 memory/latency sensitive，一边用页表访问位估计有多少长期不触碰的页，从而决定池内存分配比例。

Prediction model

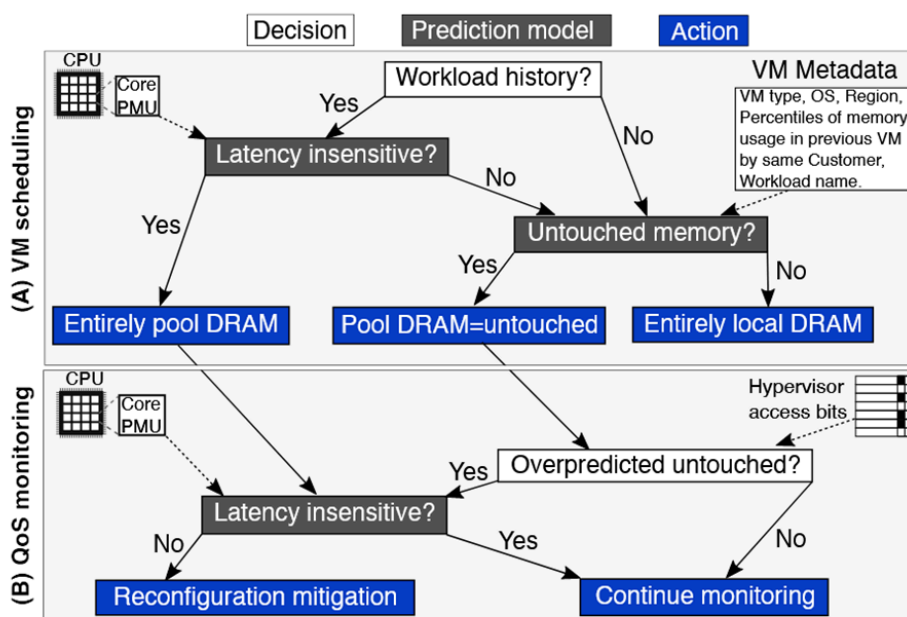


Figure 13: Pond prediction models (§4.4). Pond’s prediction models (dark grey) rely on telemetry (blue boxes) that is available for all VM types, including third-party opaque VMs.

两个预测模型

1. Latency-insensitive 模型：判断“全放池内存是否仍在 PDM 内”。若是 → 全池化。

特征：PMU/TMA 等硬件计数器；标签：本地 vs 池的 slowdown；每天重训。

2. **Untouched-memory (UM) 模型**：预测 VM 生命周期最小冷内存。UM>0 → 给 VM 一个 **GB 对齐** 的 zNUMA；UM=0 → 全本地。特征：VM 元数据 + 客户最近一周 UM 分位数；标签：最小 untouched memory。

代码块

```
1  if 能匹配到历史：
2      if 延迟不敏感模型 == 是：
3          VM 全放池内存
4      else:
5          zNUMA_size = 预测的 untouched memory (更保守的分位数)
6          VM: 本地 = 总内存 - zNUMA_size, 池 = zNUMA_size
7  else:
8      更保守 (通常不给全池, 更多依赖 UM 预测/默认策略)
9  运行中:
10     监控 access bits + PMU/TMA + QoS
11     若 spill/性能超 PDM:
12         触发一次性纠偏 → 搬回全本地
```

Implementation

不是用 gem5 那种周期级仿真来评估系统策略，而是直接在真实服务器上，用一种方法把“像 CXL pool 那样更慢的内存”做出来 (emulate)，然后跑真实工作负载。

1. 用双路服务器“假装”单路 + CXL pool

emulate a single-socket system with a CXL pool on a two-socket server by disabling all cores in one socket, while keeping its memory accessible from the other socket. This memory mimics the pool.

这句非常关键，它解释了 Pond 的“CXL emulation 技巧”：

Socket0 cores <-> DRAM0 (local, fast)

| 互连

Socket1 cores <-> DRAM1 (local for socket1, remote for socket0)

- 把 Socket1 的 **所有 CPU 核禁用** (不让它跑任何线程)
- 但不拔掉 DRAM1，保持 DRAM1 仍然存在、可寻址、可访问

于是系统变成：

- 只有 Socket0 有 CPU 能跑程序
- 但系统里依然有两堆内存：DRAM0 (快) + DRAM1 (慢，因为要跨 socket 访问)
- 这样，那个“没有 CPU 核但有 DRAM 的 socket”就很像 **zNUMA / CXL pool**:
 - 它的内存访问延迟更高 (因为跨 socket)

- 它“cpuleless”（没有 core），符合他们想模拟的“池内存没有本地计算”的特性

2. 改 Azure hypervisor 支持任意 zNUMA 拓扑，并把拓扑从控制面传下来

change Azure's hypervisor to instantiate arbitrary zNUMA topologies... extend the API ...
pass the desired zNUMA topology to the hypervisor.

意思是：

- 他们不仅仅手动搞一个固定 zNUMA，而是让 hypervisor 能创建“任意大小的 zNUMA 节点”（zNUMA 多大由模型预测决定）。
- 为此还要改控制面→host 的 API：调度器决定 VM 要多少 zNUMA，然后把这个“NUMA 拓扑配置”传给 hypervisor。

3. 在 hypervisor 里补齐训练模型需要的遥测（telemetry）

因为 VM 的 vCPU 不是一直固定绑在某个物理核上，它会被 hypervisor 的调度器挪来挪去。

如果你直接读某个物理核的 PMU，那是“这个核的计数器”，不一定属于某个 VM。

他们的做法是：

- 给每个虚拟核（vCPU）维护一份“PMU 状态快照”
- 当 vCPU 被换到另一个物理核上运行时，把 PMU 状态也一起迁移/对齐
这样你才能做“按 VM 归因”的 PMU 特征，给模型训练用。

问题：

他们为什么需要双路服务器？

因为要用远端 NUMA 内存模拟“慢的池内存”，并禁掉远端核模拟 cpuleless。

为什么要改 hypervisor？

为了让 VM 启动时能拿到“可变大小的 zNUMA”，并把配置从控制面传下去。

为什么要 per-vCPU 维护 PMU 状态？

因为 vCPU 会在不同物理核之间迁移，否则计数器归因不到 VM。

为什么 access bit 扫描要 reset？

为了区分“最近是否访问”，用窗口统计 untouched pages。

Evaluation

Evaluation

zNUMA works (when sized right), but spill hurts

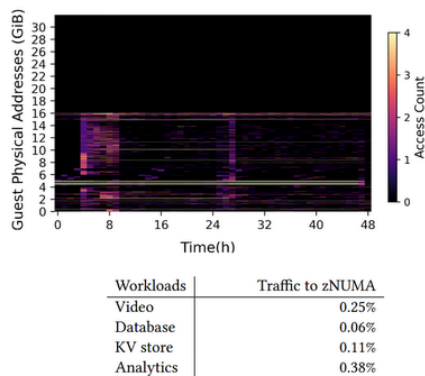


Figure 15: Effectiveness of zNUMA (§6.2). Latency sensitive workloads get a local vNUMA node large enough to cover the workload's footprint. zNUMA nodes holds the VM's remaining memory on Pond CXL pool. Access bit scans, e.g., for Video (right), show that this configuration indeed minimizes traffic to the zNUMA node.

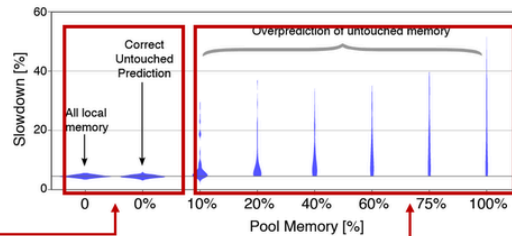


Figure 16: Slowdown under different pool allocations (§6.3). Performance for local memory and a correctly-sized zNUMA is comparable. Small slowdowns arise from run-to-run variation. Slowdown becomes noticeable as soon as the workload spills into the zNUMA and steadily increases until the whole workload is allocated on pool memory (100% spilled).

- zNUMA is barely touched: zNUMA traffic on production nodes is usually <0.4% —proving that the OS prioritizes local NUMA
- Proper sizing $\approx$ full local performance: When the zNUMA size is approximately equal to the untouched memory, the performance is close to all-local.
- Incorrect sizing $\rightarrow$ spill $\rightarrow$ significant performance degradation: Once the working set spills over to zNUMA (slow memory), the slowdown rises rapidly (some workloads can reach 30%+).
- Conclusion: "Prediction + fallback" is necessary; memory cannot be put into the pool at a fixed ratio.

Evaluation

Two-model control: bound mispredictions, maximize pooled DRAM

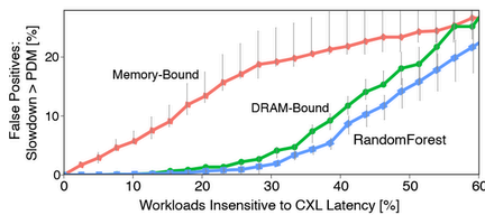


Figure 17: Latency insensitivity model (§6.4). As we increase how many workloads are marked as insensitive (LI), the rate of false positives (FP) increases. Pond's RandomForest slightly outperforms a heuristic based only on the DRAM-bound TMA performance counter.

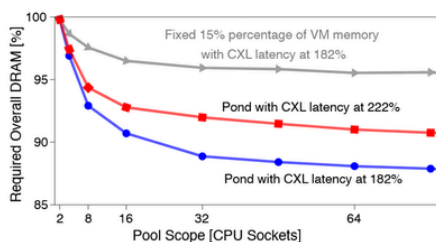


Figure 21: Memory savings under performance constraints (§6.5). Simulated end-to-end evaluation of memory savings achieved by Pond under PDM = 5% and scheduling mispredictions TP = 98%.

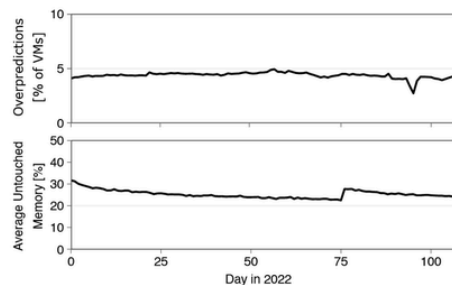


Figure 20: Untouched memory ML model performance in production (§6.4). Our production model targets 4% overpredictions (OP). Its average untouched memory percentage is similar to the simulated model (Figure 19).

- Model 1 (Latency-insensitive): Control false positives (misclassifying latency-sensitive VMs as insensitive) to avoid "full pooling risks."
- Model 2 (Untouched memory): Allocate a reasonable size to zNUMA, place "cold pages" into the pool, and minimize spill triggers.
- QoS monitor as fallback: A small number of misclassifications trigger reconfiguration or migration back to fully local memory to guarantee throughput (TP).
- End-to-end benefit: Under 16-socket pool and performance constraints, Pond still reduces overall DRAM demand by about 7–9% (outperforming static fixed-ratio policies).

- 模型 1 (Latency-insensitive) : 控制 FP (把敏感 VM 误判为不敏感) $\rightarrow$ 防止 "全池化踩雷"。
- 模型 2 (Untouched memory) : 给 zNUMA 合理大小 $\rightarrow$ 把 "冷页" 放池里, 尽量不触发 spill。
- QoS monitor 兜底: 少量误判触发重配置/迁回全本地, 保证 TP。

端到端收益: 在 16-socket pool、性能约束下, Pond 仍能降低整体 DRAM 需求约 7–9% (优于静态固定比例策略)

Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices

备用知识

Memory interface（内存接口）CPU———DIMM

传统服务器CPU架构：

CPU 连接内存靠的是 **DDR 内存通道（DDR channels）**：

- CPU 芯片里有 **IMC（Integrated Memory Controller，集成内存控制器）**
- IMC 通过主板的线路连到 DIMM 插槽
- 一个通道有固定的电气规范、频率、时序、信号训练等

所以“固定”包括：

1. **接口类型固定**：这代 CPU 支持 DDR5，你就只能插 DDR5（插 DDR4 不行）。
2. **通道数量固定**：CPU 只有 8 通道，你最多就 8 通道带宽（想变 12 通道不可能）。
3. **每通道 DIMM 数量/速率受限**：插多条会降频或变不稳定，带宽/容量都受约束。

| CPU 的“内存插口规格”和“插口数量”是写死在 CPU 里的。

“解耦/decouple”

CXL 让 CPU 通过 **PCIe 物理层** 去访问“挂在外面”的内存设备（CXL Type-3 memory expander）。

关键点：

- CPU 不再直接用 DDR 信号去驱动那堆 DRAM 芯片
- 中间多了一个 **CXL 控制器**（在设备端）
- 设备端控制器可以接 **DDR4/DDR5/甚至其他内存技术**（协议上允许）

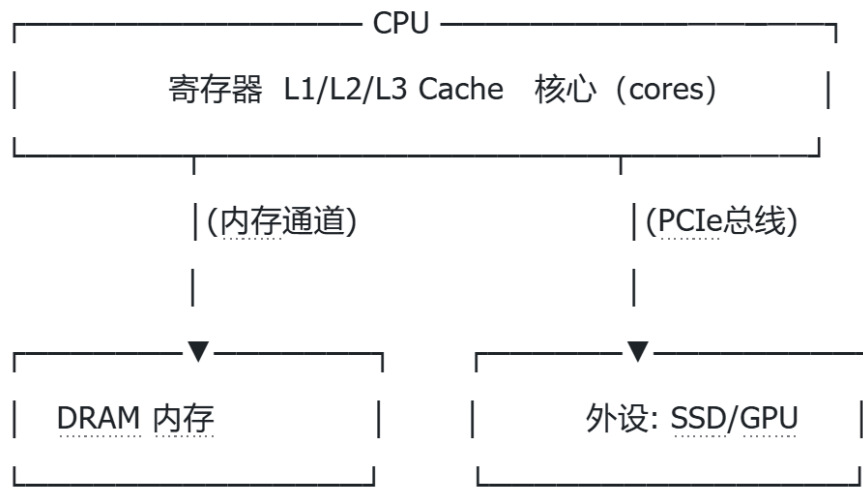
所以“decouple”指：

| CPU 不必只绑定某种 DDR 接口和某种内存技术，内存设备可以在 CPU 外部自己演化。

Cache miss path

CPU 取数据有“层级”：

- L1（最快，最小）
- L2
- LLC（Last Level Cache，最后一级缓存，一般就是 L3）
- 内存（DRAM 或 CXL 内存）



对本地 DRAM 来说：

Core → (L1 miss) → (L2 miss) → (LLC miss) → IMC → DDR 通道 → DIMM → 回来

对 CXL 来说：

Core → … → (LLC miss) → CXL port → PCIe/CXL 链路 → 设备端控制器 → 设备端 DRAM → 回来

CXL Port arbitration

CPU 里多个“请求者”会发内存请求，比如：

- 多个 CPU 核心同时缺数据
- 预取器（prefetcher）也在发请求
- 写回（writeback）也在占用链路

而 **CXL port** 的带宽是有限的（比如 x8）。

所以必须有一个机制决定：

在同一时刻，哪些请求先走，哪些后走，如何排队，如何公平/优先级。

这就是 **arbitration（仲裁/调度）**。会导致：

- **排队延迟（queueing delay）**：请求太多时，延迟暴涨
- **带宽效率下降**：仲裁/切换/协议开销让吞吐不满

延迟（latency）：拿到一口数据要等多久（像快递多久到）

带宽（bandwidth）：单位时间能搬多少数据（像每分钟能运几箱）

带宽密集型应用（bandwidth-intensive）

就是那种：

不是等快递慢，而是“货运量太大，卡在吞吐”。

典型特征：

- 数据流式读写、大数组扫描

- 工作集大、缓存命中率低
- 多核并行时内存通道被打满（memory controller 饱和）

例子（直觉版）：

- 大规模矩阵运算、科学计算（streaming）
- 图计算里某些遍历（但也可能是 latency-bound，看访问模式）
- SPEC 中一些 MPKI 很高的 benchmark
- 推荐系统 embedding 查表（可能带宽 + 延迟混合）

额外带宽

传统只有 DDR 通道提供带宽。假设你 CPU 有 8 通道 DDR5，总带宽就被这个限制。

CXL 设备相当于又给你开了一条“外部内存通道”：

- 本地 DDR 通道：8 条
- 外部 CXL link：比如 x8 / x16，也能提供几十 GB/s

所以“额外带宽”就是：

你除了本地 DDR 的吞吐外，还能从 CXL 再拿一部分吞吐，两边加起来总吞吐变大。

这也是论文里把 CXL 当成 **bandwidth expander（带宽扩展器）** 的意思

GT/s：Giga Transfers per second 每秒传输次数，不是字节。

GB/s：Giga Bytes per second 每秒字节数，我们日常看速度用的。

TLP（Transaction Layer Packet）：事务层“包”，相当于“我读/写某地址”的请求包

DLLP（Data Link Layer Packet）：链路层包，负责可靠传输、确认等

Home Agent（HA） CPU 内部的“内存路由器”：

当 CPU 核心发出 load/store：

- 如果地址属于本地 DDR：走传统内存控制器
- 如果地址属于 CXL 内存范围：HA 把请求导向 CXL 端口/控制器

关键点：OS 看到的样子

为了让软件不用改，HA 会把 CXL 内存伪装成一个“没有 CPU 核心的 NUMA 节点（只有内存）”。

Chiplet

就是“把一颗大芯片拆成几块小芯片拼在一起”。

传统 CPU 是一整块大硅片（monolithic die）。现在很多高端 CPU 为了良率、成本、扩展性，会把核心/缓存/IO 等拆成多个小 die，用高速互连拼起来。SPR 有 4 个 chiplet（Intel 也叫 tile/die），每块有一部分核心、缓存和内存通道。

同一个 socket 内部也会有“远近”差异：核心访问自己 chiplet 上的资源更近，访问别的 chiplet 上的资源更远。

所以出现 SNC 模式时, chiplet 直接决定 NUMA 切分方式、LLC 的隔离边界。

锁频（固定频率）锁频”就是把 CPU 固定在一个频率（比如论文锁在 2.1GHz），不让它上上下下 CPU 的**运行频率（GHz）**在现代系统里**不是固定的**，会动态变化：

- 负载高 → 提升频率（Turbo）**Intel Turbo Boost**：按需自动提升 CPU 主频，短时间爆发性能
- Turbo（睿频）** CPU 在满足功耗/温度条件时，**自动把频率提高**，短时间更快。
- 温度高/功耗超标 → 降频
- 空闲 → 省电降频

你要测内存延迟/带宽时，如果 CPU 自己频率在变，你测到的时间就会变，无法区分是：

CXL/DDR 的差异 还是 CPU 频率变化导致的差异

功耗墙（power limit / power wall）

CPU 有一个“最大允许功耗”（比如 250W）。当你负载很重：

- 功耗接近上限
- CPU 必须降频才能不超功耗/不烧
这就是“撞到功耗墙”

Hard IP vs Soft IP

Hard IP（硬核 IP）

- 在硅片里做成**专用电路**（ASIC）
- 速度快、延迟低、功耗低、吞吐高
- 可定制能力相对差，但性能最好

Soft IP（软核 IP）

- 用 FPGA 里的可编程逻辑实现
- 优点：灵活、好改、适合原型
- 缺点：时钟频率低、逻辑开销大、性能差（尤其带宽效率差）

论文里用一个 FPGA-based soft IP 的 CXL 设备（CXL-C），就是为了展示：
控制器实现方式会显著影响 CXL 性能。

类比 Hard IP = 专用硬件计算器 Soft IP = 你用 Excel/脚本算，灵活但慢

Benchmark 就是用来“测性能”的标准程序/工作负载

两类常见目的：

- 测**延迟**：比如响应时间、p99
- 测**吞吐**：比如 QPS、每秒任务数、SPEC score

Microbenchmark（微基准）

- 很小、很“纯”
- 专门测试某一个硬件行为/路径
- 例如：只测内存读延迟、只测 memcpy 带宽、只测某条指令的代价

论文里：

- Intel MLC：测内存延迟/带宽（经典微基准）
- memo：作者写的更细粒度微基准（能测特定指令，如 non-temporal load/store）

线程（thread） = CPU 需要执行的一串指令 + 这串指令的上下文（寄存器、栈、程序计数器等）。

thread（线程） 理解成：

- 一份自己的 **PC 程序计数器**
- 一套自己的 **registers（寄存器：保存该线程临时状态的数据存储单元）**
- 自己的执行进度
- 每个线程都有自己的指令流 **instruction stream（指令流：按程序顺序的一串指令）**。

多线程能：

- **并行**（多核同时跑）
- **并发**（一核轮流跑，提升响应）

超线程 Hyper-Threading

一个物理核心（physical core）对外表现成两个逻辑核心（logical CPU）。

操作系统会看到：（**同一个核心“同时服务两个线程”**）

- 你买的是 8 核 CPU
- 开了超线程后，OS 会显示 16 个“CPU”（16 个 logical CPU）

为什么他们要关超线程？

因为他们在测“内存系统”的差异（DDR vs CXL）时，最怕混入额外变量。

开着超线程会带来：

- 两个线程抢 cache / 内存带宽
- 延迟波动更大
- 结果更难复现

所以论文关掉超线程，是为了：

让“测到的差异尽可能来自内存类型”，而不是来自核心内部资源争用。

inclusive vs non-inclusive cache architecture

inclusive（包含式） “L3 是 L2 的全集”

- 如果一条数据在 L1/L2 里，那么它**一定也在 L3（LLC）里有副本**

- 好处：做一致性/探测比较简单（去 L3 查就知道）
- 缺点：L3 会被“重复副本”占容量，有点浪费

non-inclusive（非包含式） “L3 不保证有 L2 的副本”

关键行为：

当发生 LLC miss、需要去内存拿数据时：

- 数据先被装进 **该核心的私有 L2**
- 而不是直接装进共享 L3
之后当 L2 里的 cacheline 被挤出去（evict）时，才会被放进 L3
所以 L3 在这里扮演 **victim cache（受害者缓存/后备缓存）**：谁从 L2 掉下来，L3 就接住。

CXL 4.0（联盟官网明确写“Available Now”）

The CXL 4.0 doubles bandwidth from 64GTs to 128GTs, adds support for bundled ports, and enhances memory RAS features.

Key features of the CXL 4.0 specification

- Doubles the bandwidth to 128GTs with zero added latency
 - Enables rapid data movement between CXL devices, directly improving system performance
 - Maintains previously enabled CXL 3.x protocol enhancements with the 256B Flit format
 - Introduces the concept of native x2 width to support increased fan-out in the platform
 - Support for up to four retimers for increased channel reach
- Implements CXL bundled port capabilities
 - Ability to combine device ports between Host and CXL accelerators (Type 1/2 devices) to increase bandwidth of the connection
- Delivers memory RAS enhancements
 - Improves reliability, error visibility, and maintenance efficiency
- Continued full backward compatibility with CXL 3.x, 2.0, 1.1, and 1.0
- **链路速率：从 64 GT/s 提升到 128 GT/s**（联盟原话：*doubles bandwidth from 64GTs to 128GTs*）
- **“零额外延迟（zero added latency）”**：这里指的是规范层面宣称“带宽翻倍不引入额外协议延迟”，不是说端到端内存访问延迟为 0。
- **沿用 CXL 3.x 的 256B FLIT 格式**（关键点：CXL 3.x/4.0 进入 FLIT 化的数据传输）。
- **新增 native x2 link width（原生 x2 宽度概念）**，用于更高 fan-out 的平台连接。
- **支持最多 4 个 retimer（更长链路距离）**。

- **Bundled Ports（端口捆绑）**：可以把 Host 与 CXL 加速器（Type1/2）的多个端口 “绑成更高带宽连接”。
- **增强内存 RAS（可靠性/可维护性）**，并向下兼容 **CXL 3.x/2.0/1.x**。

表格里 “bit-transfer rate” 那行，最新版：**CXL 4.0：128 GT/s**（规范层面的 link rate）

CXL

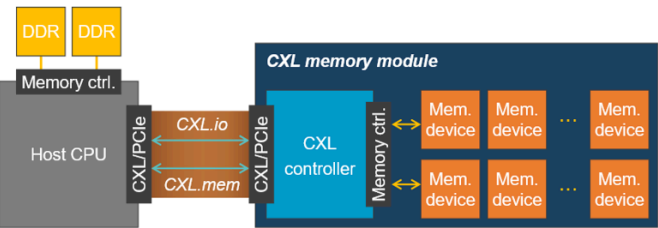


Figure 1: CXL memory module architecture.

The CXL 4.0 doubles bandwidth from 64GTs to 128GTs, adds support for bundled ports, and enhances memory RAS features.

Key features of the CXL 4.0 specification

- Doubles the bandwidth to **128GTs** with **zero added latency**
- Support for up to **four retimers** for increased channel reach
-

	CXL	PCIe 4.0	DDR
bit-transfer rates	Same as PCIe (CXL is built on the PCIe PHY)	16 Gbps/lane (PCIe 4.0 x16: ≤2556Gbps)	3.2 Gbps/pin(DDR4-3200) (DDR4-3200: ≤25.6 Gbp/channel)
power	Same as PCIe	6 pJ/bit	22 pJ/bit(DDR4)
link latency	Same as PCIe	~40 ns	<1 ns(DDR4)
load and store	yes	no	yes
pins	~3× fewer pins than DDR5		DDR5 : 288 pins/channel

项目	CXL 3.2	CXL 4.0 (最新版)	对照：PCIe 6.0
发布时间（公开发布）	2024-12（公开发布公告）(Compute Express Link -)	2025-11/12（公开发布公告/讲义）(Compute Express Link -)	PCI-SIG 官方规格页（已发布）(PCI-SIG)
链路速率（raw, per-lane）	与所基于的 PCIe 代际一致（3.x 对应 64 GT/s 时代）	128 GT/s （官方：从 64GT/s 翻倍到 128GT/s）(Compute Express Link -)	64.0 GT/s （raw）(PCI-SIG)
“带宽翻倍但不增加延迟”表述	—	官方说法： doubles bandwidth… with zero added latency （指规范/协议层不额外增加）(Compute Express Link -)	—
FLIT（流控单元）	3.x 进入 FLIT 化（用于更高效率/可靠性）(Compute Express Link -)	继续使用 256B FLIT （官方讲义）(Compute Express Link -)	PCIe 6.0: Flit mode + FEC/C （官方）(PCI-SIG)
关键新增能力（与内存系统/拓扑相关）	CHMU 热页监控单元 （用于内存分层/热页监控）(Compute Express Link -)；管理/事件记录/MMPT/在线固件激活等(Compute Express Link -)	Native x2 link width （原生 x2 宽度，提升 fan-out）(Compute Express Link -)； **Bundled Ports （端口捆绑） **提升聚合带宽 (Compute Express Link -)； 最多 4 个 retimer 延长链路(Compute Express Link -)； 内存 RAS 增强 (Compute Express Link -)	PCIe 6.0 更偏 PHY/编码（PAM4、FEC、Flit mode）(PCI-SIG)
安全	Trusted Security Protocol (TSP) （官方）(Compute Express Link -)	延续/增强（4.0 公告强调整体能力增强与兼容）(Compute Express Link -)	—
向后兼容	—	官方明确：兼容 CXL 3.x/2.0/1.1/1.0 (Compute Express Link -)	PCIe 各代通常向后兼容（常识时可补 PCI-SIG 引用）

Item	CXL 3.2	CXL 4.0 (Latest)	Reference: PCIe 6.0
Public release	Dec 2024 (public announcement) (Compute Express Link -)	Nov/Dec 2025 (public announcement/webinar) (Compute Express Link -)	PCI-SIG spec page (PCI-SIG)
Raw link rate (per lane)	Aligns with the underlying PCIe era (3.x era ~ 64 GT/s)	128 GT/s (official: doubles from 64GT/s to 128GT/s) (Compute Express Link -)	64.0 GT/s raw (PCI-SIG)
“Zero added latency” wording	—	Official wording: “ doubles bandwidth… with zero added latency ” (spec/protocol-level phrasing) (Compute Express Link -)	—
FLIT / encoding	3.x moves to FLIT-based transfers (ecosystem material) (Compute Express Link -)	Maintains 256B FLIT (official webinar) (Compute Express Link -)	PCIe 6.0: Flit mode + FEC/CF (PCI-SIG)
Major new capabilities	CHMU for hot-page monitoring / memory tiering (Compute Express Link -); plus monitoring/management improvements (Compute Express Link -)	Native x2 link width (Compute Express Link -); Bundled Ports (Compute Express Link -); up to 4 retimers (Compute Express Link -); memory RAS enhancements (Compute Express Link -)	PCIe 6.0 focuses on PAM4 + F and Flit mode (PCI-SIG)
Security	Trusted Security Protocol (TSP) (Compute Express Link -)	Continued/enhanced (release notes emphasize improvements & compatibility) (Compute Express Link -)	—
Backward compatibility	—	Official: compatible with CXL 3.x/2.0/1.1/1.0 (Compute Express Link -)	Generally backward compatible (can add PCI-SIG citation if needed)

当 CPU 访问 CXL 内存时，会在 cache hierarchy 的每一层缓存 CXL 数据；除了 persistent memory，以前别的内存扩展技术做不到。

这意味着：

 | CXL 内存不是“每次都去远端拿”，CPU 仍然可以靠 L1/L2/L3 缓存把大部分访问拦截住。

这就会引出很多微妙问题：

- cache hit 时几乎跟本地一样快
- cache miss 才会付出 CXL 的额外延迟
- 还会涉及 cache coherence（多个核心一致性）检查

也正因为它能缓存，所以后面才会出现论文 C3 那种现象：**CXL 访问会影响 LLC 行为，甚至在 SNC 下改变等效 LLC 容量。**

CXL 需要 CPU 与设备两端硬件支持；Sapphire Rapids 和 AMD Genoa 是最早支持 CXL 1.1 的服务器 CPU。

CXL-ready Systems and Memory Devices

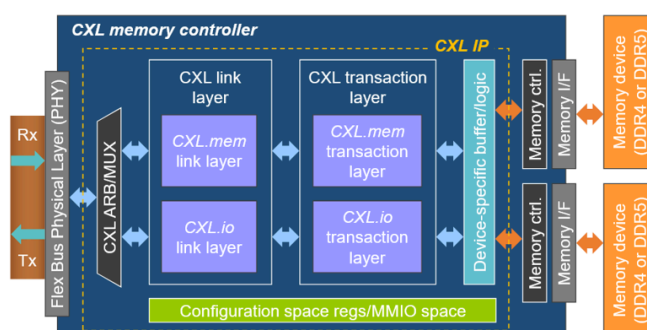


Figure 2: CXL.mem controller architecture.

CXL.io

CXL.Cache CPU memory $\xrightarrow{\text{visit}}$ Device

CXL.mem CPU $\xrightarrow{\text{visit}}$ Device memory

It caches data from/to the CXL memory in every level of its cache hierarchy

CXL.mem is implemented by **HA + CXL controller**. The OS exposes it as "remote NUMA node memory", and the CPU accesses it using **load/store**.

CXL.mem controller 主要包括

- (1) PCIe 物理层
- (2) CXL link layer
- (3) CXL transaction layer
- (4) memory controller blocks

并把 (2)(3) 等组件统称为 CXL IP

总结：

1. CXL 建在 PCIe PHY 上，但通过 CXL.io/cache/mem 提供更像“内存”的语义。
2. CXL.mem 由 HA + CXL controller 实现，OS 把它暴露为“远程 NUMA 节点内存”，CPU 用 load/store 访问。
3. CXL 最大的体系结构特点之一：访问数据会进入 CPU 各级 cache。
4. CXL memory device 本身没有 core/cache/长互连，这让它和“远程 NUMA 模拟”在硬件行为上本不同。

论文结论

C1: 真 CXL 的性能范围很大（不同设备差很多）

不同 CXL controller 设计 / 设备里用的 DRAM 技术不同 → 延迟、带宽差别会很大。

C2: 真 CXL 甚至能比“模拟 CXL (远程 NUMA)”更快 (某些情况下)

真 CXL 最高能比模拟 CXL 低 26% 的延迟、以及高 3-66% 的带宽效率 (看指令类型、看设备)。

为什么会出现“真 CXL 反而更快”？

真 CXL 设备 **没有 CPU 核心，也没有 cache**，虽然它被 OS 当成 NUMA node，但它不像远程 socket 那样有一整颗 CPU 会参与一致性/缓存干扰。

远程 NUMA (双路服务器)：访问“另一颗 CPU 的内存”，那颗 CPU 里面有很多核心和缓存，它会参与 cache coherence (缓存一致性) 的检查与通信，还可能有各种内部互连开销 → 这会让访问更“绕”。

真 CXL 设备：它只是一块“内存设备”，没有核心/缓存参与一致性 → CPU 这边反而可以用更直接的方式做一致性判断。

CPU 会实现一个 **片上 (on-chip) 的硬件结构**，让它对真 CXL 的一致性检查更快。

C3: SNC 模式下，访问 CXL 会“破坏 LLC 隔离”，反而像送你更大的缓存

SNC (Sub-NUMA Clustering)

Intel 把一颗 CPU 分成多个“更小的 NUMA 区” (每区有自己更偏本地的 LLC slices)，目的：

让不同任务互不干扰，给你更强的 LLC 隔离。

论文描述 SNC 的关键机制是：

当一个核心的 L2 缓存行被驱逐 (evict) 到 LLC 时，如果数据来自该 SNC 的本地 DDR，那么它会被“定向”放进同一个 SNC 的 LLC slices，维持隔离。

但访问 CXL 时发生了异常行为：

来自 CXL 的 L2 cache line，可能被 evict 到“任何 SNC 节点”的 LLC slices，而不是只在本 SNC 内。

结果是什么？

- 访问本地 DDR：你只能用“本 SNC 的那一份 LLC”
- 访问 CXL：你等效可以蹭到“多个 SNC 的 LLC 总和”，等效 LLC 容量 **2-4×**

这会产生一个很重要的“补偿效应”：

虽然 CXL 更慢，但如果应用很 cache-friendly (很多命中 LLC)，更大的等效 LLC 能抵消一部分延迟劣势。

总结 C1-C3：

真 CXL 与远程 NUMA 模拟在“硬件一致性路径、缓存交互、SNC 行为”上都有根本差异，所以“用远程 NUMA 代替 CXL”可能会误导系统设计结论。

F1: 微秒级 (μ s-scale) 延迟敏感应用会明显变差

例子：Redis / KV store 这类。论文说：

- 这类应用对内存访问延迟极敏感
- 把页面放到 CXL，会让 tail latency（比如 p99）上升 **10-82%**（相对本地 DDR）

某个当时最先进的 CXL-aware 页面放置策略（tiered memory policy）反而比静态分区更糟，因为它会做 page migration（页迁移），迁移本身有额外开销。

理解

Redis 这种像“秒回客服”的系统，你给它任何额外抖动和迁移干扰，它 p99 立刻爆炸。

所以：**别用“迁来迁去”的策略折腾延迟敏感应用。**

F2：毫秒级（ms-scale）复杂应用，放很多页到 CXL 影响反而小

例子：社交网络微服务（请求链路很长、CPU/网络/存储/排队都有）。论文说：

即使大部分页面在 CXL，tail latency 也只会边际增加。因为 CXL 增加的那点内存延迟，在整个端到端 ms 延迟里占比很小。

直觉：

如果一个请求本来就 30ms，内存慢 $0.1\mu s$ 对总时间几乎无感。

所以这类应用“更能忍”

F3：带宽密集型应用，“默认 50% 页进 CXL”也可能吞吐下降

既然 DDR + CXL 总带宽更大，那把一半页放 CXL 不就更快？不一定。

naive 地按默认 OS policy 把 50% 页放 CXL，吞吐可能更低，尽管 DDR+CXL 的聚合带宽更高。

为什么会这样？

- “聚合带宽更高”不等于“我的应用就能用上这份带宽”
- 如果你的访问模式导致更多请求走到更高延迟的那边、或产生更差的 locality、或触发不均衡/排队，吞吐反而掉。

这也正是论文后面要提出 Caption 的理由：

需要一个动态策略去找到“到底该放多少页在 CXL 才最赚”，而不是 50%。

实验

The server consists of **two Intel Sapphire Rapids (SPR) CPU sockets**. One socket is populated with **eight 4800 MT/s DDR5 DRAM DIMMs** (128 GB) across eight memory channels.

Table 1: System configurations.

Dual-socket server system			
Component	Description		
OS (kernel)	Ubuntu 22.04.2 LTS (Linux kernel v6.2)		
CPU	2× Intel® Xeon 6430 CPUs @2.1 GHz [37], 32 cores and 60 MB LLC per CPU, Hyper-Threading disabled		
Memory	Socket 0: 8× DDR5-4800 channels		
	Socket 1: 1× DDR5-4800 channel (emulated CXL memory)		
CXL memory devices			
Device	CXL IP	Memory technology	Max. bandwidth
CXL-A	Hard IP	DDR5-4800	38.4 GB/s per channel
CXL-B	Hard IP	2× DDR4-2400	19.2 GB/s per channel
CXL-C	Soft IP	DDR4-3200	25.6 GB/s per channel

clock frequency

Let "the detected differences come as much as possible from memory types"

Different devices will have different latency/bandwidth.

"The performance range of true CXL memory is very large."

clock frequency

Let "the detected differences come as much as possible from memory types"

Different devices will have different latency/bandwidth.
"The performance range of true CXL memory is very large."

一台双路 Intel Sapphire Rapids (SPR) 服务器：一颗 socket 插满 **8 条 DDR5-4800 DIMM (8 个通道)**，另一颗 socket 只插 **1 条 DDR5-4800 DIMM**，用来“模拟 CXL 的带宽和容量”。

The server consists of **two Intel Sapphire Rapids (SPR) CPU sockets**. One socket is populated with **eight 4800 MT/s DDR5 DRAM DIMMs** (128 GB) across eight memory channels.

远程 socket 只留 **1 个 DDR 通道**（带宽更小）用它当作“emulated CXL memory”（论文里后面叫 DDR5-R）

SPR CPU 里有 **4 个 chiplet**，每个最多 15 核、2 个 DDR5 通道；可以把 4 个 chiplet 当成一个统一 CPU，也可以在 **SNC (Sub-NUMA Clustering) 模式**下把每个 chiplet（或两个）当成一个 NUMA node，用来隔离共享资源（尤其是 LLC）。

发现：
在 **SNC** 下，访问 **CXL memory** 会“破坏 LLC 隔离”，甚至让你等效用到更大的 LLC。

为什么关 Hyper-Threading、锁频到 2.1GHz？

论文说他们关闭超线程，并把频率固定到 2.1GHz，让结果更可预测。

做性能测量最怕两个东西：

- 1. **频率乱跳**（Turbo、功耗墙）→ 同一个程序每次跑时间不一样
- 2. **超线程共享资源** → 两个逻辑核抢同一套执行资源/缓存/带宽，噪声很大

所以他们把实验环境“钉死”，让变化主要来自 **内存类型 (DDR vs CXL)**，而不是 CPU 自己发疯。

CXL memory devices：为什么要选 3 个设备，还故意让它们差异很大？

论文说他们用了 3 个 CXL 内存设备，每个设备的：

- **CXL IP** 不同：有 ASIC-based hard IP，也有 FPGA-based soft IP
- **DRAM 技术**不同：DDR5-4800 / DDR4-2400 / DDR4-3200

并强调：CXL 协议不规定底层一定要 DRAM，也可以是持久化内存、flash 等，所以不同设备会有不同延迟/带宽。

CXL 设备不是“一个数”

这点很关键：很多人把 CXL 当作“统一的一种慢内存”。作者用三种差异很大的设备，就是要告诉你：

- **控制器设计**能把性能拉开很大
- **底层内存技术**也会改变特性
- FPGA soft IP 往往会更慢/更低效

“true CXL memory 的性能范围很大”。

microbenchmark

First, we use **Intel Memory Latency Checker (MLC)** [42], a tool used to measure memory latency and bandwidth for various usage scenarios. Second, we use a microbenchmark dubbed **memo** (measuring efficiency of memory subsystems). It shares some features with Intel MLC, but we develop it to give more control over characterizing memory subsystem performance in diverse ways. For instance, it can measure the latency and bandwidth of a specific memory access instruction (e.g., AVX-512 non-temporal load and store instructions).

论文说他们用两个 microbenchmark：

1. **Intel Memory Latency Checker (MLC)**：经典工具，测 latency / bandwidth（各种场景）
2. **memo**：他们自己写的，用来更灵活地测“特定指令”的延迟/带宽，比如 AVX-512 的 non-temporal load/store

MLC 不够细，memo 用来“拆指令级差异”

非常强调 **不同指令类型**下，CXL 的表现差异巨大（比如：

- normal load vs non-temporal load
- store vs non-temporal store

为什么？

因为这些指令会走不同的微架构路径：

- 是否进入 cache？
- 是否触发 write-allocate（store miss 先读一条 cacheline）？
- 是否更依赖 coherence check？

MLC 作为“通用工具”很强，但它不一定能精确隔离这些指令语义差异。

MLC 给“宏观常规测量”，memo 给“指令级可控测量”。

这就是他们后面能解释“为什么真 CXL 比模拟 NUMA 更低延迟”的关键实验手段之一。

总结：

1. **emulated CXL = remote NUMA + 只留 1 个 DDR 通道**（人为压带宽）
2. **true CXL 设备差异很大**（hard IP vs soft IP + DDR4/DDR5）
3. 测量不仅看“平均延迟/带宽”，还看“指令类型”（MLC + memo）

benchmark

Latency-sensitive applications

1. Redis [69], a popular high-performance in-memory key-value store, with YCSB

Redis：内存 KV 存储（key→value），大量随机访问，典型在线服务。

YCSB：专门压 KV 存储的 benchmark（可控读写比例、key 分布等）。

- Redis 这种“请求本来就短 + 很多随机内存访问”的服务
→ **延迟敏感**
→ CXL 让 miss 更慢 → 单次请求变慢 → 吞吐边缘变差 → **p99 被排队放大**
- uniform key
→ 让 miss 次数增加
→ 更能体现 DDR vs CXL 的差距

2. **DSB** 是一个“微服务全家桶” benchmark，包含：

web 后端、load balancer、caching、storage 甚至 ML inference 组件，通过 Docker 启动多个服务拼成一个完整应用。

为什么它重要？

因为真实生产系统不是“一个进程”，而是一串服务链路。

这类系统的端到端延迟往往是 **ms 级**，其中只有一部分是内存访问造成的。

作者选了 3 个 workload：

- a. compose posts（发帖）
 - b. read user timelines（读用户时间线）
 - c. mixed（混合：10%发帖 + 30%读用户时间线 + 60%读首页时间线）
3. **FIO** 是 storage benchmark 工具，可以发各种 I/O 模式（随机读写、顺序读写、IO depth 等）。

用 FIO 来评估：如果把 CXL 内存用于 **OS page cache**，会怎样影响 I/O 延迟。

Throughput applications

吞吐类应用特点：

- 更关心单位时间完成多少（QPS、instances/s、SPEC rate）
- 往往更吃 **内存带宽**（而不是单次延迟）

1. DLRM inference（推荐系统）：embedding 查表=内存大户

DLRM 是深度学习推荐模型，核心瓶颈常在 embedding table 查表。

作者说 embedding reduction 有大内存 footprint，贡献 50–70% 推理延迟。

它非常典型地代表：

- 大量随机访问大表（cache 命中率低）
- 多线程推理时会把内存系统打满
这类应用非常可能“需要额外带宽”，也就是 CXL 作为 bandwidth expander 的潜在收益点。

2. SPECrate CPU2017：数据中心吞吐标杆（多实例并发）

- SPEC CPU2017：经典 CPU benchmark 套件
- **SPECrate**：不是跑一个实例看时间，而是跑很多实例看总吞吐（rate），SPECrate 常用于**数据中心吞吐评估**。

他们先算每个 benchmark 的 **MPKI（misses per kilo instructions）**，挑 MPKI 最高的四个：

- a. fotonik3d
- b. mcf
- c. roms
- d. cactuBSSN

MPKI 高意味着什么？

每执行 1000 条指令就有很多 cache miss，需要频繁访问内存。

这就更容易变成 **内存带宽/延迟受限**，能更清晰看到 CXL 带来的变化。

他们还会：

- 跑多个相同 benchmark 实例
- 或混跑两个不同 benchmark
这就是在模拟真实服务器“多租户/多任务 co-running”场景。

总结：

延迟敏感（p99）：Redis（ μ s 级）+ 微服务（ms 级）+ page cache（I/O 路径）

→ 看 CXL 延迟增加会不会把 SLA 搞崩

吞吐导向（带宽压力）：DLRM + 高 MPKI 的 SPECrate

→ 看 CXL 的额外带宽能不能转化成系统吞吐提升（以及为什么 naive 分页会翻车）

latency

memo

clflush：把某些缓存行从 CPU 缓存里踢出去（确保下一次访问会 miss）

mfence：内存栅栏，保证前面的 flush 动作都完成了，别“还没刷完就开始测”

让你测到的是“去内存”的延迟，而不是误测到缓存命中。

rdtsc: 读一个“CPU 时钟计数器”，像秒表

memo for four memory access instruction types:

- (1) temporal load (ld), (2) non-temporal load (nt-ld), (3) temporal store (st), and (4) non-temporal store (nt-st).

temporal (普通): 默认会走缓存 (cacheable)，适合“马上还要用”的数据

non-temporal (非时间局部性): 倾向于绕开缓存/减少 cache 污染，适合大流式数据

执行 16 条内存访问指令 back-to-back，访问 16 个随机地址，区域是 cacheable。

并行随机延迟 + 指令语义差异

测到的是：设备延迟 + 链路延迟 + 排队/并行度影响而不是 pointer-chasing 那种纯串行。

指令类型	是否看 Cache	Miss 时的路线	CXL 性能表现	核心价值
ld	是	CXL——Cache——CPU	受延迟限制严重（可能出现缓存污染）	模拟普通程序为
nt-ld	否	CXL——CPU	较稳定	测纯读取带宽
st	是	CXL——Cache——修改——CXL	最慢 (有读回开销)（可能出现缓存污染）	模拟复杂对象改
nt-st	否	CPU——合并缓冲区 (Write Combining Buffer)——CXL	最快 (直接写入)	测纯写入极限

Latency

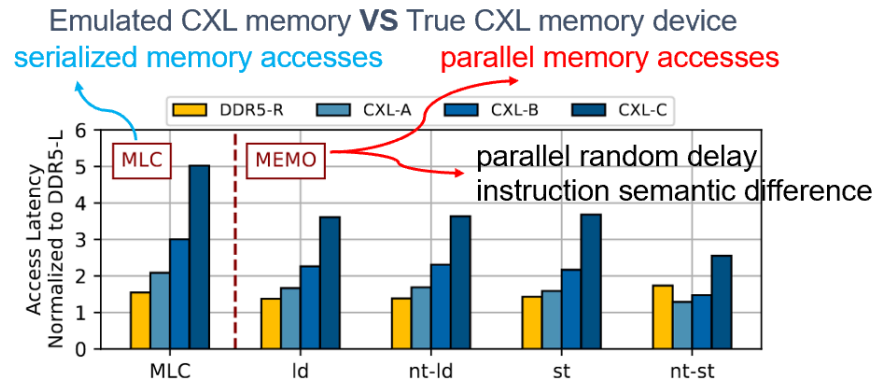


Figure 3: Random memory access latency of various memory devices (DDR5-R, CXL-A, CXL-B, and CXL-C), measured by Intel MLC and memo.

Instruction	Cache Policy	Data Path (on Miss)	CXL Performance Impact
ld	Use	CXL → Cache → CPU	High Latency: Sensitive to CXL distance.
nt-ld	Bypass	CXL → CPU	Raw Read BW: Stable, no cache pollution.
st	Use	CXL → Cache (RFO)	Slowest: Requires "Read-before-Write."
nt-st	Bypass	CPU → CXL	Fastest: Direct write; ignores old data.

1. 并行度高的访问模式，更能利用 CXL/UPI 这类链路的吞吐与流水能力，因此“平均每次访问”的成本可能比串行模式小很多。串行测延迟会“把链路能力浪费掉”，并行测延迟才能看出 full-duplex 的优势。

串行依赖 (pointer chasing)：CXL 额外延迟会更明显 → 更受伤

可并行 (很多 outstanding misses)：CXL 的额外延迟能部分被隐藏，结果更取决于带宽和队列 → 可能没你想象那么糟，甚至能用上“额外带宽”

2. **真 CXL 的延迟高度依赖 CXL 控制器设计。**比如 CXL-A 的 ld 延迟只比 DDR5-R 长 35%，但 CXL-B/CXL-C 能到 $\sim 2\times/\sim 3\times$ 。甚至同为 DDR4，CXL-C 还能比 CXL-B 慢很多。
3. **emulated CXL (remote NUMA)** 要做一致性检查：本地 CPU 需要通过 UPI 跟远端 CPU 确认 cache coherence；而且请求还要穿过远端 CPU 内部的长互连到达内存控制器。这些都会增加开销，且 CPU 核心越多越明显。相比之下真 CXL 没有 CPU 核心/缓存参与，CPU 反而能在片上用更快的硬件结构做一致性检查；设备端控制器到内存控制器路径也更短

论文在 O3 后面还解释了一个你会非常容易混淆的点：**store 的延迟通常比 load 更高**，特别是 temporal store (st)，因为 Intel CPU 有 **write-allocate** 策略：store miss 时，CPU 会先把对应的 64B cacheline 从内存读到 cache（相当于隐式做了一次 load），再写进去。

这会把“穿越 UPI/CXL 链路 + 一致性检查”的成本放大（来回次数更多），所以 st 更惨。

cache write-allocate policy

当你做 store，但目标 cacheline 不在缓存里 (LLC miss)：

- aa. CPU 先把那 64B 从内存读到 cache（这一步等价于隐式执行一次 load）
- ab. 再在 cacheline 里改你要写的部分
- ac. 之后再写回 (writeback)

所以 st 的代价往往是：

一次读 + 一次写 + 一致性检查 (ddr5)

而 ld 主要是一次读 + 一致性检查

nt-st 的行为会因内存区域是否 cacheable 而不同，他们的实验是在 cacheable 区域做的。

Bandwidth

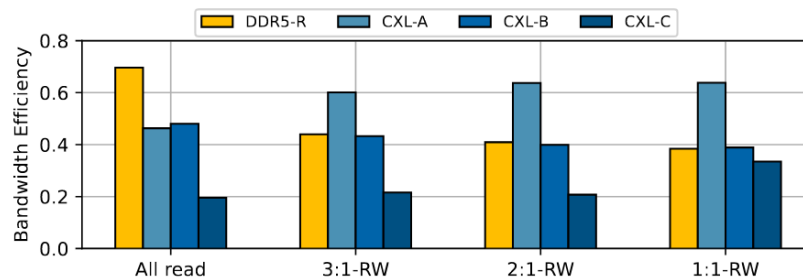
代码块

```
1  理论上限 (DRAM tech: DDR5-4800/DDR4-3200/2400)
2      ↓ (决定 Max BW)
3  实际带宽 = min( DRAM上限,
4                  控制器吞吐效率,
5                  链路/协议开销,
6                  一致性检查/写分配开销,
7                  排队与调度 )
8      ↓
```

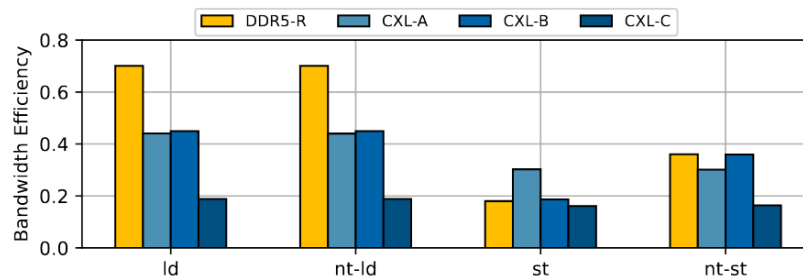
Bandwidth efficiency = Measured BW / Theoretical Max BW

这样做的意义是：

让不同 DRAM 技术（DDR5-4800、DDR4-3200、DDR4-2400）之间可以公平比较。



(a) MLC with various read and write ratios



(b) memo with different memory instruction types

Figure 4: Efficiency of maximum sequential memory access bandwidth across different memory types.

理论值：

DDR5-4800: **38.4 GB/s**

DDR4-3200: **25.6 GB/s**

DDR4-2400: **19.2 GB/s**

Figure 4a 显示（全读）效率：

DDR5-R: 70% CXL-A: 46% CXL-B: 47% CXL-C: 20%

代码块

- 1 All read (纯读)
- 2 → 主要考验“读路径”的控制器效率 + 链路开销
- 3 → DDR5-R (传统CPU内存控制器/成熟路径) 更能吃满
- 4 → CXL-A/B (设备端控制器) 读路径效率不如DDR5-R
- 5 → CXL-C (FPGA soft IP) 最差

为什么 CXL-A (也是 DDR5-4800) 却比 DDR5-R 低很多？

论文的推测是：

CXL-A 的内存控制器（device-side controller）处理 read 的效率不如 DDR5-R 的控制器，所以吃不

满。

CXL-A for ‘2:1-RW’ is 23 percentage points higher than that of DDR5-R.

推测，CXLA 的内存控制器在处理交错的内存读写访问方面，设计得比 DDR5-R 和 CXL-B 的内存控制器更高效。

代码块

- 1 读写交错 (R/W mixed)
- 2 → 很考验控制器的“读写切换/调度/合并”能力
- 3 → CXL-A 可能在 mixed traffic 上做了更好的硬件优化
- 4 → 所以它在 2:1-RW 这种现实常见模式下效率更高

可能原因：**st** 在 cache miss 时会隐式做一次 ld（write-allocate），所以 st 本质是读+写混合（你前面已经懂了）。

代码块

- 1 st（普通写）
- 2 → write-allocate：先读64B cacheline 再写
- 3 → 等价于：读 + 写 + coherence check
- 4 → 流量更多、开销更多
- 5 → 带宽效率下降

CXL-A 在其它指令类型上反而更差，所以它在 st 上更好不一定是“真 CXL 的神奇属性”，更可能是 controller 的特定优化。

结论：

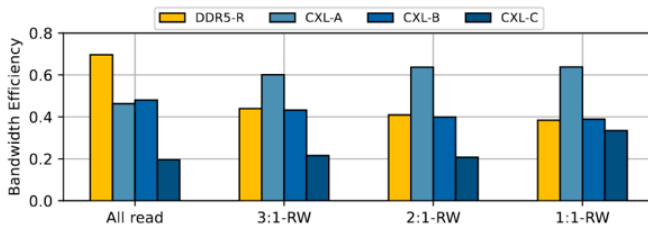
1. **CXL controller 是决定带宽效率的关键瓶颈**：DRAM 规格高不代表你能吃满（CXL-A vs DDR5-R；CXL-C 明明 DDR4-3200 但效率很差）。
2. **读/写混合模式会暴露 controller 设计差异**：CXL-A 读不强但读写交错强，说明 controller 策略不同。
3. **真 CXL 在 store 场景下更可能优于 remote NUMA 模拟**：因为 remote NUMA 的 coherence check 开销更重，st/nt-st 会放大差异。

代码块

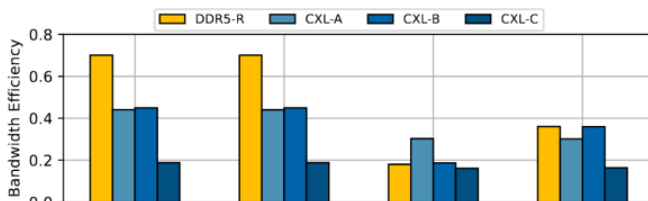
- 1 remote NUMA 模拟（DDR5-R）：
- 2 st = 隐式读 + 写 + 一致性检查
- 3 一致性检查要跨 socket（UPI）→ 更重、更拥塞

- 4 → st 的效率更低
- 5
- 6 nt-st (非时间写)
- 7 → 不做 write-allocate (不隐式读64B)
- 8 → coherence 压力也小很多
- 9 → remote NUMA 的额外开销不再被放大
- 10 → 所以真/模拟差距缩小

Bandwidth



(a) MLC with various read and write ratios



(b) memo with different memory instruction types

Figure 4: Efficiency of maximum sequential memory access bandwidth across different memory types.

CXL memory devices			
Device	CXL IP	Memory technology	Max. bandwidth
CXL-A	Hard IP	DDR5-4800	38.4 GB/s per channel
CXL-B	Hard IP	2× DDR4-2400	19.2 GB/s per channel
CXL-C	Soft IP	DDR4-3200	25.6 GB/s per channel

The bandwidth is strongly dependent on the efficiency of **CXL controllers**. (e.g. CXL-C: DDR4-3200)

CXL-A has weak read performance but **strong read-write interleaving performance**, which indicates that the **controller strategies** are different.

True CXL typically outperforms remote NUMA emulation in **store** operations.

Actual bandwidth =

**min(DRAM capacity limit,
controller throughput efficiency,
link/protocol overhead,
coherence check & write allocation overhead,
queueing & scheduling)**

Interaction with Cache Hierarchy

it loads data from the memory into a cache line in the CPU core' s (private) L2 cache rather than the (shared) LLC, in contrast to a CPU core with inclusive cache architecture.

当数据来自“远程内存”（包括 emulated CXL 和 true CXL）时，L2 淘汰的 cacheline 可能被放进**任何 SNC node 的 LLC slice**，而不是只在本 SNC 内。——访问 CXL 会“打破隔离”

- SNC 模式下，总 LLC 约 60MB（4 个 SNC node 总和）
- 他们用 32MB buffer:
 - 大于单个 SNC 的 LLC
 - 小于全部 4 个 SNC 的 LLC

然后测随机访问平均延迟：

- buffer 在 CXL-A: **41 ns**
- buffer 在 DDR5-L: **76.8 ns**

这不是“CXL 更快”，是“CXL 让你更容易命中更大的 L3” CPU cores accessing CXL memory can benefit from larger effective LLC capacity than the CPU cores accessing local DDR memory in the SNC mode.

Impact of Using Cxl Memory on Application Performance

如何分配page比例？

numactl 的三种模式

Linux Memory Allocation Policies

Mode	Allocation Strategy	Research Purpose
Membind	Force allocation on a specific node.(CXL/Local Dram)	Establish performance baselines (best/worst case).
Preferred	Fill local node first, then overflow to remote nodes.	Simulate capacity expansion in real-world apps.
Interleaved	Distribute pages evenly (50/50) across chosen nodes.	Leverage aggregate bandwidth of multiple nodes.
Weighted Interleaved	Allocate by specific percentage (e.g., 75:25).	Find the performance "tipping point."

模式	策略逻辑	实验目的
绑定模式 (Membind)	强制在特定节点上分配所有内存。	确立性能基准（最好与最差情况）。
优先模式 (Preferred)	优先填满本地节点，空间不足再使用远端。	模拟现实中作为容量扩展的使用场景。
交织模式 (Interleaved)	在选定节点间均匀 (50/50) 分布页面。	利用多节点的聚合带宽。
加权交织 (Weighted Interleaved)	按特定百分比比例（如 75:25）分配页面。	寻找性能大幅下降的**“临界点”**。

作者选了最均衡的 CXL-A 设备，通过 Linux numactl 把应用内存页按比例分配到 CXL（OS 视其为 NUMA node），并利用内核 patch 精确控制 CXL 页占比（如 25%/50%），以研究页比例对应用指标（p99/吞吐）的影响；同时开启 SNC 只保留 2 条 DDR5 + 1 条 CXL 通道，让单个 CXL 设备在总带宽中有足够贡献，构造与未来多 CXL 设备平台相似的通道比例。

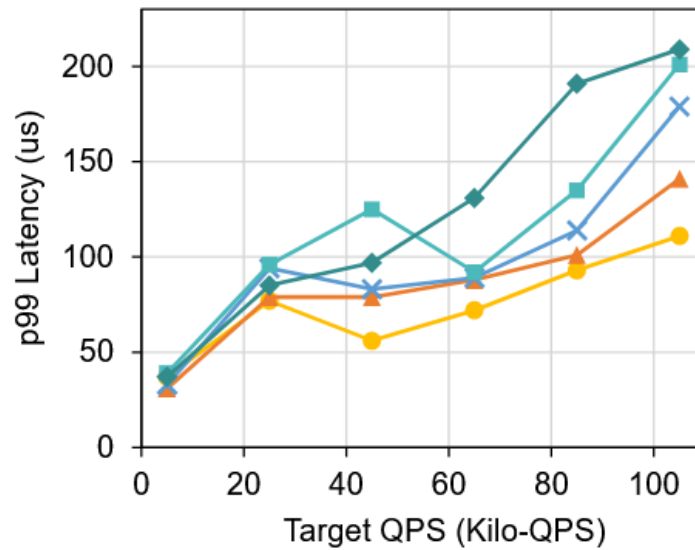
Latency

1. Redis with YCSB workload A (50% read and 50% update)

对象是：Redis 进程自己用的内存页（存 KV 数据、哈希表、对象等）

READ（读）：给我 key，对应 value 读出来（Redis 里就是 GET key）

UPDATE（更新）：给我 key，把这个 key 的 value 改掉（Redis 里类似 SET key new_value，或对 value 做修改）



(a) [Redis] YCSB-A (RD:UPD = 5:5)

结论1：100% 放 CXL，p99 明显比 100% 放 DDR 高，而且 QPS 越高差距越大

结论 2：CXL 页比例越高，p99 几乎“按比例”上升（线性趋势）

结论 3：DDR100% 最稳、最低

Redis 一个请求本来就只有几百微秒甚至更低，里面很大一部分时间就是在等内存（随机访问）。

你把 miss 延迟变成 $\sim 2\times$ ，再把 QPS 拉高让队列变长，p99 就会被放大。

2. Redis+TPP.

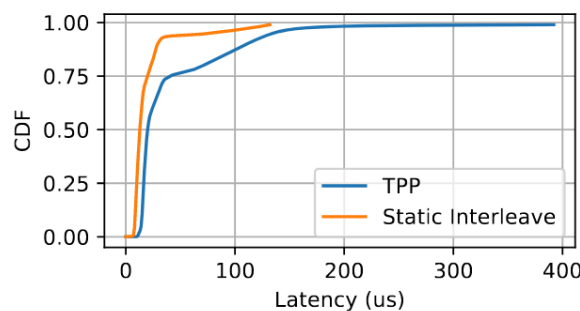
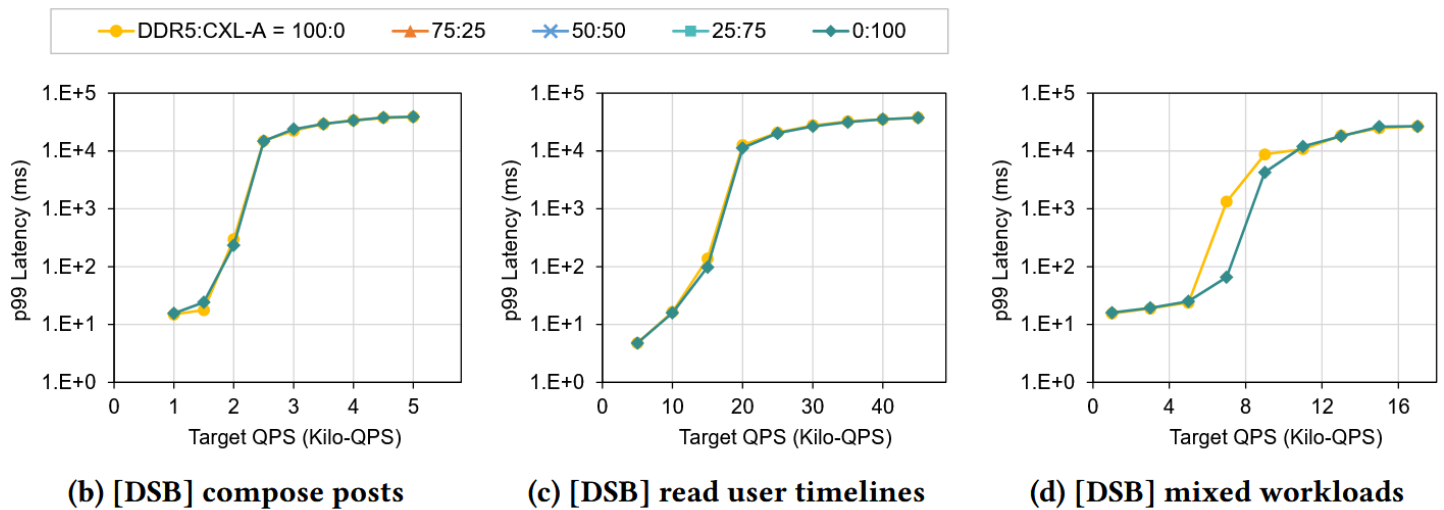


Figure 7: Impact of TPP on latency of Redis compared with statically allocating 25% of (random) pages to CXL memory. We show the distributions up to the p99 latency.

Redis + TPP（透明页放置）反而更差

因为TPP过程中有（1）page copy、（2）update the OS page table entries associated 反而导致了 latency增加

3. DSB



A. DSB 的 p99 对 CXL 不敏感：

原因 1：端到端延迟主要来自前端/逻辑组件（它们都在 DDR）

所以“数据库那边慢一点”在端到端里被摊薄了。

就像：整个链路 30ms，数据库多 0.1ms，影响很小。

原因 2：DSB 本来就是 ms 级延迟，不像 Redis μ s 级那么敏感

B. mixed workload 在 5K-11K QPS 之间，CXL100% 反而 p99 更低

作者解释：mixed 更“带宽密集”。

他们测到在吞吐饱和区间，平均带宽消耗：

- mixed：32 GB/s
- compose：7 GB/s
- read timeline：10 GB/s

当一个应用吃这么多带宽时，它的吞吐（吞吐 $\approx 1/\text{延迟}$ ）就对“可用带宽”敏感。

直觉解释：

mixed 在某段 QPS 下卡的是带宽，不是延迟。

CXL 虽然慢，但提供了额外总带宽，使系统更不容易带宽饱和，于是端到端延迟略低。

4. FIO page cache 实验里

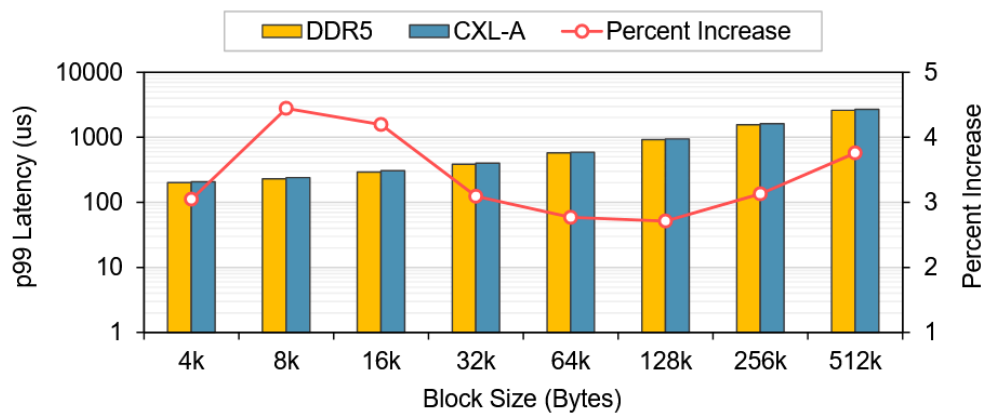


Figure 8: p99 latency values of FIO for various block sizes, and percentage values of increase in p99 latency by allocating page cache to CXL.

对象是：Linux 内核的 page cache 占用的内存页（缓存文件数据的页）

不同点：Redis pages 属于用户进程；page cache pages 属于内核缓存。

I/O latency $\approx$ 固定内核开销 + 数据搬运开销 + （如果 miss）SSD 开销

4KB：固定内核开销占比最大 $\rightarrow$ DDR/CXL 差异被淹没

8KB：固定开销被摊薄 $\rightarrow$ DDR/CXL 差异更明显

更大块：miss 更多 + SSD 开销变主导 $\rightarrow$ DDR/CXL 差异又变小

超大块：开始撞到 CXL 带宽瓶颈 $\rightarrow$ CXL 变差

总结：

(F1) μ s 级简单内存密集应用：只要放一点页到 CXL，p99 就按比例上升

原因：它们对“内存访问延迟”极其敏感。

(F2) 智能迁移策略可能更糟，因为迁移要 copy 页 + 更新页表，会带来额外开销，延迟敏感应用扛不住。

(F3) ms 级复杂应用：合理把部分页放 CXL 不一定增加 p99

Troughput

CXL supplements DDR bandwidth, 增加总带宽。

- **DLRM**：像“搬货公司”，路（带宽）不够 $\rightarrow$ 多一条路（CXL）就能多搬很多货 $\rightarrow$ 吞吐上升
- **Redis**：像“问答客服”，每个问题都要等一个关键回复（延迟） $\rightarrow$ 回复慢一点，整条链就慢 $\rightarrow$ 吞吐下降

对带宽密集型应用，naively 按 OS 默认把 50% 页放 CXL，可能比 100% DDR 吞吐更低——即使 DDR+CXL 的总带宽更高。

必须有一个动态策略，根据(1) 具体 CXL 设备的带宽能力(2) co-running 应用当前消耗的带宽自动配置“该给 CXL 分多少新页”

验证“CXL 打破 LLC 隔离”对应用到底是好是坏

caseA: 1 SNC node（只有 SNC-0 工作）

- 只有一个 SNC node（SNC-0）跑 8 个 DLRM 线程
- 另外 3 个 SNC node 完全空闲

Case B: 4 SNC nodes（四个 SNC node 都工作，制造 LLC 干扰）

- 每个 SNC node 各跑 8 个 DLRM 线程（总 32 线程）
- 只有 **SNC-0** 把 100% pages 放到 **DDR** 或 **CXL**（两种对照）
- 另外 3 个 SNC node（SNC-1/2/3）只用各自本地 DDR（不碰 CXL）

SNC-0 访问 CXL 时，原本可以把 L2 淘汰的 line 放到“任何 SNC 的 LLC slices”

- 所以在 1 SNC node（其他 node 空闲）时：
 - 等效 LLC = 几乎整颗 CPU 的 LLC（2-4× 大）
 - cache 命中率更高
 - DLRM 吞吐更高（因为 embedding reduction 很吃带宽/缓存）

但在 4 SNC nodes 时：

- 其他 3 个 SNC node 也在疯狂使用自己的 LLC slices
- 它们会不断把自己的数据塞进 LLC，同时驱逐（evict）别人的 line

其他 3 个 SNC node 虽然主要在自己的 LLC slices 内驱逐，但也会不可避免地驱逐掉很多属于 SNC-0 的 cache lines（尤其是 SNC-0 的 line 被“散落”到了所有 node 的 slices 里）。

结果：SNC-0 的有效 LLC 容量下降 → 吞吐下降。

SNC-0（CXL100%）会把自己 L2 淘汰的 line 扔进其他 node 的 LLC slices，破坏隔离，污染别人。

所以其他 3 个 SNC node 的性能也会被影响。

线程跨 SNC ≠ 内存就变远/LLC 就变共享

在他们之前测 DLRM 吞吐时（§ 5.2），如果 SNC-0 要跑超过 8 个线程，线程会被调度到其他 SNC nodes 的 CPU core 上。

但：

这些线程即使跑在 SNC-1/2/3 的核心上，也仍然访问的是 SNC-0 的 DDR 内存；并且因为这些 cacheline 来自 SNC-0 的 DDR，它们从这些核心的 L2 淘汰时仍会回到 SNC-0 的 LLC slices（不会跨到别的 slices）。

这句话的目的：

避免误以为“线程跑到别的 node 核心上，就一定会打破 LLC 隔离”。

实际上打破隔离的关键不是“线程在哪个 core”，而是“访问的是 CXL/远程内存还是本地 DDR”。

场景	SNC-0 用 DDR	SNC-0 用 CXL
只有 SNC-0 工作（1 SNC node）	LLC 隔离正常（只用自己 slices）	能利用更大 LLC（福利最大）
4 个 SNC node 都工作（4 SNC nodes）	各自隔离，互不污染	共享池子太挤：SNC-0 被驱逐更多 + 还污染别人

Caption 动态策略

对带宽密集型应用来说，CXL 可以提供额外带宽，但如果“傻瓜式”按 OS 默认把 50% 页放 CXL，吞吐可能反而下降。Naïvely used CXL memory considered harmful

所以 Caption 的目标不是“把热页都迁移到 DDR/把冷页都放 CXL”的经典 tiering，而是：

把 CXL 当成 **bandwidth expander**（带宽扩展器），动态找到“DDR : CXL 页比例”的最佳点，让系统总吞吐最高。

代码块

```
1  state: p    # 当前新页分配到CXL的比例
2  loop every T seconds:
3      read counters -> estimate Perf(p)    # 估计当前吞吐/性能
4
5  on new_page_allocation:
6      # 记住历史
7      Perf_old = recent_estimated_perf
8      p_old = p
9
10     # 试探：往CXL多分一点
11     p_try = p_old + Δ
12     allocate next pages using p_try for a short window
13     Perf_try = estimate_perf()
14
15     if Perf_try > Perf_old:
16         p = p_try    # 变好了，继续往这个方向
17     else:
18         p = max(0, p_old - Δ)    # 变差了，往回/反方向
```

Caption — a CXL-memory-aware dynamic page allocation policy

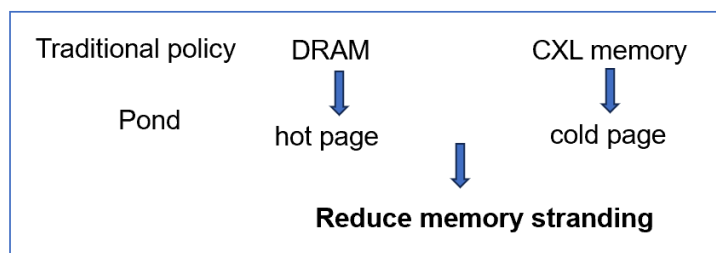
Step1: Determine the bandwidth of manufacturerspecific CXL memory devices.

Step2: Monitor various CPU counters, such as memory access latency experienced by (co-running) applications. Assess the bandwidth consumed by them at runtime.

Step3: Caption estimates memory-subsystem performance over periods.

When a given application demands an allocation of new pages, Caption considers the history of memory subsystem performance and the percentage of pages allocated to CXL memory in the past.

Step4: **Adjust the percentage of the pages** allocated to CXL memory to improve the overall system throughput using a simple greedy algorithm.



Maximize throughput

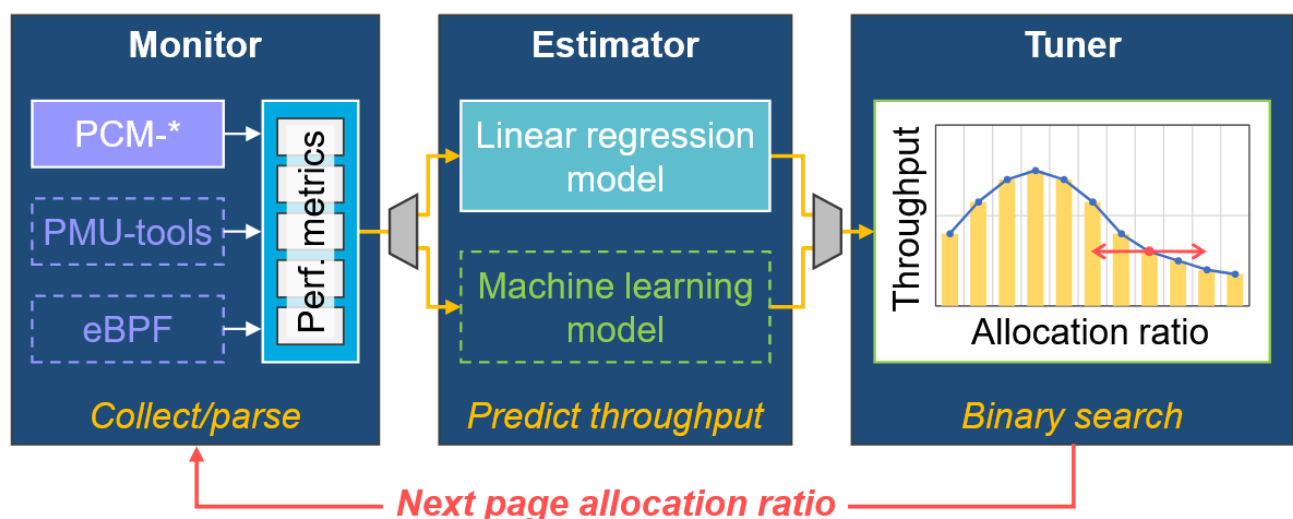


Figure 10: Overview of Caption. The components in dotted boxes can be used for better performance.

M1用 Intel PCM 采样 CPU counters (每 1 秒一次)

Table 4: CPU counters pertinent to memory-subsystem perf.

Metric	Tool	Description
L1 miss latency	pcm-latency	Average L1 miss latency (ns)
DDR read latency	pcm-latency	DDR read latency (ns)
IPC	pcm	Instructions per cycle

M2 estimate 用线性回归模型把 counters → “估计吞吐/性能” Y

训练一个线性模型：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots$$

- Y：估计的内存子系统性能（在这篇里你可以把它当 “吞吐代理值” ）
- Xn：各种 counter
- βn：线性回归学到的权重

M3 调参：用 greedy/控制算法调 “CXL 页比例”

变量

- p：当前 “新页分配到 CXL 的比例” （或等价的配置值）
- step：每次调整的步长（例如 9%），不会无限变小，防止停在 “几乎不动” 导致无法探索

核心逻辑

1. 看性能变好了没：比较 “当前周期估计性能 Y_now” 和 “上一周期 Y_prev”
2. 如果变好：说明上次方向对了，继续按同方向走（p += step 或 p -= step）
3. 如果变差：说明上次方向错了，反向走，但步子减半（step = step/2，p 反向调）

Algorithm 1: Caption tuning algorithm. *state*, *step* and *ratio* represent memory subsystem performance, unit of tuning page allocation ratio, and ratio of page allocation between DDR and CXL memory.

```
1 while true do
2   curr_state  $\leftarrow$  estimator()
3   if curr_state < prev_state then
4     curr_step  $\leftarrow$  prev_step  $\times$  (-0.5) // reverse
5   curr_ratio  $\leftarrow$  prev_ratio + curr_step
6   check_ratio_bound()
7   set_ratio(curr_ratio)
8   if new allocations then
9     prev_state  $\leftarrow$  curr_state
10    prev_step  $\leftarrow$  curr_step
11    prev_ratio  $\leftarrow$  curr_ratio
12  sleep(tune_interval)
```

`$\leftarrow$` : 赋值（把右边结果放进左边变量）

`if ... then` : 条件判断

`// reverse` : 注释

`while true do` : 无限循环

Evaluation

虽然 Caption 基于 DLRM 分析构建，但它监控的 L1 miss latency、DDR read latency 和 IPC 都是通用的内存子系统健康指标；作者通过在未预先 profiling 的情况下混跑 SPEC-Mix 与 Redis+DLRM 来验证其泛化性，并用 Pearson 相关系数衡量模型输出与真实吞吐的同步性。由于 Caption 的 greedy 算法只依赖性能变化方向，Figure 12 中大多为正的 Pearson r 表明估计模型能提供足够可靠的方向信号，从而支撑在线调参。

最大的优点是在于提出了 cxl memory 不等于 remote NUMA node（但早期 cxl 设备简陋，其他大部分结论是作者推测，证明不有力），并给出一定的真是 cxl device 的数据验证文章的结论，文章提出的动态页分配策略更适用于带宽密集型应用中，对于减少延迟倒没有什么说法，这个策略倒是很简单

1. CXL 版本/拓扑/设备数量都偏早期

- 用的是 **CXL 1.1** 时代平台（SPR/Genoa 的早期 CXL.mem），很多关键能力（比如更复杂的 fabric、更多设备、switch、动态容量等）在后续版本才完善。

- **只有 1 个 CXL memory device**，所以他们不得不用“开 SNC + 降 DDR 通道数”的方式去模拟未来通道比例。这个方法聪明，但也会引入新的变量（比如 SNC 带来的 LLC 行为变化）。换句话说：**他们的工作负载结果可能强依赖这个“人为构造”的平台比例**，外推到多设备、多 switch 的真实 CXL pool 需要更谨慎。

2. SNC 相关现象（LLC 隔离被打破）很有意思，但可能“平台特定”

他们发现 CXL/远程内存访问会破坏 SNC 的 LLC 隔离，导致“等效 LLC 变大”。这是个亮点，但也可能是：

- 特定厂商、特定代际、特定实现细节（cache routing/eviction policy）导致的行为。
如果换平台（不同 CPU、不同 BIOS 配置、不同 SNC 模式实现），这个效应是否存在/多大，不一定保持一致。
所以这点更像“非常重要的观察”，但作为“普遍规律”还需要跨平台验证。

3. Caption 的适用面偏窄：主要优化吞吐，不保证 SLA/尾延迟

Caption 基本是为 **memory-bandwidth-intensive** 的吞吐优化而设计。它对 **p99** 这种延迟目标并没有优化承诺，甚至可能冲突：

- greedy 调比例可能会在某些阶段短时间变差（探索成本），对 SLA 很敏感的在线服务不友好。
- 他们用的目标函数是“内存子系统性能/吞吐代理”，不是“p99 约束下最大吞吐”。这在云场景里往往不够。

4. Caption 的“控制粒度”比较粗：只调全局比例，难以做 per-app / per-cgroup 的精细控制 本质更像“全局旋钮”。

但真实混部场景里，最需要的是：

- 哪个应用该多用 CXL，哪个应用必须留 DDR
- 哪类页（匿名页/文件页/特定 mmap 区）该放哪儿
- 按 cgroup / VM / container 分级管理

他们虽然用 counters 做了估计，但控制动作还是偏粗。后续工作可以做：

- **按应用/容器分 ratio**
- **按 VMA/页类型分策略**
- 甚至结合硬件的 hot-page 监控（他们也提到现在拿不到 CXL 相关 counters）。

CXLfork

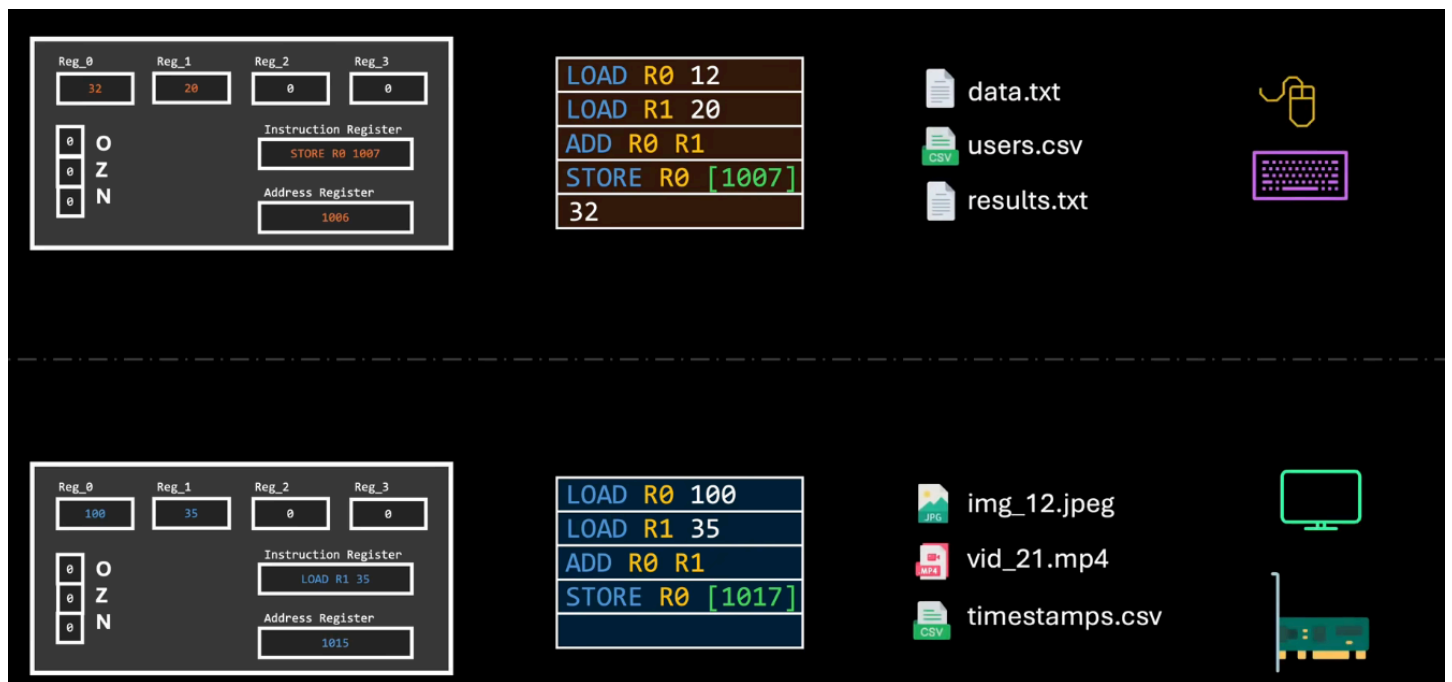
在“另一台机器”上，快速复制出一个正在运行的进程/容器实例，并且尽量：

- 不把内存一页页打包、传输、再解包（少“搬家”）
- 能让很多实例共享同一份相同的只读状态（省内存）
- 真要修改时再分裂出私有副本（写时才复制）

备用知识：

集群（Computer Cluster）是指通过高速网络连接起来的多台计算机（每个称为“节点”）。

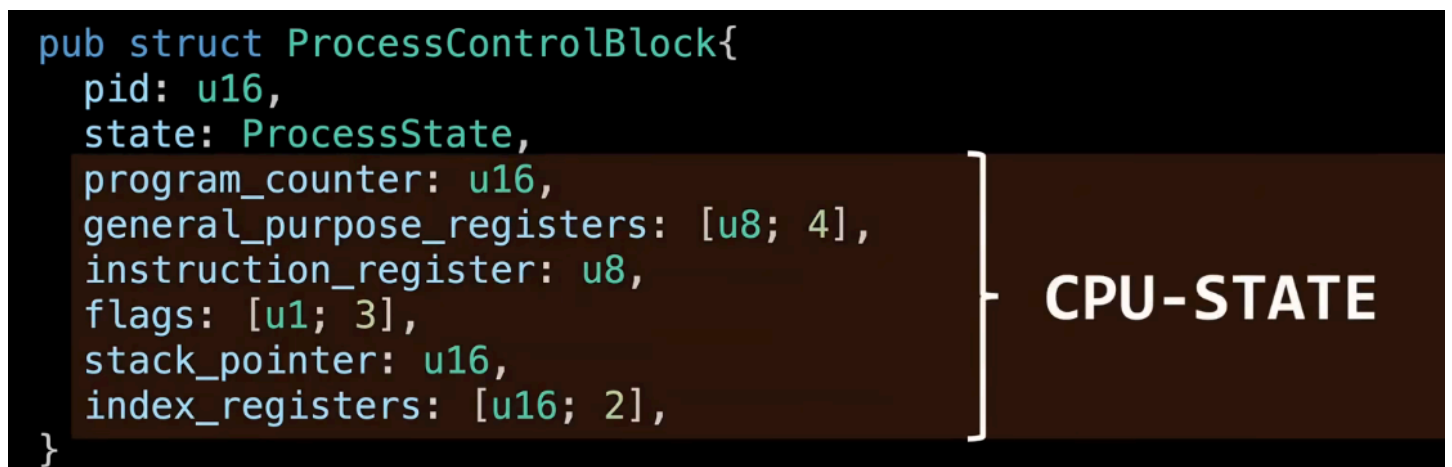
进程（Process） 进程可能有什么？ <https://www.youtube.com/watch?v=LDhoD4IVElk>



把进程想成：一个正在运行的“程序实例” + 它的全部运行现场。

运行现场包括：

- 程序代码（机器指令）
- 数据（堆/栈/全局变量）
- 打开的文件、网络连接
- CPU 寄存器状态（现在执行到哪条指令）
- 以及一大坨 OS 维护的管理信息



进程都有自己的地址空间，但是由于并发可能导致一个进程的地址空间被另一个进程的地址空间占用，在进程控制块（PCB）中可以通过设置memory_limits来设置局部空间限制一个进程的地址空间，具体可以看linux源代码


```
pub struct ProcessControlBlock{
    pid: u16,
    state: ProcessState,
    program_counter: u16,
    general_purpose_registers: [u8; 4],
    instruction_register: u8,
    flags: [u1; 3],
    stack_pointer: u16,
    index_registers: [u16; 2],
    memory_limits: [u16; 2],
    io_devices: Vec<IO_Device>,
    open_files: Vec<File>,
}
```

结构体里还有的参数

```
parent: Arc<ProcessControlBlock>,
children: Vec<Arc<ProcessControlBlock>>
```

线程 (Thread)

线程是进程内部的“执行流”。一个进程可以有多个线程共享同一片内存。

进程状态转换： 进程在生命周期内会在不同状态间切换。

- **就绪 (Ready)：** 具备运行条件，等待分配 CPU。
- **运行 (Running)：** 正在占用 CPU 执行指令。
- **阻塞 (Blocked/Waiting)：** 等待某个事件（如 I/O 完成）而暂时停止运行。

调度算法

为了让多个程序“同时”运行，系统需要切换进程。常见的调度策略包括：

- 先来先服务 (FCFS)
- 短作业优先 (SJF)
- 时间片轮转 (RR)： 每个进程运行一小段时间，轮流执行。

虚拟内存 (VA) 程序看到的地址：每个进程都以为自己独占整个内存：**每个进程看到的地址（虚拟地址）是假的。**

虚拟地址 = 虚拟页号 (VPN) + 页内偏移 (Offset)

虚拟页号 (VPN, Virtual Page Number)： 用于在页表中定位具体的“页”。

页内偏移量 (VPO, Virtual Page Offset)： 用于在确定的物理页内定位具体的字节。

由于页的大小一般是4KB（即 2^{12} 字节），因此虚拟地址的低 12 位通常就是偏移量 (Offset)，

物理地址 (PA)：真正的 DRAM/CXL 里的地址

中间靠 **页表 (Page Table)** 来翻译。

MMU (Memory Management Unit)：这是一个硬件单元，负责将 VA 转换为 PA。

TLB (Translation Lookaside Buffer)：转换检测缓冲区，本质上是 MMU 内部的一个**地址转换高速缓存**。它存储了最近使用的 VPN 到 PPN（物理页号）的映射。

- **命中 (Hit)：**直接返回 PA，极快。
- **缺失 (Miss)：**需要去内存里的“页表”查找，速度较慢。

分页 (Paging)：将内存划分为固定大小的页，减少外部碎片。

交换 (Swapping)：当物理内存不足时，将暂时不用的数据置换到硬盘（Swap 分区/虚拟内存文件）中

页 (Page)

内存按固定大小切块（常见 4KB=4096 字节。，也可能有 2MB/1GB 的大页）。

页表不逐字节映射，而是**按页映射**。**这一整页的虚拟地址范围 → 映射到哪一页物理内存。**

原因：

1. 隔离：进程 A 不能乱读写进程 B
2. 简化：每个进程都以为自己从 0 开始一大片连续空间
3. 高效：可以把不常用页换出去（swap）、可以共享只读页（库代码）

多级页表 (Multi-level Page Tables)

在 64 位系统中，如果使用一级页表，页表本身会占用巨大的内存。因此系统采用“树状结构”：

1. 顶级页表 (PGD/L4)：指向下一级页表的地址。
2. 中间级页表 (PUD, PMD)：逐级缩小范围。
3. 页表项 (PTE, Page Table Entry)：最终指向物理页的起始地址。

PTE 里不仅有地址：还包含了权限位（可读、可写、可执行）、存在位（P，表示是否在物理内存中）、访问位（A，用于 LRU 算法）等。

fork 最核心的意思是：创建一个新的进程

- 一个程序正在运行 → 叫**父进程**
- 调用 `fork()` → **复制出一个几乎一模一样的新进程** → 叫**子进程**，子进程是父进程的**副本**（内存、文件描述符等基本都复制）
- 两个进程**同时、独立**往下运行
- 父子进程**PID 不同**，是两个独立进程
- `fork()` 会返回两次：
 - 在父进程：返回**子进程的 PID**

- 在子进程：返回 0

代码块

```
1  pid_t pid = fork();
2
3  if (pid == 0) {
4      // 这里是子进程在跑
5  } else if (pid > 0) {
6      // 这里是父进程在跑
7  }
```

现实中 fork 怎么能这么快？—— Copy-on-Write（写时复制，CoW）

如果真的把内存全部复制一遍，fork 会慢到爆炸。

所以 OS 用 **CoW**：

- fork 时：父子进程先**共享同一份物理页**，都标成“只读”
- 当某一方第一次写这页：触发页错误（page fault）
- OS 才复制这一页，给写入方一个私有副本

remote fork（跨机）：把“子进程”放到另一台机器上。难点是：另一台机器怎么得到这堆“进程状态”（内存页 + OS 管的元数据，比如页表、VMA 等）。

Remote fork 复制的到底是什么？

1. 用户态内存（user memory）**大**而**多**

- 代码段、库、堆、栈
- 进程 mmap 的匿名页、文件映射页
- 这些通常占大头（MB~GB）

2. OS 维护的进程状态（OS-managed state）**复杂且和 OS 内部结构强相关**。

超级关键的部分：

- 地址空间元数据（VMA 列表、权限、mmap 区域信息）
- 页表（page tables）
- 线程/寄存器上下文（PC 指令指针、SP 栈指针等）
- 文件描述符（打开了哪些文件/管道/Socket）
- 信号处理器、定时器、调度信息等

这些东西不恢复完整，进程就“长得像，但跑不起来”。

传统Remote fork：

CRIU(Checkpoint and Restore in Userspace)

代码块

```
1  冻结进程
2      ↓
3  抓取进程状态（把用户态内存页读出来把 OS 状态（VMA、页表、FD 等）读出来）checkpoint
4      ↓
5  序列化serialization
6      ↓
7  写入文件
8      ↓
9  传输transfer
10     ↓
11 反序列化deserialization
12     ↓
13 恢复进程restore
```

它序列化的东西非常多：

OS state（VMA、页表、打开文件、namespace、寄存器…）+ **进程内存页**。

The first approach (CRIU) decouples the checkpointed state from the OS instance that created it

checkpoint 结果和创建它的那个 OS 实例“解耦”了。

这个“解耦（decouple）”特别重要。什么叫“和 OS 实例解耦”？

意思是：保存下来的 checkpoint 文件，不再依赖“原来那台机器的内核里的那些具体地址/对象”。

- 原来进程运行时，OS 里有很多复杂对象和指针关系
- CRIU 会把它们“翻译”成一种可移植的文件格式
- 所以这个 checkpoint 文件就像一个“标准化快照包”
- 任意节点都可以拿去恢复

all references (pointers or dependencies) to the checkpoint-initiating OS have been stripped

所有指向原来那个 OS 的引用、依赖、指针关系都被剥离掉了。

所以 CRIU 的 checkpoint 文件具备三个好处：

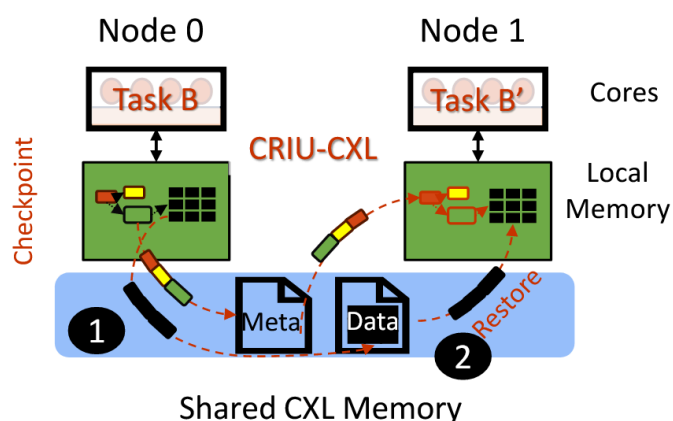
1. 可以存起来
2. 可以传给任何节点
3. 任何节点都能恢复

代价：

process restoration suffers from significant deserialization overhead

恢复的时候，反序列化开销非常大。

serialization deserialization这两个步骤非常慢，网络传输大量内存

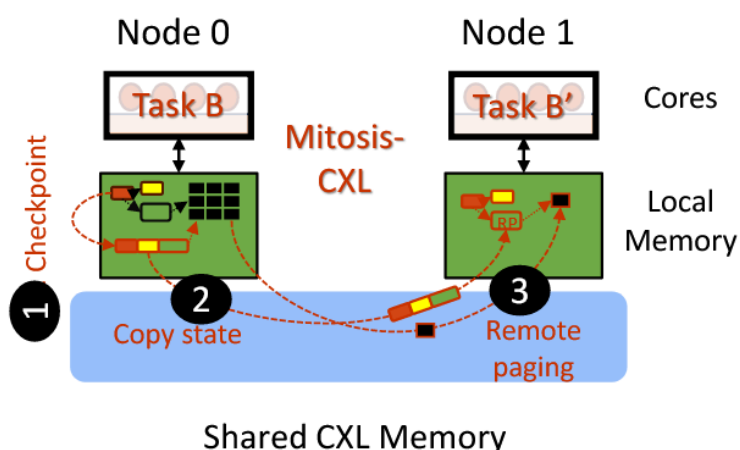


(a) CRIU over CXL

Mitosis (lazy copy)

代码块

```
1  checkpoint:
2    本机做 shadow copy + 序列化 OS state
3
4  restore:
5    传 OS state → 反序列化建子进程（目标节点）
6
7  run:
8    子进程访问某页 → 特殊 page fault → RDMA 拉该页 → 本地复制一份
```



(b) Mitosis over CXL

The second approach (Mitosis) skips serialization for process data, but couples the checkpointed state with the node and process that created it.

Mitosis 不对 process data 做那种重型 serialization，

但它把 checkpoint 状态 绑定在创建它的那个节点/父进程上

为什么会“绑定”？

因为 Mitosis 的做法不是把状态变成独立文件，而是：

- 保留在原节点内存里
- 子进程运行时需要某页，就从父节点按需拉过去

所以 checkpoint 状态实际上还“活在”原节点的上下文里，没真正独立出来。

处理流程：

- 目标机器创建子进程“壳”（有 VMA、页表框架）
- 用户态内存不立刻全复制
- 页表里标记：这些页“暂时缺失”
- 目标进程运行时一访问某页 → 触发 page fault
- page fault 处理函数用 RDMA 从源机器把那一页拷过来
- 更新页表，继续跑

出现的问题：

1. parent process cannot exit until all its descendants (i.e., remote children) have terminated, 父进程不会退出，直到所有子进程退出
2. the node where the parent process and the checkpoint reside, acts as a point of failure 父进程变成单点故障
3. The parent node acts as a point of congestion and its network uplink is a potential bottleneck. 原节点变成拥塞点，因为都从父进程拉取页
4. Such a design also limits the potential to exploit shared-memory fabrics

最终还是复制：访问到的页必须拷贝到本地 DRAM（传统假设内存不共享）

page fault 太多：每次缺页都要陷入内核处理，还要走网络/RDMA，像“边跑边搬家”，抖动大

跨机延迟和带宽开销：页是 4KB 颗粒度，频繁搬页会很贵

checkpoint/restore：先把进程状态“存一份快照”（checkpoint），以后在某台机器把这个快照恢复成一个可运行进程（restore）。

序列化（serialization）（传统 checkpoint）

很多系统（例如 CRIU）会把进程状态转成某种可移植格式：

- 写文件/写网络
- 再读回来重建

VMA：虚拟内存区域（Virtual Memory Area）

进程的地址空间不是随便一坨，它分成很多段：

- 代码段 (text)
- 数据段 (data/bss)
- 堆 (heap)
- 栈 (stack)
- mmap 映射的库、文件、匿名内存等等

OS 用 VMA 记录“哪段地址范围是什么属性（可读/可写/可执行）”。

Page fault: 页错误（其实是“缺页/权限异常”的总称）

当进程访问某个地址，页表说：

- 没映射 → 缺页
- 没权限（比如 CoW 写只读页）→ 保护异常
就会陷入内核处理。

mmap: 把一段内存“映射”进进程地址空间

mmap 可以把：

- 文件内容映射成内存
- 也可以映射匿名内存
- 也可以映射设备内存（重要！）

CXLfork “restore 不拷贝”，核心就是**把 CXL 里的快照 mmap 进来**。

pointer / kernel data structure: 为什么跨机器会炸？

内核里维护进程状态的结构体里充满“指针”（指向内存里别的结构）。

如果你把这些结构拷贝到另一块内存位置，指针就可能失效。

CXLfork 摘要里提到的 **rebasing**，就是在说：

“我把这些结构搬家后，要把里面的指针修一遍，让它们改成指向新家地址。”

这点非常关键：这也是为什么传统 checkpoint 往往选择序列化——因为序列化天然会“重建对象关系”，不用修指针；但代价是慢。

zero-serialization: 几乎不用把进程状态“打包成文件/字节流”再解包

zero-copy: 恢复时尽量不把内存页拷贝回本地 DRAM，而是直接在 CXL 共享内存上“就地使用”

Serialization（序列化）: 把复杂内存/数据结构变成可传输的字节流（比如写文件、发网络），另一端再反序列化回来。

Deduplication（去重）: 多个实例共享同一份相同内容（尤其是只读部分），节省内存。

Rebasing（重定位/重基址）: 把“指针指向的地址”改成“在 CXL 共享内存中也成立的表示”，让不同节点/不同 OS 实例能正确访问。

Copy-on-Write（写时复制）: 读共享；写的时候才复制一份私有页。

Tiering（内存分层）：本地 DRAM 更快更贵；CXL 共享内存更慢但能共享省容量。在两者之间做放置策略

Serverless（无服务器计算）

如阿里云中的serverless函数计算FC（function computing）

它不是“没有服务器”，而是“你不用管服务器”。

你只提供一段代码（通常叫“函数”），平台负责：

- 把它放在哪台机器上跑
- 什么时候起一个实例、起几个实例
- 负载高了自动扩容，负载低了缩回去
- 计费按“用多少算多少”（比如执行时间、调用次数、内存占用）

痛点 A：流量不稳定、难做容量规划

很多业务是“潮汐流量”：

- 白天高峰、夜里低谷
- 活动期间暴涨
- 某些 API 平时很少用，但偶尔会突然被调用

传统模式你得提前准备机器容量：

- 准备少了 → 爆了、宕机
- 准备多了 → 大部分时间闲置浪费钱

Serverless 的逻辑是：让平台替你承担“预测+扩缩容”的麻烦。

痛点 B：运维成本高

你不用自己管：

- 机器挂了怎么办
- 自动扩容怎么做
- 部署、镜像、补丁、容量、监控（至少大部分不用你管）

对于很多团队，这很香：**更快把功能上线。**

痛点 C：更细粒度的按用量付费

对于“偶尔跑一下”的任务（比如图片缩放、PDF 转换、定时任务、Webhook 处理），传统租一台服务器非常亏。

Background

解决问题：cold start

多个请求 → 平台需要创建多个 **实例**，这些实例来自 **同一个代码**，初始化后的 **很多内存页是完全相同的**

如果没有共享机制，每台机器都会 **复制一份**

如果有共享内存，可以 **去重这些相同的页**

每个实例本身就是一个进程（或者一组进程）

在 serverless 或多实例系统中，大量实例拥有相同的内存内容（不同节点上的进程实例往往包含大量相同的只读内存页）。传统系统由于内存隔离会产生大量重复数据，如果利用 CXL 共享内存，可以让多个节点共享这些只读页，通过 copy-on-write 保证隔离，从而减少内存复制并加快实例启动。

一个进程的状态不仅仅是内存页，还有很多 OS 数据结构：

页表

VMA

文件描述符

内核对象

线程状态

这些东西里有：大量指针，当你把进程状态放到 **CXL memory** 时：这些指针会失效

所以论文提出了一个关键技术：rebasing——修复这些指针

共享 CXL 内存 + 去重 + 分层控制

代码块

```
1  Checkpoint (在节点0)
2    [进程状态(数据页 + 大部分OS状态)]
3      | 直接memcpy到 CXL共享内存 (尽量不序列化)
4      v
5    [CXL中的快照] + 做rebasing (把指针/引用改成“指向CXL里”的形式)
6      |
7  Restore (在节点1/2/...)
8    直接把CXL快照 mmap 到新进程地址空间 (不拷回本地)
9      |
10   读：可能直接从CXL读 (缓存命中就快)
11   写：CoW，写的时候把那一页拉回本地再改
```

挑战1：怎么 checkpoint 不靠 serialization?

→ “as-is memory copies” 内存直接拷贝，把进程数据和大部分 OS 维护状态直接拷到 CXL。

1.5：OS 结构里有指针，跨机器会失效怎么办？

→ **rebasing**：遍历这些 OS 结构，把指针改成“按 CXL 地址空间可重定位”的形式，让别的 OS 实例也能解引用。

挑战2：怎么让多台机器共享同一份快照还不乱？

→ 默认“直接映射”快照，**只读共享**；写的时候 CoW，保证共享快照仍然干净可复用。

默认情况下，远程节点并不拷贝数据，而是直接将自己的地址空间指向 CXL 内存里的那个备份（映射）。所有节点共享同一份只读数据，实现了“集群级去重”。

按需修改：只有当某个节点想要修改数据时，才会通过“写时复制”在本地内存创建一个副本。

为什么还要“tiering”（分层放置）？

直接在 CXL 上跑，虽然“启动/恢复”很快，但 **运行时可能变慢**（CXL 访问延迟更高）。

所以 CXLfork 提供 **细粒度分层控制**：哪些页留在 CXL（共享省内存），哪些页尽量放回本地（性能更好）。

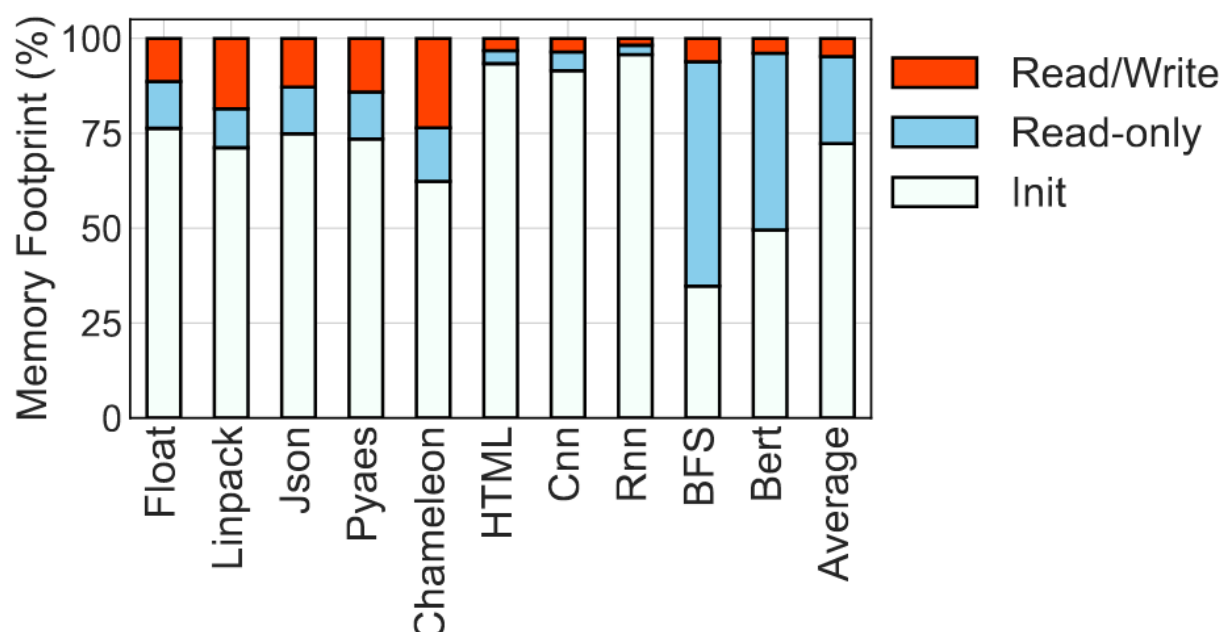


Figure 1. Breakdown of the memory footprint of different FaaS functions. Init and Read-only data dominate.

“Init 很少访问”是否对所有函数都成立？

有些函数初始化后会频繁访问大模型权重/大字典/缓存结构，Init 可能变“热”。

如果 Init 变热，放 CXL 会拖慢。

“工作集小→cache 能挡住”是有条件的

- 多核并发、多个函数同时跑，会互相挤 cache
- cache 能挡住多少取决于工作集、访问模式、LLC 容量、干扰情况
所以他们后面如果有“多租户/高并发”实验才更有说服力。

How to Store the Checkpointed State?

CXLfork distinguishes between private and global process state.

区分：是否依赖进程外部 OS 公共结构

private state all the data and metadata uniquely owned by a process

is self-contained——it does not link to OS structures outside the process.修复指针（rebasing）

代码块

```
1  源节点：
2  A -> B -> C
3
4  复制到 CXL：
5  A' -> B' -> C'    （通过 rebasing 修好）
```

global state requires explicit unplugging (detach) and plugging (attach) for checkpoint and restore respectively, to keep the OS metadata consistent.

代码块

```
1  源节点：
2  process -> old OS global object
3
4  checkpoint:
5  detach this relation
6
7  目标节点 restore:
8  process' -> new OS global object    (attach)
```

CXLfork 的关键不是简单地“把 checkpoint 放进 CXL”，而是把 checkpoint 变成一种**既与原节点 OS 解耦、又能以原生内存形态存在于共享介质中的状态表示**。

为实现这一点，它将进程状态区分为 private state 和 global state：前者通过 rebasing 修复内部指针，后者通过 detach/attach 维护跨 OS 实例的一致性。

CXLfork具体流程

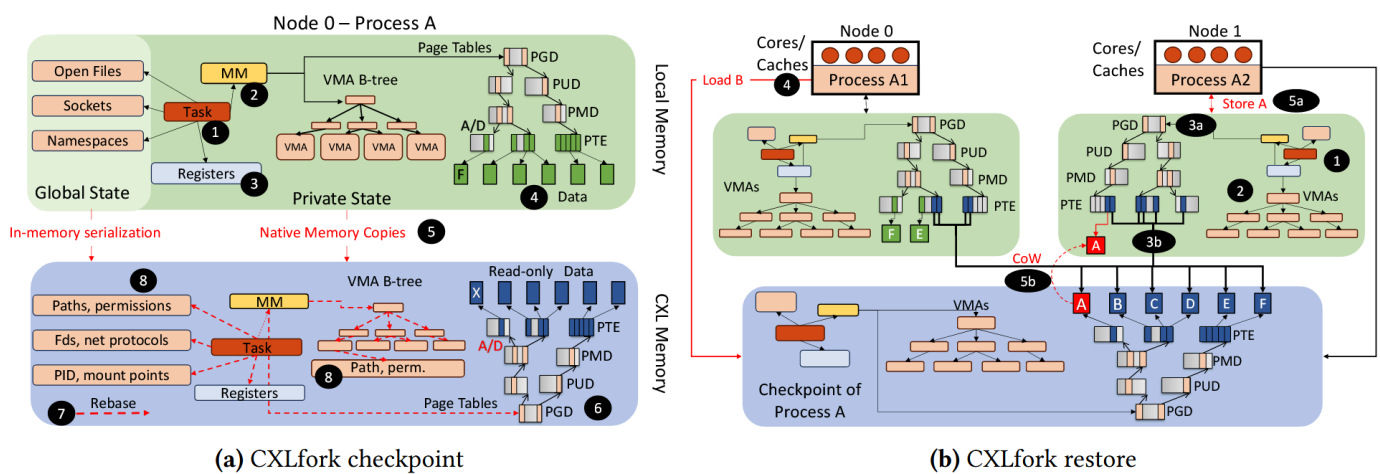
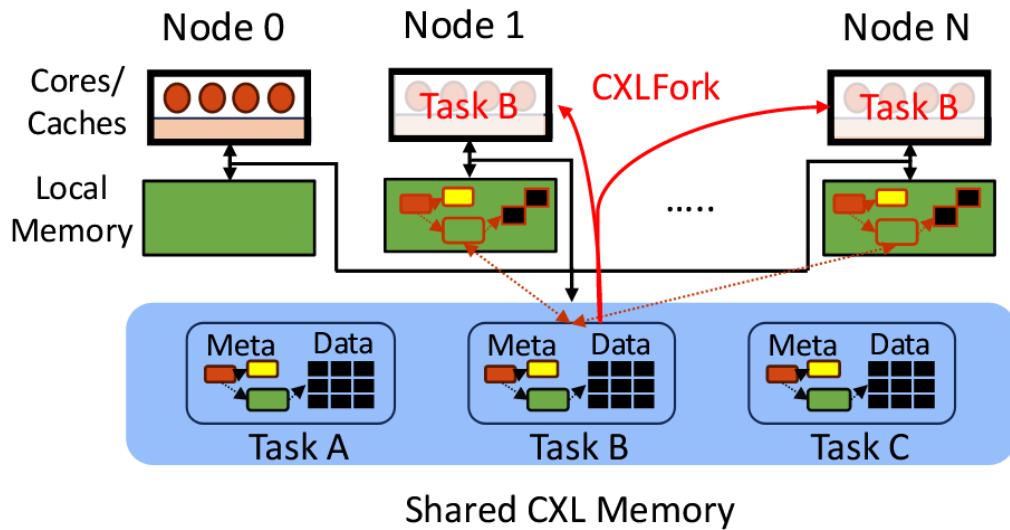


Figure 4. Workflow of CXLfork operations: (a) checkpoint and (b) restore.

CXLfork Checkpoint

checkpoint前

代码块

```

1  process VA
2    ↓
3  page table
4    ↓
5  node0 DRAM page F
6
7  kernel structs
8  task → mm → vma
9        ↑
10     DRAM address

```

checkpoint后

```

1 代码块 process VA
2      ↓
3  checkpoint page table
4      ↓
5  CXL page X
6
7  kernel structs
8  task' → mm' → vma'
9          ↑
10         CXL address

```

Task structure 进程在 Linux 内核里的“主控块”。

Memory descriptor (MM) 这个进程整个虚拟地址空间的管理总表

MM 包括：**VMA tree**、**page tables**

VMA tree VMA = Virtual Memory Area

一个进程的地址空间不是一整坨，而是分成很多区域：

- 代码段
- 数据段
- 堆
- 栈
- mmap 的库文件
- 匿名映射等等

VMA tree 就是在记录这些区域：

- 起始地址
- 结束地址
- 权限 (R/W/X)
- 是匿名页还是文件映射
- 对应哪个 backing file

CPU register contents

- 程序执行到哪了 (PC / instruction pointer)
- 栈指针
- 通用寄存器值
- 其他架构相关寄存器

读共享，写迁移。

checkpoint 的 metadata + data 都放在 CXL→restore 时直接映射，不重建、不大规模拷贝→读共享页时直接读 CXL→写时触发 CoW，把页迁到本地 DRAM→这样只读状态可以长期共享，cache 会自动缓存热点部分

也就是：

- **read-only / 没写的页**：留在 CXL，大家共享
- **一旦写**：触发 CoW，把页迁到本地 DRAM 变成私有页

这和本机 fork 的思路其实一脉相承，只不过：

- 本机 fork 共享的是本机物理内存页
- CXLfork 共享的是 CXL 里的 checkpoint 页

tiering policy

migrate-on-write（默认）

migrate-on-access

hybrid tiering

“reconfigured during restore”

在 checkpoint 里不把这些状态当成“绝对不能变”的东西，restore 到新节点时，根据新节点环境重新设置。

几类可重配置状态：CPU affinity，NUMA affinity（**NUMA 亲和性**），namespace configuration，cgroup configuration

namespace 是 Linux 容器隔离的一个核心机制。

常见 namespace 有：

- PID namespace
- mount namespace
- network namespace
- UTS namespace
- IPC namespace
- user namespace

CXLfork Restore

Zero-Copy, Seamless State Sharing

restore 这个时刻，不把 checkpoint 的那些数据页整体拷回目标节点本地 DRAM，先直接映射 CXL 上那份页，再运行，恢复阶段不进行 bulk copy（大规模数据复制）

workflow:

1. CXLfork first **creates a new process on the new node** that calls CXLfork-restore
2. map the checkpointed data, as read-only, into the new process address space

A. 不复制 process data，数据页不拷贝。

B. 可能复制 page table entries

页表项（PTE）本身是 metadata，不是 process data。

最朴素的方法是：

- checkpoint 时页表已经被改好了，指向 CXL 页
- restore 时把这些 PTE 拿到本地，作为子进程本地页表的一部分
- child 继承原进程的虚拟空间使用，child 的虚拟页号和 parent 一样，所以 PTEs 可以直接 copy

private state / 数据页

代码块

```
1  checkpointed private state in CXL
2      ↓
3  映射到 child 地址空间
4      ↓
5  只读共享
6      ↓
7  读：直接从CXL取（cache miss时）
8  写：CoW fault → copy到本地DRAM → 本地写
```

global state

代码块

```
1  checkpointed lightweight serialized metadata
2      ↓
3  在目标节点 redo operations
4      ↓
5  reopen files / sockets
6  restore file mappings
7  rebuild file structs / callbacks
8  restore mount points / PID namespaces
```

相较于直接复制PTEs到远端本地更efficient的方法：

```

代码块
1 child local upper VMA nodes
2   ↓
3   attached CXL VMA leaves
4
5   child local upper page-table nodes
6   ↓
7   attached CXL PTE leaves
8   ↓
9   CXL data page

```

checkpoint state = 数据页 + OS 元数据（页表、VMA 等）checkpoint后的”数据”：

1) 数据页（data pages）

heap 页、stack 页、私有 file-backed 页、代码/库页的副本

2) 页表叶子（PTE leaves）

这不是数据内容本身，而是：

“某些虚拟页现在映射到哪些 CXL 数据页”的映射表它们也是 checkpoint 后保存在 CXL 里的元数据。

3) VMA 叶子（VMA leaves）

这也不是数据内容本身，而是：

“地址空间有哪些区域、权限是什么、对应什么 mapping”的区域描述它们同样是 checkpoint 后保存在 CXL 里的元数据。

尽管CXLfork实现了消除read fault，但是在write fault上触发CoW时

1. Data moving要花时间

2. The overhead of TLB shutdown / coherence

TLB（快表） 硬件缓存，在 CPU 内部，专门存最近用过的虚拟地址→物理地址映射。

Page Table（页表） 数据结构，在内存里，保存整个程序所有虚拟地址→物理地址的映射。

优化

```

代码块
1 检查 checkpointed PTE 的 D bit
2  ↓
3  如果某页很可能会被写
4  ↓
5  提前 prefetch 到 local DRAM
6  ↓
7  真正 store 时避免/减少 CoW fault
8  ↓

```


CXLfork tiering

策略	什么时候把页搬回本地 DRAM	优点	缺点
migrate-on-write	写的时候	restore 最快、共享最多	读热点页若常 miss 可能慢
migrate-on-access	第一次访问就搬	运行时更像本地内存	restore/运行中 fault 多，本地内存占用大
hybrid tiering	只有“热页”访问时搬	兼顾共享与性能	需要热页识别机制

migrate-on-access

代码块

```

1  第一次访问某页
2      ↓
3  触发 CXL fault
4      ↓
5  把页从 CXL 拷到本地 DRAM
6      ↓
7  设置 child 的 PTE
8      ↓
9  以后访问本地页
  
```

hybrid tiering：根据热度决定是否分配到本地dram

- A bit = **Accessed bit** 这个页最近被访问过

A bit = 0，说明这页不算热（至少没显示频繁被访问）

→ 访问时 **不搬回本地**

→ 继续留在 CXL

A bit = 1，说明这页被访问过，可能是热页

→ 访问时 **搬回本地 DRAM**

- D bit = 这个页被写过

备选方案：

一种替代方案是，在 restore 时同步预取所有 A=1 的页。

但是：拉长 restore tail latency（恢复尾延迟），让 restore 不再是“轻量 attach”，提前搬很多也许暂时还不会访问的页

CXLporter

新建一个实例instance**250–500 ms + 130 ms**：

(1) 函数自身初始化**250–500 ms**：Python runtime 启动、库加载、模型权重初始化、用户态数据结构建立

(2) 容器创建**130 ms**：Docker/container 创建、network 配置、namespace / cgroup 设置

减少时间的开销：

用 **checkpoint + CXLfork restore** 避免函数初始化

用 **ghost containers** 避免容器创建

workflow：

(1) takes appropriately-timed checkpoints of functions

第 16 次调用后 checkpoint

不能立即启动，要等到steady state，

在第一次调用之后，就会清掉函数本地页表里的 A/D bits。checkpoint 里的 A/D bits 反映的是：**steady-state 的访问模式**

(2) maintains an object store of checkpoints

CXL fabric <user, function>—>CID = checkpoint identifier

代码块

```
1 <alice, bert_infer> -> CID_42
2 <bob, image_resize> -> CID_17
```

(3) maintains a pool of ghost containers

Ghost Containers = 已经配置好、但里面没有函数实例的空容器

为什么需要ghost containers？因为Container Creation创建新容器开销：**约 130 ms固定**，使用ghost containers可以抵消这个Container Creation过程

- network setup
- namespace
- cgroup

传统和ghost containers对比

代码块

1	普通的	Ghost Containers
2	请求来了	请求来了
3	↓	↓

4	创建新容器		找一个现成空壳容器
5	↓	VS	↓
6	部署函数		通过 control socket 发 restore request
7	↓		↓
8	初始化函数状态		在这个空壳里 CXLfork 恢复函数

(4) controls the CXLfork tiering policies

默认 migrate-on-write, SLO 压力大 → hybrid, 内存太高 → 禁止更多 hybrid

什么时候触发 restore?

serverless runtime 决定要部署一个函数时, CXLporter 先去 object store 查这个函数有没有 checkpoint

有 checkpoint: → 用 CXLfork-restore

没有 checkpoint: → 回退到普通 cold start

(5) dynamically adjusts keep-alive windows.

keep-alive: serverless runtime 通常会把空闲函数实例在内存里保留几分钟。

原因是: 未来如果很快又来请求, 可以直接复用, 避免 cold start。

问题: keep-alive 当然能减少 cold start,

但它也会导致:

- 很多 idle function 长时间占内存
- 降低系统整体密度
- 加剧 memory pressure

做法: 把 keep-alive window 动态缩短到 10 秒

CXL Hardware Setup

总:

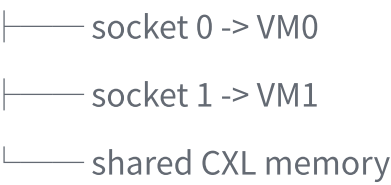
Experimental Setup

- **Dual-socket** Intel Sapphire Rapids host
 - 64 cores total, 64MB L3 / socket, 128GB DDR5 / socket
- Intel Agilex 7 FPGA CXL memory device
 - 16GB DDR4, attached as a **CPU-less NUMA node**
- Two VMs (one per socket) emulate a **2-node cluster with shared CXL memory**
- Measured CXL round-trip latency: **391 ns**
- Linux v6.6 + QEMU/KVM; QEMU+SST for latency sensitivity studies
- **Workloads:**
 - FunctionBench CPU and memory functions
 - HTML, BFS, Bert
- **Evaluation Scenarios:**
 - **Scenario 1:** Performance of CXLfork
 - compare **CXLfork**, **CRIU-CXL**, **Mitosis-CXL**, unsandboxed (no containers)
 - **Scenario 2:** Performance of CXLfork Bursts with CXLporter
 - Azure serverless traces
 - ghost containers for CXLfork and Mitosis-CXL
- **Metrics:**
 - the average and tail latency of function execution
 - the amount of local memory consumed.

硬件：一台 **双路 Intel Sapphire Rapids** 服务器

因为他们只有一台 host，所以在两个 socket 上各起一个 VM，用这两个 VM 模拟“两节点分布式机器”

物理 host



外接一个 **Intel Agilex 7** FPGA 的 CXL memory device

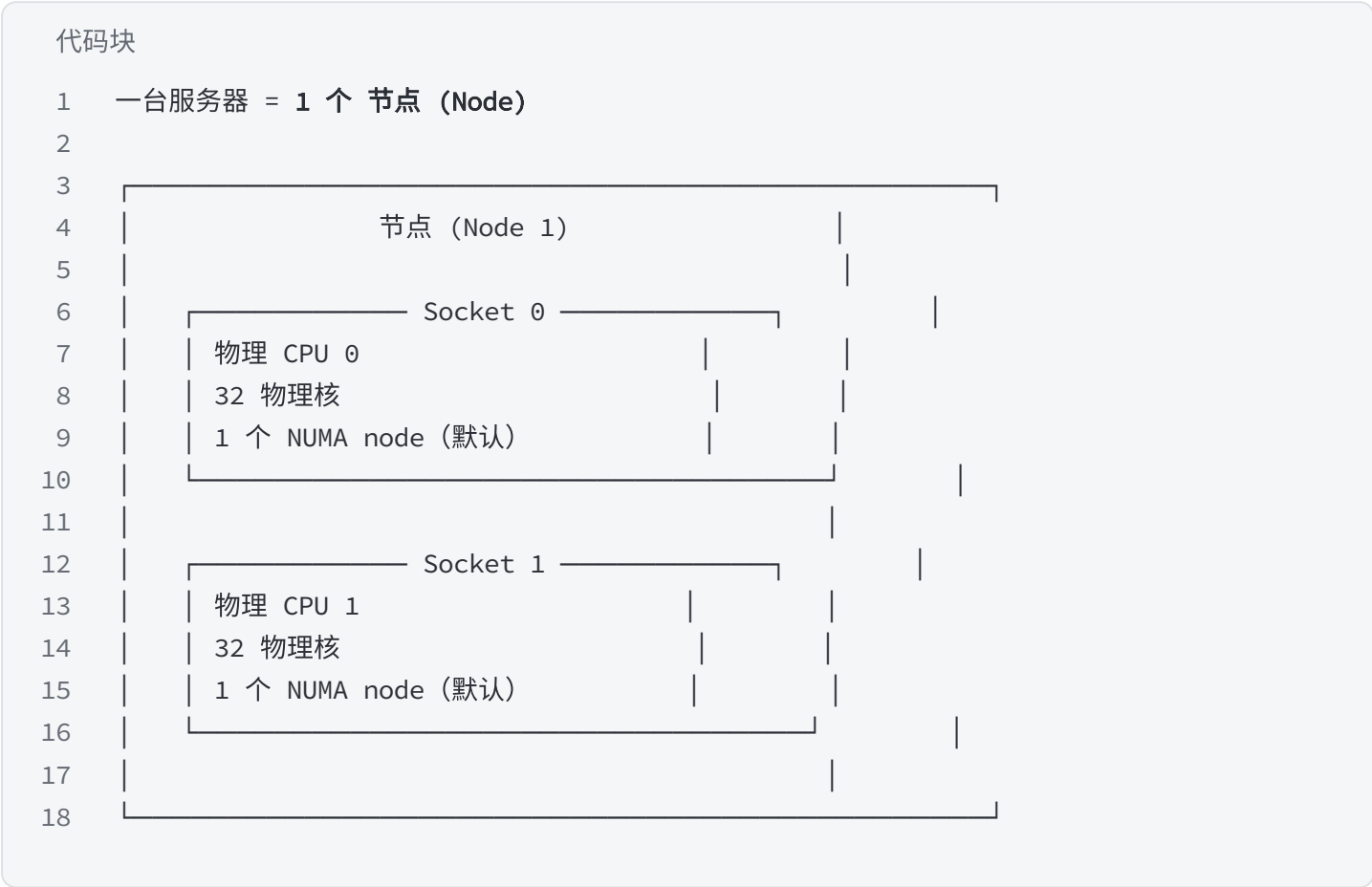
仿真：

- 再用 **QEMU + SST** 模拟不同 CXL 延迟
- 用来回答“如果 CXL 设备延迟不是现在这台机器这样，会怎样”

硬件	具体	作用
一台 双路 Intel Sapphire Rapids 服务器	1. 64MB L3 cache	host
双socket，总共 64 核	2. 128GB 本地 DDR5	
	3. 支持 CXL v1.1 attached memory	

外接一个 Intel Agilex 7 FPGA 的 CXL memory device	CXL 设备不是某个现成商用“CXL 内存盒”， 而是一个 FPGA-based prototype device。	CPU-less NUMA node 一块可被主机访问的远端内存节点 (CXL.mem)
--	---	--

这个实验平台本身是 **CXL 1.1 attach memory** 的真实设备环境，不是完整的 CXL 3.0 fabric 集群。
这里只有一个服务器作为一个节点，但是由于是双路服务器可以做模拟远端节点



Mitosis-CXL 不是完整跨 VM/跨 node 的真实实现，而是同节点内模拟 remote paging over CXL
这套实验用真实的 CXL 原型硬件做底座，通过双 VM 近似双节点共享 CXL 的环境，再辅以 SST 仿真扩展延迟范围；在此基础上，作者分别评估了 remote fork 原语本身，以及它嵌入 CXLporter 后在真实 serverless trace 驱动下的 burst 场景表现，最终用延迟和本地内存占用来衡量“更快”和“更省”。

Evaluation

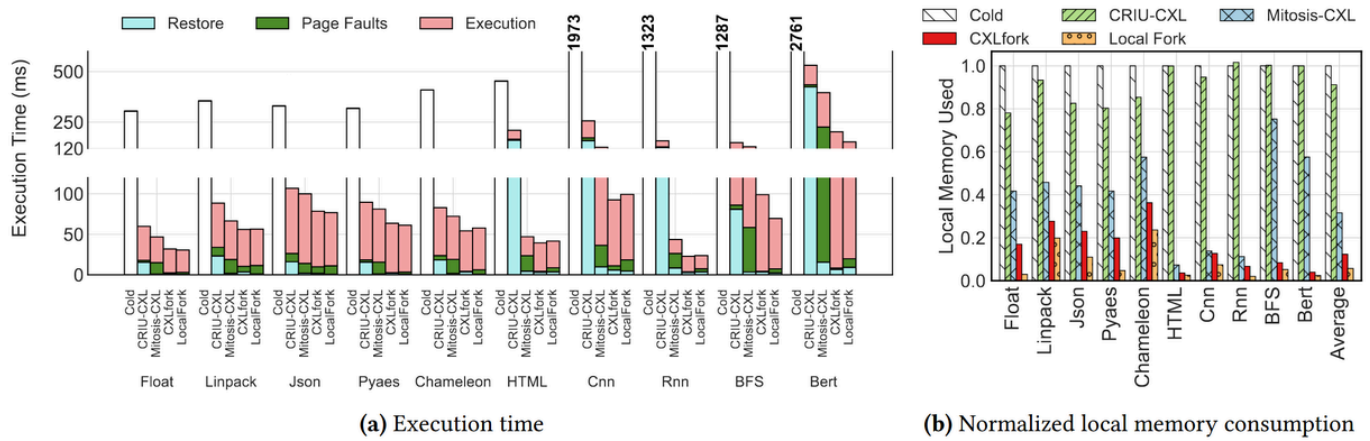


Figure 7. Remote fork performance under cold-start execution and normalized local memory consumption.

CXLfork restore latency: 1.2–6.1 ms

平均只比 LocalFork 慢 14%

比 CRIU-CXL 快 2.26x

比 Mitosis-CXL 快 1.40x

内存：

CXLfork 最省（避免复制到本地）

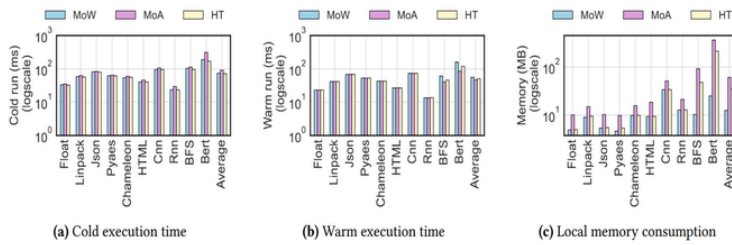
这是 Figure 7b 的 punchline：

- 比 CRIU-CXL 少 87%
- 比 Mitosis-CXL 少 61%
- 平均只需要 Cold 的 13% local memory

Execution with CXLfork requires only 13% of the local memory of a cold-started function on average, while speeding up cold execution by 11x on average.

MoW 的核心特点是：

最省内存、最利于共享，但对大读热点工作集不友好



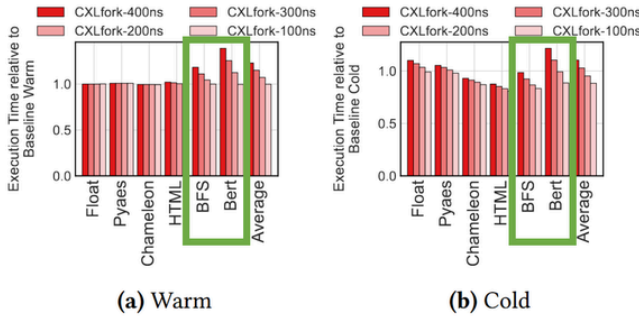
Tiering exposes a clear memory/performance trade-off

MoW: best memory efficiency, possible warm slowdown

MoA: best warm execution, **+250%** memory

HT: balanced compromise using checkpointed **A** bits

Figure 8. CXLfork tiering policies and their trade-offs between cold and warm execution time, and memory consumption.



small working-set workloads → latency insensitive
large working-set workloads → latency sensitive

Figure 9. Sensitivity of (a) warm and (b) cold function execution with CXLfork to CXL latency via simulation.

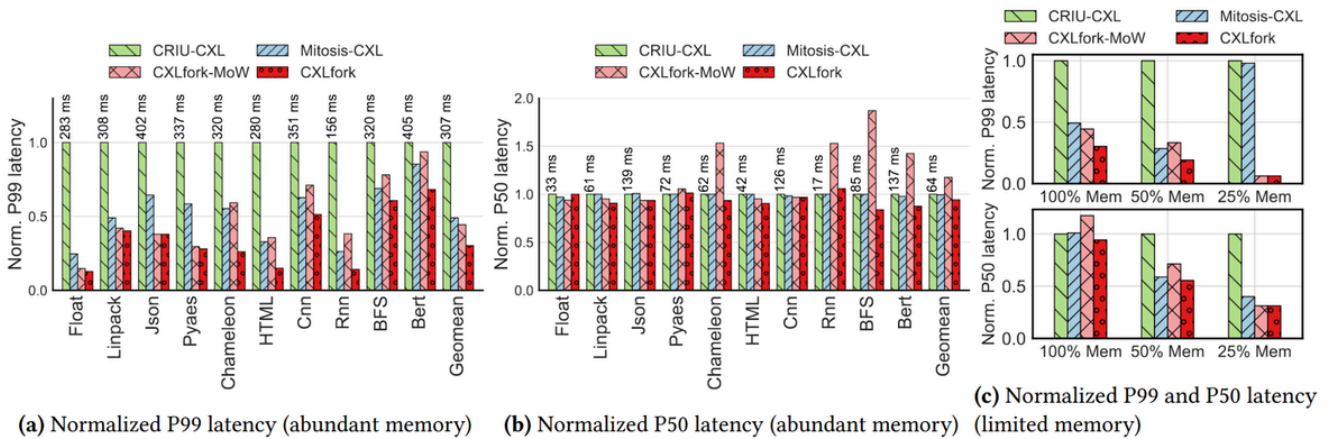


Figure 10. P99 and P50 latency of function execution under CXLporter with abundant and limited memory.

CXLporter + CXLfork / MoW / CRIU-CXL / Mitosis-CXL

•Under **abundant memory**:

- CXLfork-based CXLporter gives the best tail latency
- MoW is already strong; dynamic control improves further

•Under **limited memory**:

- CXLfork benefits become even larger
- Better sharing + lower local footprint preserve performance

Tigon: A Distributed Database for a CXL Pod

问题：传统分布式数据库里，跨主机事务太依赖网络消息和 2PC，导致延迟高、协调重。

核心思路：把真正被多主机活跃事务共同访问的那一小部分 tuple (CAT) 搬到 CXL 共享内存里，让事务 worker 直接在共享区上做同步和数据访问，从而减少跨主机协调。

摘要：For TPC-C and a variant of YCSB, Tigon achieves up to $2.5\times$ higher **throughput** compared with two optimized shared-nothing databases that use CXL memory as a transport and up to $18.5\times$ higher throughput compared with an RDMA-based distributed database.

传统分布式数据库里，跨主机事务太依赖“远程消息 + 2PC”；Tigon 想把其中一部分跨主机协调，改成在 CXL 共享内存上直接做锁和数据访问。

数据一开始分区放在各主机本地 DRAM；当某个 host 想访问不归自己拥有的数据时，它请求 owner host 把数据移到共享 CXL memory。Tigon 把这种被不同主机上的活跃事务并发访问的元组集合叫做 **CAT (Cross-host Active Tuples)**，并把它维护在 CXL memory 里。跨主机同步用的是放在硬件 cache-coherent 区域里的 latches 和 locks。

备用知识：

table 就是数据库里的“表”。

一个数据库里会有很多表

每个表里有很多行 (row / tuple)

Tigon 先按 **table** 组织数据，再对每张表做 **partition** (分区)

CXL pod CXL Pod 处于 SMP 和分布式系统之间

A CXL pod includes a small number of machines (e.g., 16) that connect directly to a shared CXL memory module via multiple ports (called a multi-headed device [2] (MHD)).

核心点在于 **Directly connect (直接连接)**。传统的网络是通过交换机 (Switch) 连接，而 Pod 内部通过 **MHD** 实现点对点直连。

MHD (Multi-Headed Device):

- **原理:** 一个存储控制器暴露多个物理 PCIe/CXL 端口。
- **现状:** 文中提到的 **SK Hynix Niagara 2.0** 是目前工业界的标杆，支持 8 个主机。

SMP (Symmetric Multi-Processing):

- **背景:** 传统的双路或四路服务器就是 SMP。它们的限制在于主板布线复杂度和信号干扰，很难做到 16 路以上。

特性	SMP (单机多路)	CXL Pod (MHD)	Distributed System (集群)
连接方式	处理器间总线 (UPI/Infinity Fabric)	CXL 直接连接	以太网/RDMA 交换机
延迟	极低 (数十 ns)	低 (数百 ns)	高 (微秒级)
一致性	全硬件自动维护	部分硬件/软件协同	纯软件维护 (分布式协议)
扩展性	受限 (通常 < 8 节点)	中等 (约 16 节点)	极高 (成千上万节点)

inter-host hardware cache coherence

CXL 3.0 引入了多主机共享内存的能力。为了保证不同主机看到的内存数据是一致的，CXL 设备（如 Switch 或 Memory Pooling Device）必须充当“协调者”。它通过 **Back-invalidations (反向作**

废) 机制来通知主机：你缓存里的数据失效了，请更新。而实现这一点的核心硬件组件就是 **Snoop Filter (窥探过滤器)**。

CXL 1.1/2.0: 主要支持 `CXL.cache` 和 `CXL.mem`，但主要针对**单主机**与设备之间的关系。

CXL 3.0: 引入了 **Fabric 架构**，支持多主机共享内存。这使得“跨主机一致性”成为了刚需，也由此产生了文中提到的硬件开销问题。

Area Budget: 面积预算。这是硅片设计 (IC Design) 中的专有名词，指芯片上能留给某个功能的物理空间。

Snoop Filter

在多核处理器中，为了保证缓存一致性，通常有两种做法：**广播窥探 (Broadcast Snooping)** 和 **目录机制 (Directory-based)**。

- **Snoop Filter** 是一种优化后的目录。它记录了当前哪些数据块被哪些 CPU 缓存了。
- **对比分析：**如果没有它，每当有一个写操作，系统必须向**所有**主机发送广播询问，这会造成严重的带宽风暴。有了它，只需要向“确实缓存了该数据”的主机发送精准通知。

Pasha架构 的精髓在于**动态性**：

- **静态分片：**初始状态下，数据按 Key 分配给 Owner 主机。
- **冲突触发移动：**如果 Host A 想写 Host B 的数据，Pasha 不会让 A 发起远程调用，而是**把数据从 B 的办公室挪到会议室 (CXL)**。
- **优势：**这种“数据就近”原则最大限度利用了本地带宽。只有在发生“跨节点争用”时，才付出 CXL 的延迟代价。

为什么现在的分布式数据库延迟降不下来？

“目前的瓶颈主要在于**通信协议栈的深度**。传统的 TCP/IP 甚至 RDMA 依然保留了‘消息传递’的语义，存在上下文切换和协议封装开销。通过 **CXL 实现 Load/Store 级别的内存传输**，结合 **Pasha 的动态分片策略**，可以将跨节点数据同步从‘网络层’降维到‘内存层’。这不仅是带宽的提升，更是从微秒级到纳秒级的量级跨越……”

RDMA Remote Direct Memory Access (远程直接内存访问)

一台机器可以直接读写另一台机器的内存，而不需要对方 CPU 参与。

代码块

```
1 User memory → NIC → 网络 → NIC → User memory
```

传统 TCP/IP 网络流程：要通过cpu，同时内容经过多次拷贝 会有**memory copy overhead**

代码块

事务是数据库管理系统（DBMS）中用于保证数据操作可靠性与一致性的核心机制，是一组不可分割的数据库操作序列，要么全部执行成功，要么全部不执行。

一个 core 同时真正推进的事务数是非常有限的，通常按线程数来算。

- 一个 core 同一瞬间主要执行一个线程
- 如果开启 SMT / Hyper-Threading，可能一个 core 暴露两个逻辑线程
- 但本质上执行资源还是共享的，不等于真有两个完整 core

从数据库系统实现看

- 一个 worker thread 处理一个事务
- 一个 core 跑一个 worker thread
- 所以近似地说：一个 core 同时主跑一个事务

一个 core 在某一时刻通常只实际执行一个事务的代码路径，但系统整体上可以让很多事务处于“活跃/等待/排队”状态。

冲突

两个事务访问同一条数据时：

- 读 + 读：通常没问题
- 读 + 写：可能冲突
- 写 + 写：肯定冲突

锁： S 锁 / 读锁：可以共享 X 锁 / 写锁：独占

latch： 保护内部数据结构的短期互斥

phantom（幻读）： 一个事务明明刚扫过某个范围，结果另一个事务往这个范围里插入了新记录，于是第一个事务再看这个范围时，凭空多出一条“幽灵记录”。

这条“突然冒出来”的记录，就叫 **phantom**

Lock： 面向事务隔离的逻辑锁

lock 是为了保证事务隔离性而加的锁

它保护的并不是“代码结构别写坏”，而是：

“事务之间不要互相看到不该看的数据，不要产生冲突结果”

2PL 要求每个事务分两个阶段进行加锁和解锁：

1. **增长阶段 (Growing Phase)：** 事务可以获得任何类型的锁（读锁/写锁），但**不能释放**任何锁。
2. **缩减阶段 (Shrinking Phase)：** 事务可以释放锁，但**不能再获得**任何新锁。

普通 2PL 只锁具体 key，不锁 key 之间的空隙（gap）。

NO_WAIT 拿不到锁就不等，直接 abort。

Tigon 用它来避免复杂的跨主机等待和死锁。论文明确写了：锁请求被拒绝时，按 NO_WAIT 直接 abort。

2PC：两阶段提交用于保证分布式事务“要么全提交，要么全回滚”。

Tigon 想减少对它的依赖，因为 2PC 会带来：

- 跨主机消息往返
- 等最慢参与者
- 更长的锁持有时间

Strict 2PL (S2PL)：

- **规则**：事务持有的所有**排他锁 (Exclusive Locks)** 必须在事务提交 (Commit) 或回滚 (Abort) 后才能释放。

TPC-C 是一个经典的 **OLTP (联机事务处理) 基准测试**

它模拟的是一个“批发商管理系统”的业务场景。

里面有：

warehouse (仓库) district (地区) customer (客户) order (订单) stock (库存) item (商品)

因为它不是只测简单读写，而是：

- 有多张表
- 有事务逻辑
- 有读有写
- 有跨分区访问
- 有热点与冲突
- 比较接近真实商业事务处理

所以很多数据库论文用 TPC-C 看：

吞吐、延迟、锁冲突、分区事务比例、日志开销、跨节点协调成本

YCSB = Yahoo! Cloud Serving Benchmark

它是另一个非常常见的 benchmark，但和 TPC-C 风格不一样。

YCSB 更像是：

- 面向 key-value / NoSQL / 云数据库
- 操作比较简单
- 多是单条记录的读写

- 用来测试吞吐、延迟、热点、负载比例

atomic 操作

这个操作要么完整发生，要么不发生；中间不会被别人看到“做了一半”。

CC = Concurrency Control（并发控制）

next-key locking

- 锁住当前记录
- 还锁住记录之间的“间隙（gap）”

它主要是为了解决：

幻读（phantom read）

假设事务 T1 查询：SELECT * FROM accounts WHERE balance > 100;

它第一次查到 5 条。这时另一个事务 T2 插入一条新记录，余额 200，并提交。如果 T1 再查一次，发现变成 6 条。这多出来的一条“幽灵记录”就是 phantom。

普通的行锁只能锁已有的行，不能阻止新插进来的行，所以为解决幻读，还需要锁住范围

logging protocols

log: (1) **UNDO logging**: 记录“改之前是什么”，用于回滚

(2) **REDO logging** 记录“改之后应该是什么”，用于重做

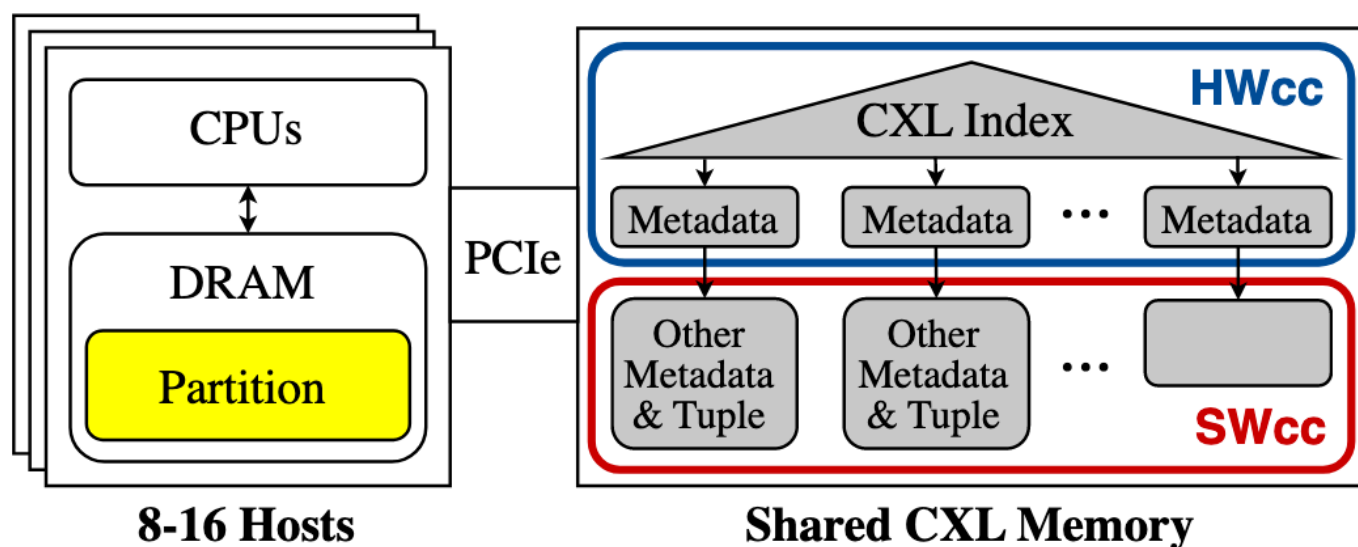


Figure 1: Tigon on a CXL pod.

HWcc: 这部分 CXL 内存具备**硬件 cache coherence 支持**

这部分内存，硬件会帮你更强力地处理多主机之间的缓存一致性问题。

所以对它做原子操作、锁操作，会更自然、更稳。

SWcc: 这部分 CXL 内存**不具备硬件 cache coherence**，所以要靠**软件协议**来维持一致性

这块内存物理上当然还是 CXL memory，

只是硬件不替你把跨主机缓存一致性全包了，

所以数据库自己写了一套“谁的缓存还能信、谁要先刷新”的规则。

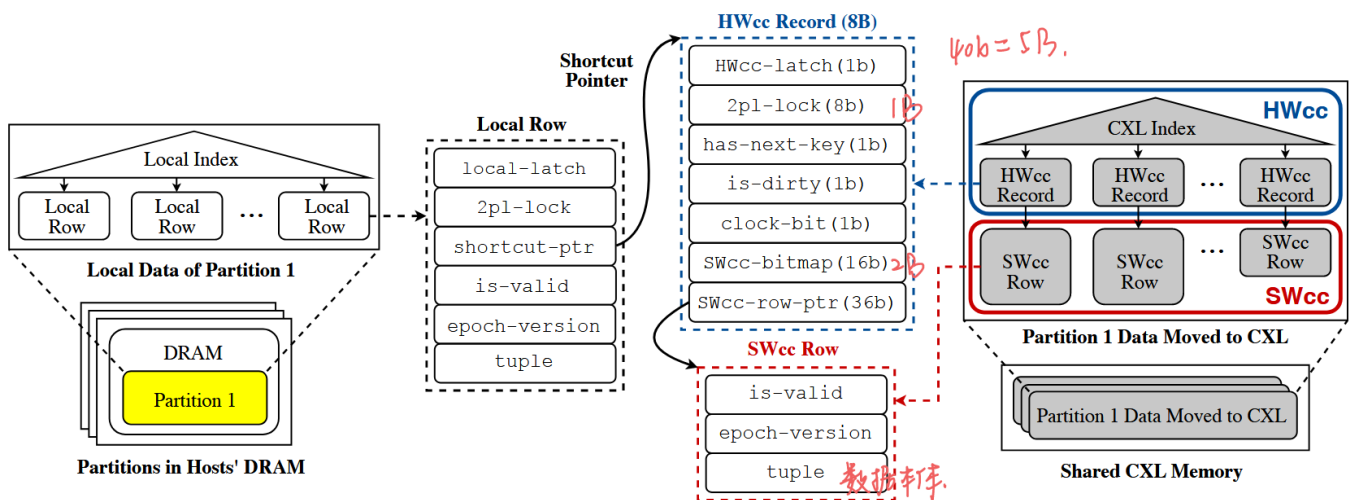
HWcc 和 SWcc 的区别

- 两者都在 CXL memory 里
- 区别不在于“一个真一个假”
- 而在于：
 - **HWcc**: 有硬件 cache coherence 支持
 - **SWcc**: 没有硬件一致性支持，要靠软件协议维护一致性

所以：

- 小而同步频繁的元数据放 HWcc
- 大块 tuple 数据放 SWcc

论文说，SWcc-bitmap 为每个 host 分配 1 个 bit，Tigon 最多支持 16 个 host。这个 bitmap 用来跟踪：哪些 host 可以对某条 SWcc row 使用 cacheable loads



local DRAM. Data is stored in the form of local row.

Each local row contains the following:

- Latch for mutual exclusion (local-latch)
- Locking metadata for 2PL (2pl-lock)
保护事务之间的读写逻辑正确性。
- Pointer to the tuple in CXL memory (shortcut-ptr)

`shortcut_ptr` 是否为 NULL?

- NULL: 说明 tuple 根本没搬到 CXL, 直接本地访问
- 非 NULL: 说明 tuple 已经搬到 CXL, 要进一步判断怎么读
- Flag indicating whether the tuple is valid (is-valid)
- Field used for logging (epoch-version)
- Database tuple (tuple)

Data and metadata in CXL memory.

Each **SWcc row** contains:

- is-valid
- epoch-version
- and tuple fields

Each **HWcc record** occupies **8 bytes** and contains the following fields:

- 1-bit exclusive latch for mutual exclusion (HWcc-latch)
保护共享数据结构和一致性元数据的安全访问
- 8-bit locking metadata for 2PL (2pl-lock)
- 1-bit flag supporting next-key locking (has-next-key)
- 1-bit flag indicating whether the tuple has been modified since being moved to CXL memory (is-dirty)

它允许 owner host 直接从本地 DRAM 读取干净的 tuple, 而不是去 CXL memory 读, 从而显著降低 CXL memory 带宽使用

`is-dirty = 0`: 这条 tuple 自从搬到 CXL 以后, 还没有被修改过。那就意味着: CXL 里的那份数据内容和 owner 本地 DRAM 里的那份 local row 中的 tuple 内容一致。所以 owner host 完全没必要绕远路去读 CXL, 直接读自己本地 DRAM 里的那份“干净副本”。

`is-dirty = 1` 表示: 这条 tuple 搬到 CXL 后已经被改过了。owner 本地 DRAM 中那份旧副本可能已经不新鲜了。只能去看 CXL 中的最新版本。

- 1-bit supporting the CLOCK policy (clock-bit)
- 16-bit bitmap for software cache coherence (SWcc-bitmap)

cacheable loads 之所以更快, 是因为它允许数据进入并复用 CPU cache, 所以后续访问有机会直接命中本地 cache, 而不用每次都重新走更远的 CXL 路径。论文也明确说, Tigon 比

`NonTemporal` 快, 因为它 “can cache the tuples, which is much faster than non-temporal reads”。

记录哪些 host 当前可以安全地对某条 SWcc row 使用 cacheable loads

bit=0 先 flush 相关 cachelines, 再从 CXL 重新读入, 并把自己的 bit 置 1

一个 host 一旦更新了某条 SWcc row, 它会把其他 host 的 bit 全部清掉。

- 36-bit pointer to SWcc row (SWcc-row-ptr)

Data movement policy

“If a host’s HWcc memory usage exceeds a predefined threshold, it will move a tuple it owns from CXL memory back to local DRAM.” (Huang 等, p. 7)

当 HWcc memory 使用超过阈值时, owner host 会把某些 tuple 从 CXL 再搬回本地 DRAM。

理想上, 应该优先搬走那些“未来不太可能再被非 owner host 访问”的 tuple

从这个目标看, LRU **Least Recently Used (LRU)** 很自然, 但 LRU 元数据维护太贵

所以 Tigon 用更便宜的 CLOCK

每个在 CXL 中的 tuple 都有一个 `clock-bit`

这个 bit 存在对应的 **HWcc record** 里

只要 **non-owner host** 访问过这个 tuple, 就把 `clock-bit` 设为 1

owner host 维护一个 circular cursor (循环游标) 扫描这些 tuple

扫描时:

- 如果 `clock-bit = 1`: 先清零, 跳过它
- 如果 `clock-bit = 0`: 把它搬回本地 DRAM (因为不常用)

Algorithm 1: Read or write a tuple in Tigon.

Input: k – key of the tuple to be read or written

```
1 if  $k$  belongs to local partition then
2    $localRow \leftarrow \text{Get}(k, \text{local index});$ 
3   Acquire local-latch for  $localRow$ ;
4   if shortcut-ptr of  $localRow$  is NULL then
5      $\text{ReadOrWrite}(localRow);$ 
6   else
7      $hwccRecord \leftarrow \text{shortcut-}ptr \text{ of } localRow;$ 
8     Acquire HWcc-latch for  $hwccRecord$ ;
9      $\text{ReadOrWrite}(hwccRecord);$ 
10    Release HWcc-latch for  $hwccRecord$ ;
11  Release local-latch for  $local \text{ row}$ ;
12 else
13    $hwccRecord \leftarrow \text{Get}(k, \text{CXL index});$ 
14   while  $hwccRecord$  is NULL do
15     Ask the owner host to move  $k$  to CXL memory;
16     if the host replies  $k$  does not exist then
17       return NOT_EXIST;
18     else
19        $hwccRecord \leftarrow \text{Get}(k, \text{CXL index});$ 
20   Acquire HWcc-latch for  $hwccRecord$ ;
21    $\text{ReadOrWrite}(hwccRecord);$ 
22   Release HWcc-latch for  $hwccRecord$ ;
```

Software cache coherence.

2PL 锁已有记录；next-key locking 进一步锁住范围/gap，防止 phantom。

next-key locking. Records are accessed via an ordered index (such as a B+-tree), and for inserts, deletes, scans, and point queries on non-existing keys, locks are acquired for both the key or key range and the next key in order.

CXL index 里的每个 tuple 都多了一个 `has-next-key` 标志，表示：

它在 CXL index 里的 next key，是否也正好是 local index 里的 next key。

has-next-key = 1 CXL index 中这个 tuple 的 next key，和 local index 中的 next key 是同一个。

has-next-key = 0 表示：CXL index 里的 next key 不能代表 local index 里的真实 next key。

Logging and Recovery

为了避免传统 2PC，Tigon 要保证：执行这个事务的 host，自己就能完成并记录这个事务涉及的 tuple 修改。让事务提交尽量本地化，不要最后再拉着多个 host 一起做提交协调。

epoch 一个时间批次 / 提交批次。

论文说：

- Tigon 给每个 transaction 关联一个 epoch number
- 每个 tuple 也带一个 epoch number 和 version number，也就是 Figure 3 里的 `epoch-version`
- 全局有一个共享的 epoch `E`，周期性递增，例如原型里每 10ms 增加一次
- version 只在同一个 epoch 内有顺序意义

崩溃恢复时，只要知道哪个日志记录拥有“最大 epoch + 对应 epoch 内最大 version”，我就知道这条 tuple 的最新值应该是什么。

Tigon 有两类线程：

- **transaction worker threads** 负责：执行事务 拿锁 修改 tuple 生成要写入日志的内容
- **logger threads** 负责：真正把日志刷到持久化设备 参与 group commit

group commit 就是：

不是每个事务一完成就立刻单独刷盘，而是把同一批次的一组事务日志攒起来，一起提交。

group commit 要等到当前 epoch 的批量提交时机，所以它擅长摊薄开销、提吞吐，但不一定让单个事务“做完立刻最早落盘”。

流程：

1. **Tigon 想减少 2PC**，所以要让一个 host 尽量自己完成并记录事务涉及的 tuple 修改。
2. 它用 **global epoch E** 把事务分成提交批次。
3. 每个 tuple 有 **epoch-version**，表示最近修改它的批次和该批次内版本顺序。
4. logger threads 按 **epoch-based group commit** 持久化日志。
5. recovery 时，从 durable 日志里选每条 tuple 的 **最高 epoch + 该 epoch 内最高 version** 作为最新值。

Evaluation

Tigon / Sundial+ / DS2PL+跑在 **CXL pod emulation** 平台上。

CXL pod emulation：

- 1 台机器
- Intel Xeon Platinum 8568Y+

- 512 GB local DRAM
- 128 GB CXL 1.1 memory device
- local DRAM 和 CXL memory 都是 DDR5-4800
- CXL device 单通道，通过 PCIe 5.0 x8 接到 CPU
- CXL latency 是本地 DRAM 的 1.6 倍 (259ns vs 159ns)
- CXL bandwidth 只有本地 DRAM 的 13% (31.8 GB/s vs 238.3 GB/s)
- 还有 Samsung Datacenter SSD 和 100 Gbps Mellanox BlueField-2 ConnectX-6 NIC

Motor platform

- 4 台机器
- 每台 2× Intel Xeon Platinum 8380
- 256 GB local DRAM
- 25 Gbps Mellanox ConnectX-6 NIC
- 1 台 compute server + 3 台 memory servers
- memory servers 还做 3-way replication 以保证 durability
- CPU core 数也尽量配到和 Tigon/其他 baseline 一致 (40 cores)

Evaluation: Baselines

Existing partition-based distributed in-memory DB

- Sundial¹ - optimistic concurrency control (OCC)
- DS2PL - pessimistic concurrency control (2PL)

Improved baselines

- Improvements
 1. Replace network transport with CXL message queues
 2. Repurpose a network thread for transaction execution
- Improved baselines
 - **Sundial+** - Sundial with improvements 1 & 2
 - **DS2PL+** - DS2PL with improvements 1 & 2

RDMA-based shared-memory DB - Motor²

